

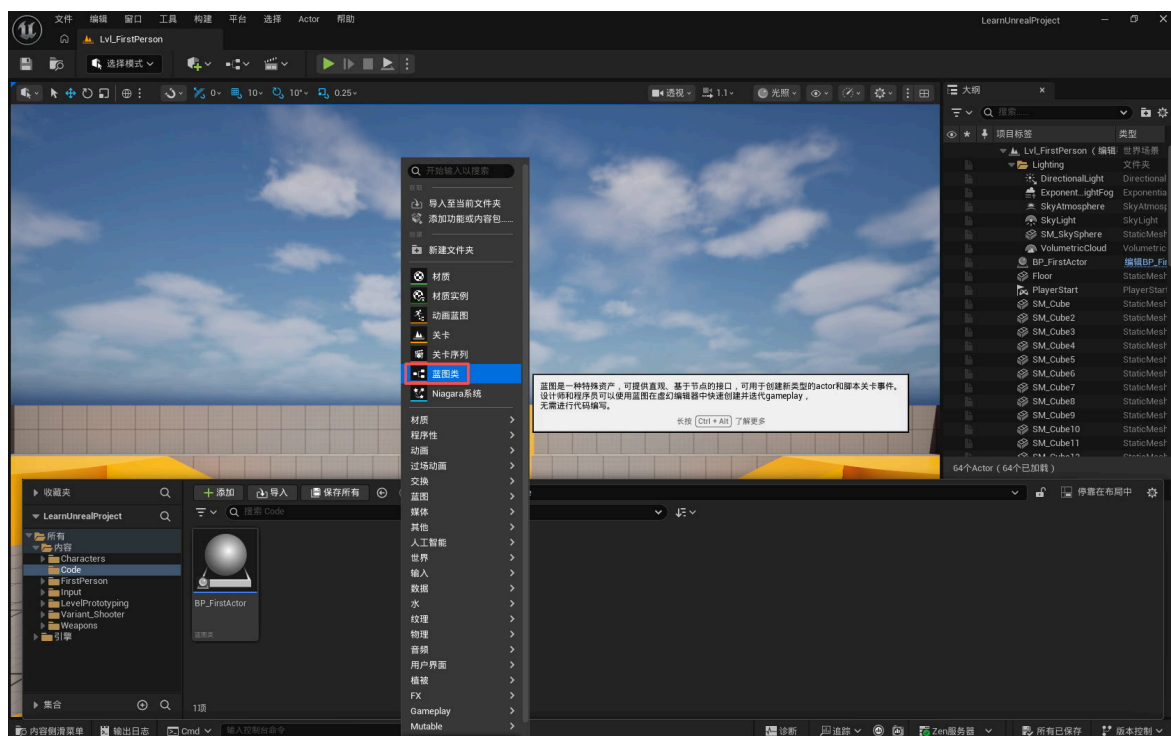
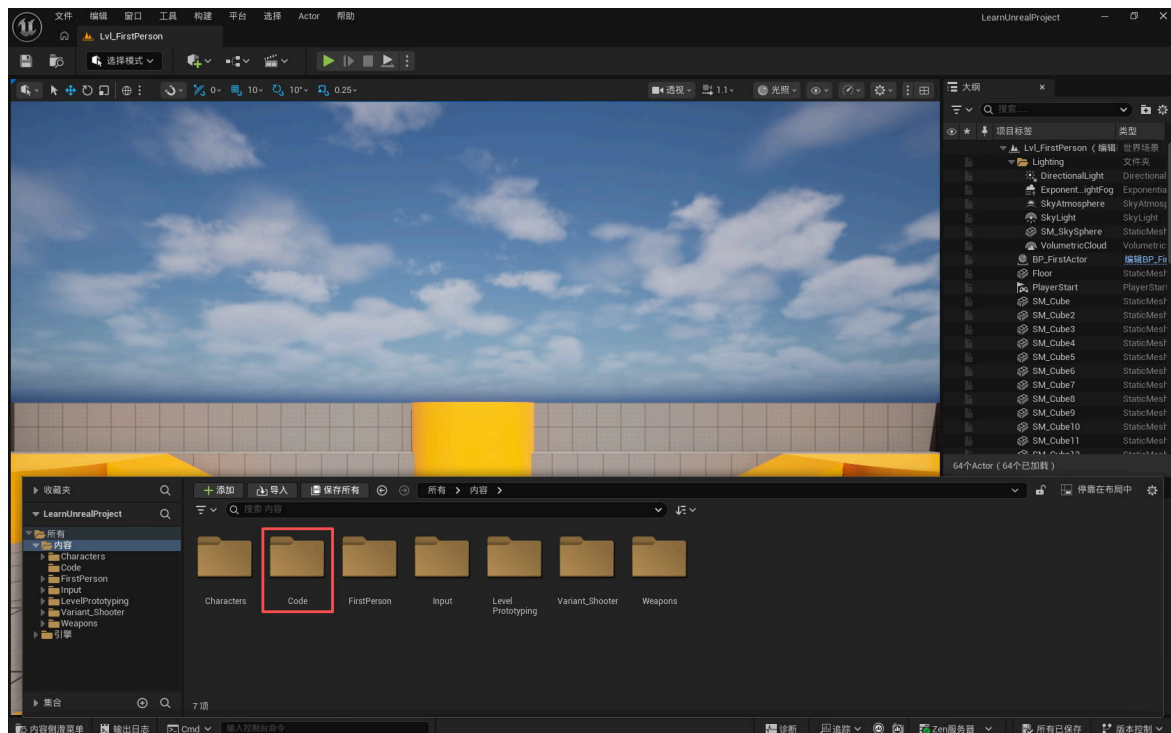
蓝图编程基础

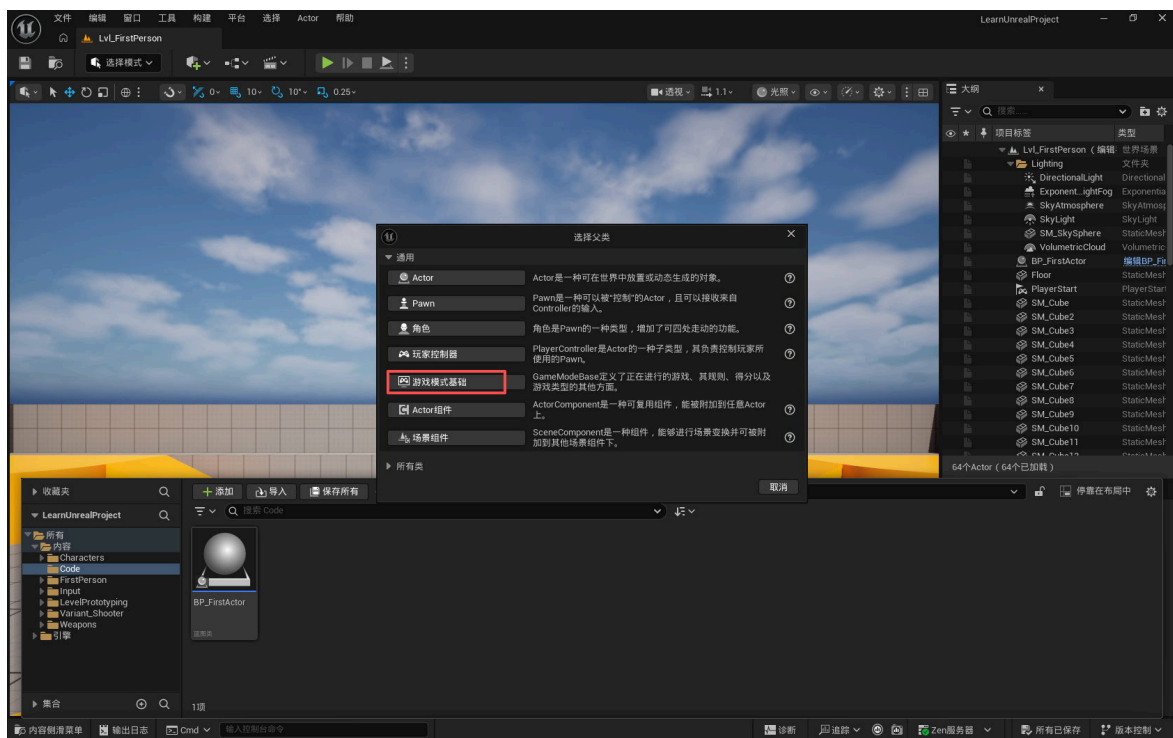
一、新建游戏模板基础和角色

1. 新增游戏模板基础

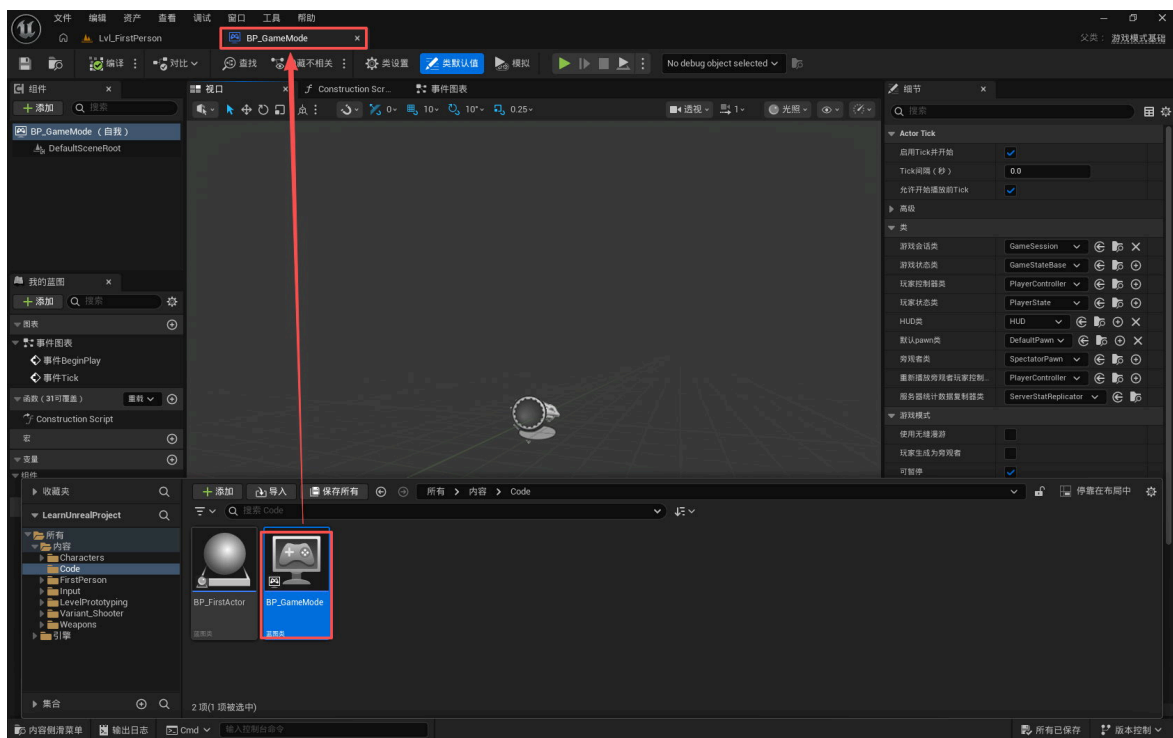
由于蓝图基础中的第一人称关卡模板自带可移动人物等设置，为方便后续从零开始学习，新建关卡和角色

在内容侧滑菜单中的Code目录下（蓝图基础中新增目录），右键新增蓝图类- **游戏模式基础**，命名为BP_GameMode（Blueprint 蓝图）



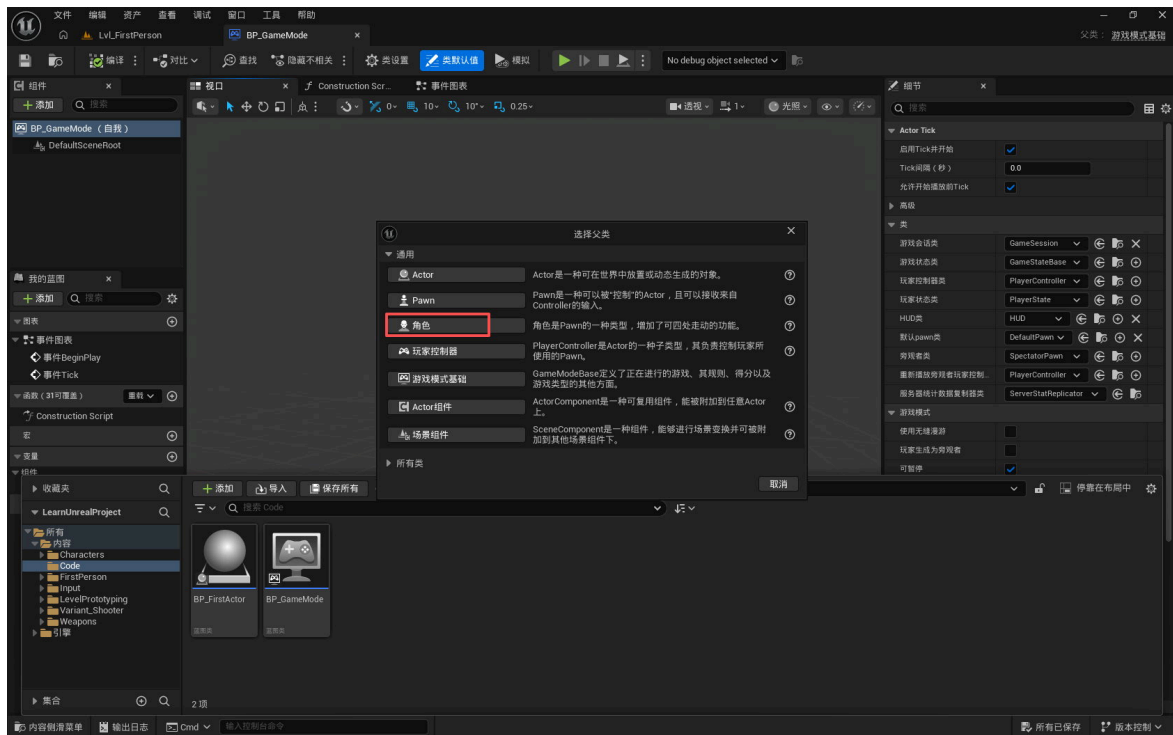


双击游戏模式基础，进入相应界面

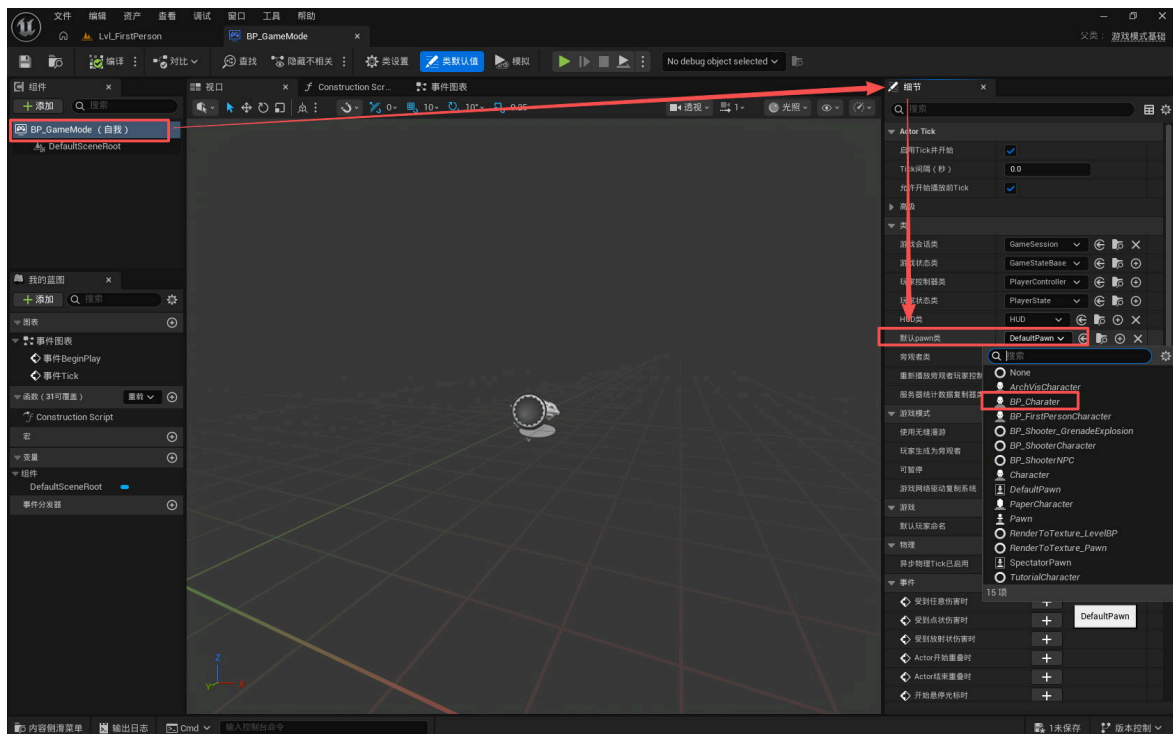


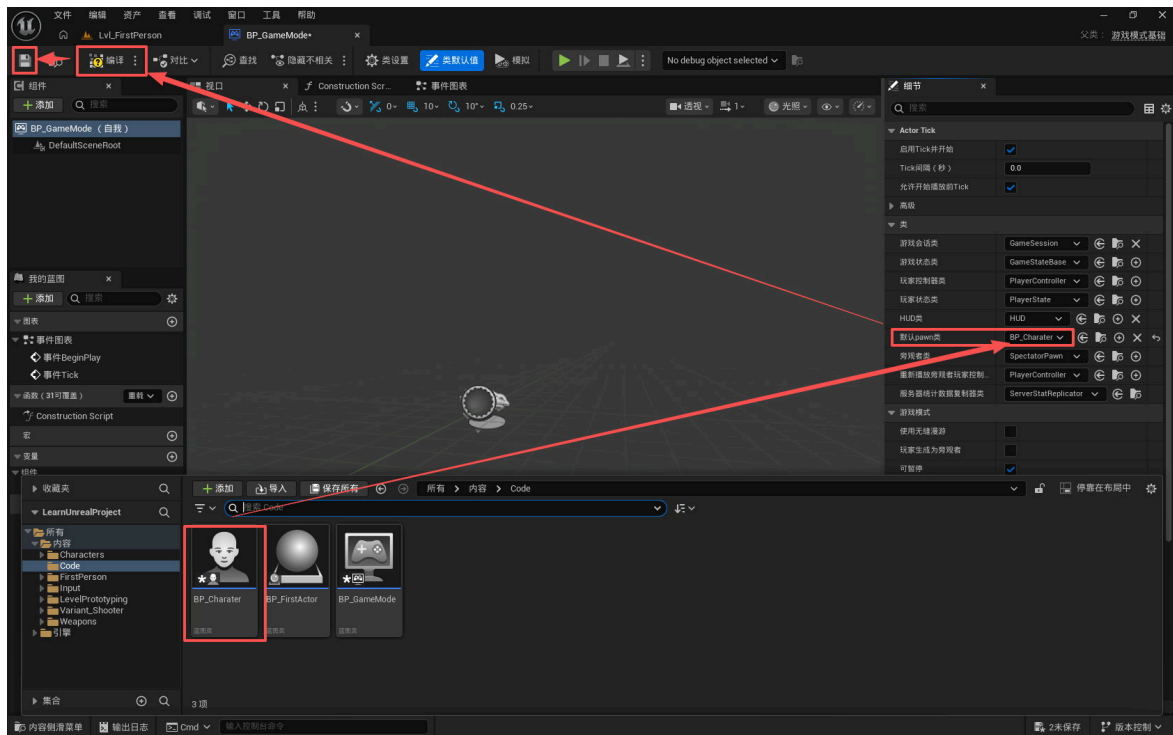
2. 新增角色

右键新增蓝图类-角色，创建用于编写的角色,命名为BP_Character

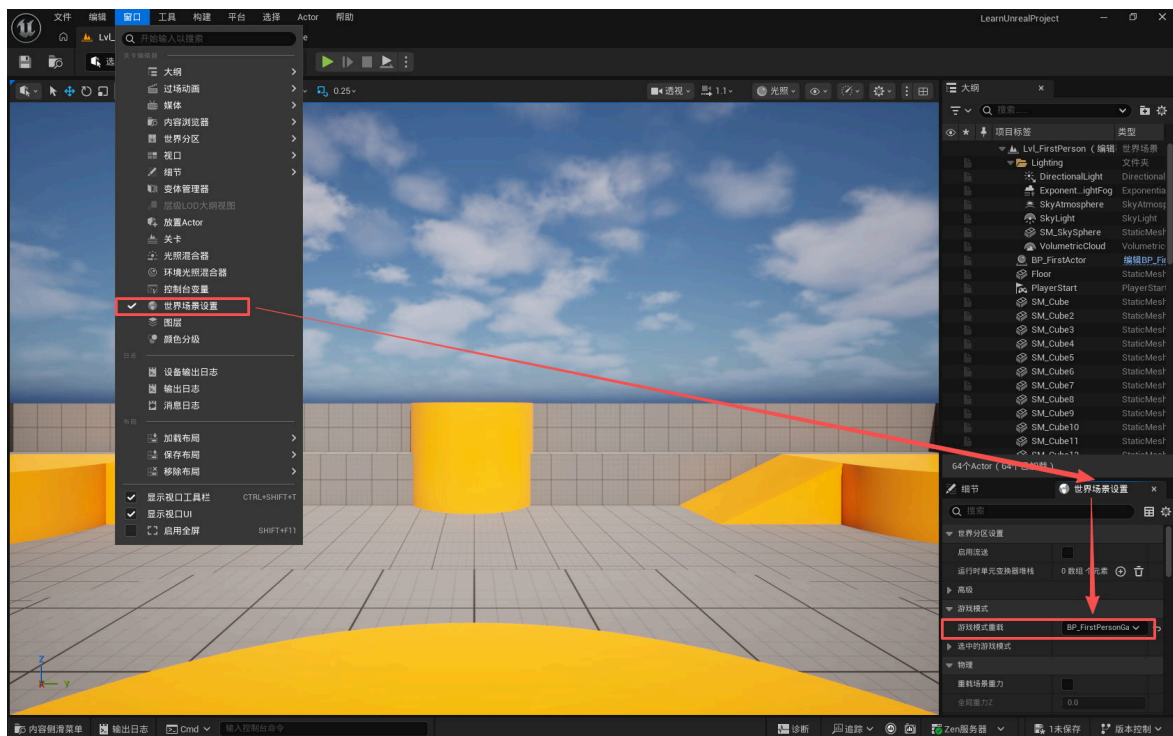


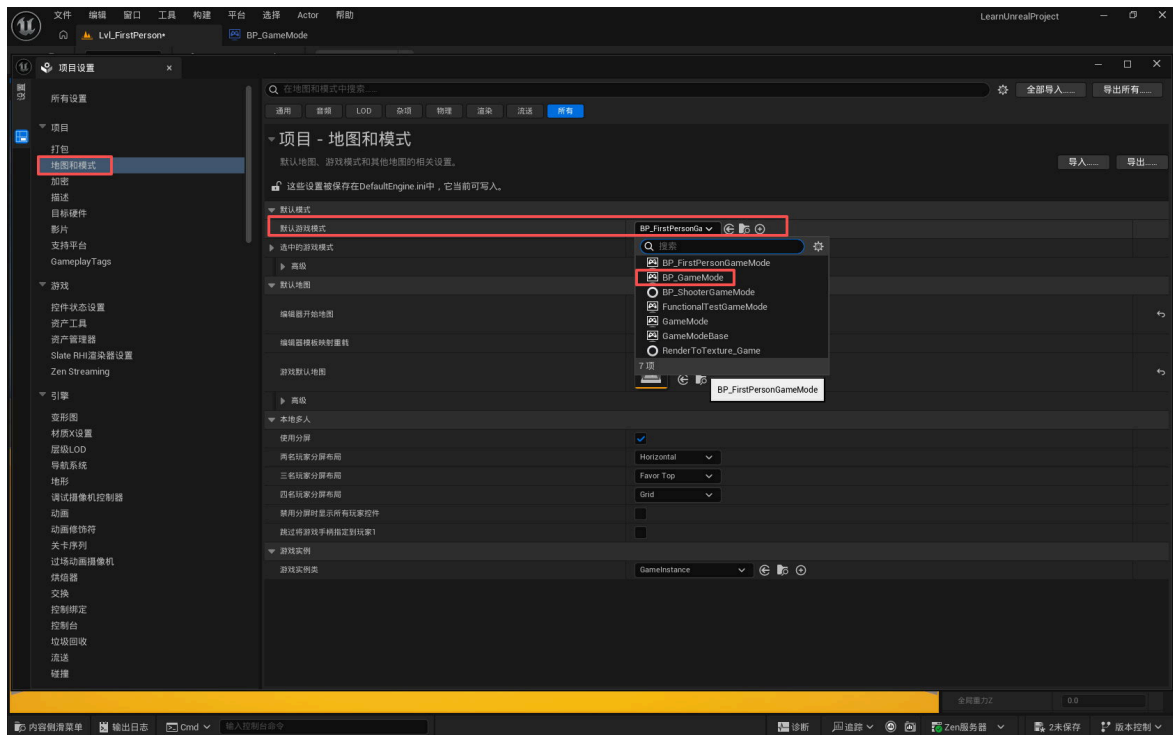
- 修改游戏模式基础的细节界面中默认Pawn类为新增的BP_Character
- 注意：如果误点击了别处，可以左键单击组件界面的BP_GameMode唤出细节界面
 - 修改完默认Pawn配置后，记得点击编译和保存





- 返回之前所在的FirstPerson关卡中，将新增的游戏模式基础设置到关卡中
- 在世界场景设置中，将**游戏模式重载**设置为Node或者刚刚新增的 **BP_GameMode**
 - 注意，如果右侧默认没有世界场景设置，可以在窗口中点击添加
- 在**编辑-项目设置**中的地图和模式，设置默认游戏模式为新增的**BP_GameMode**

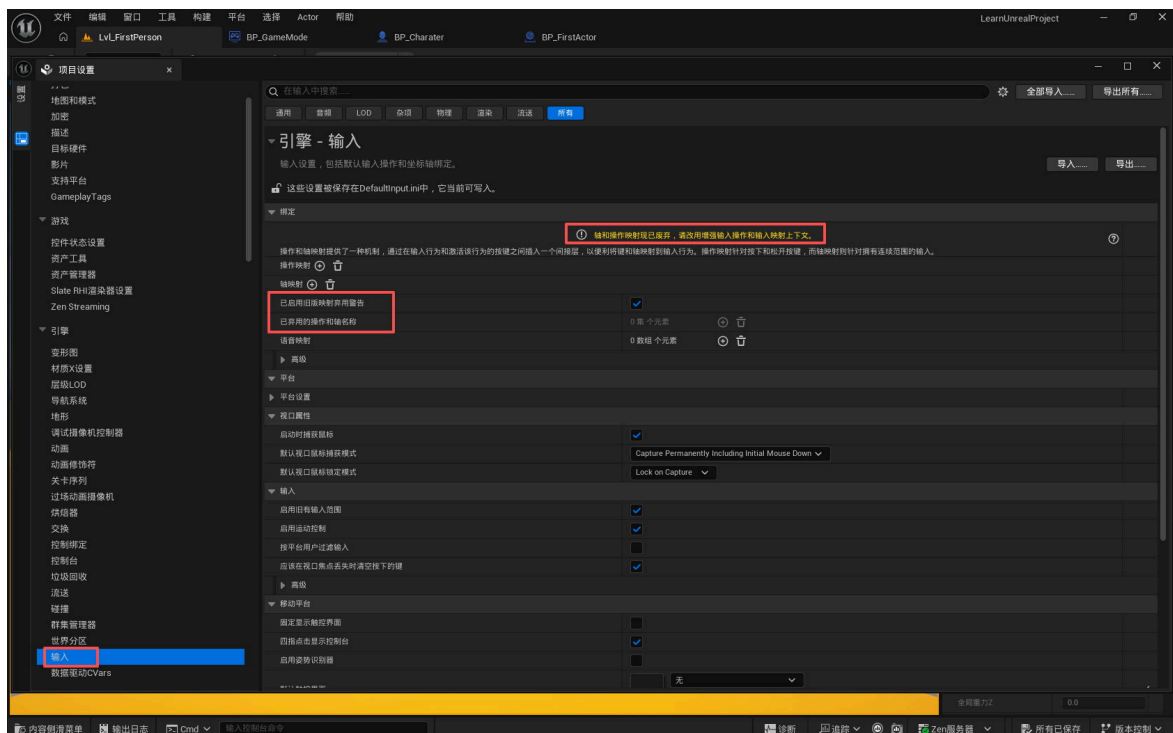




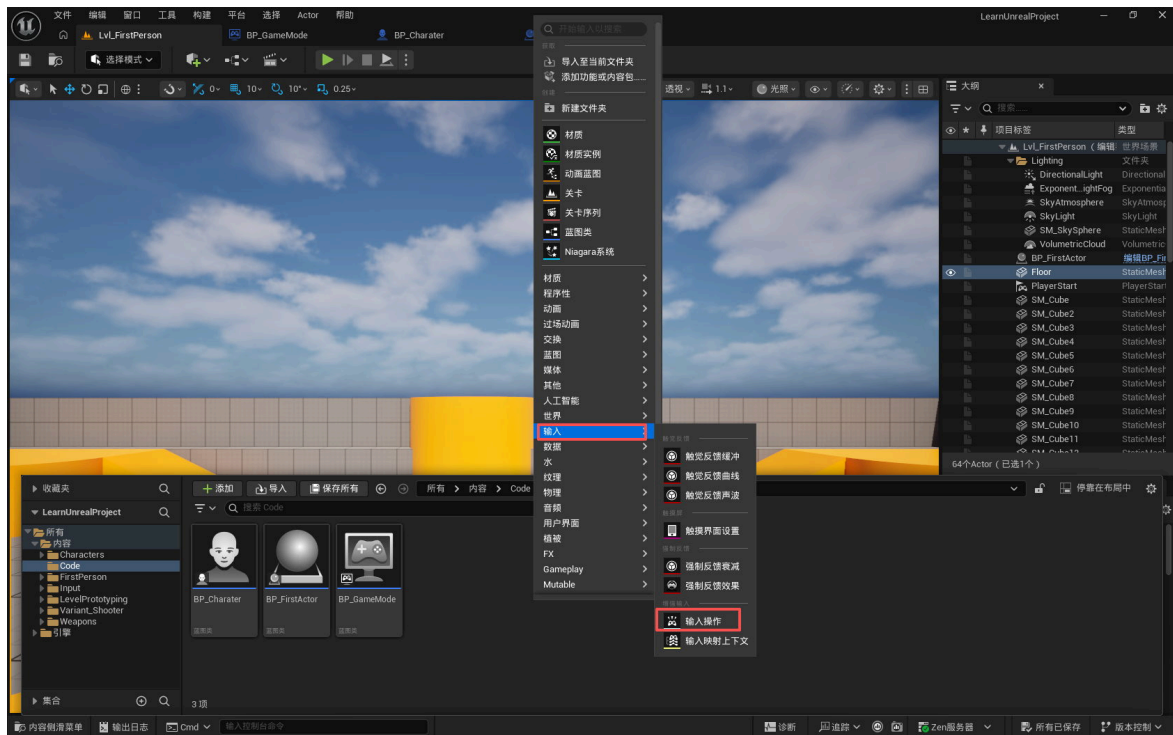
二、实现角色移动

1. 新增映射上下文

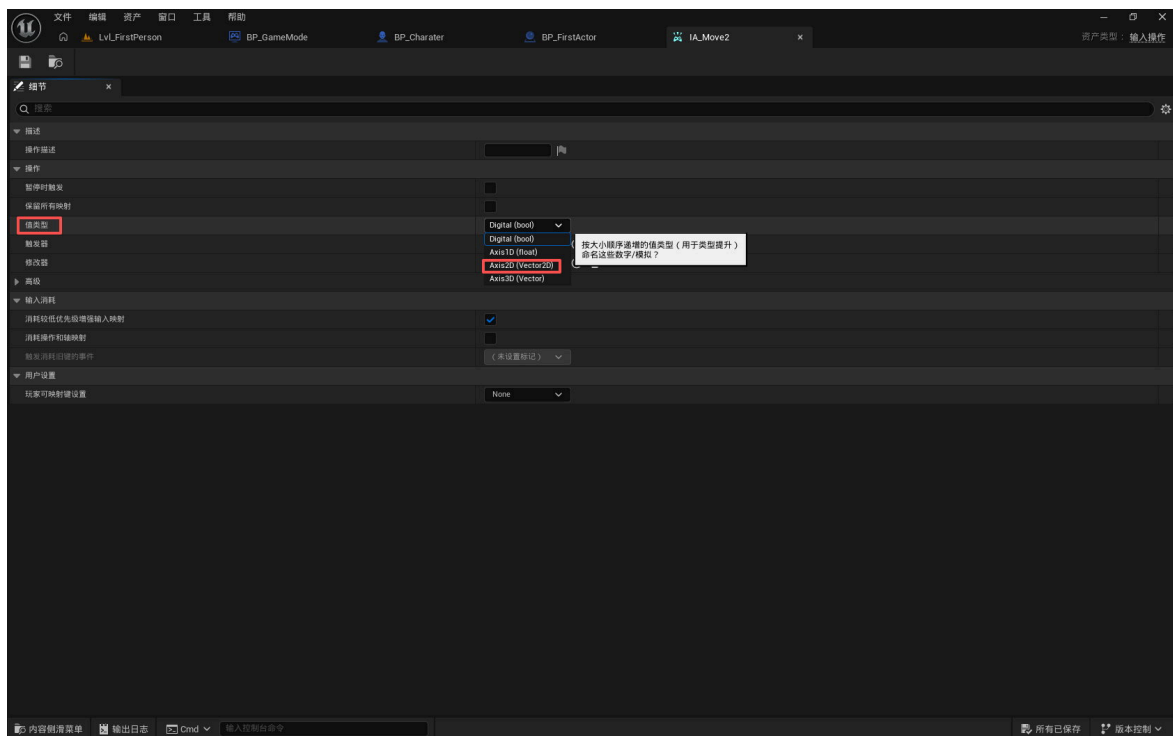
由于轴映射和键盘映射的输入方式已经被弃用，因此采用增强输入的方式实现复杂的输入功能



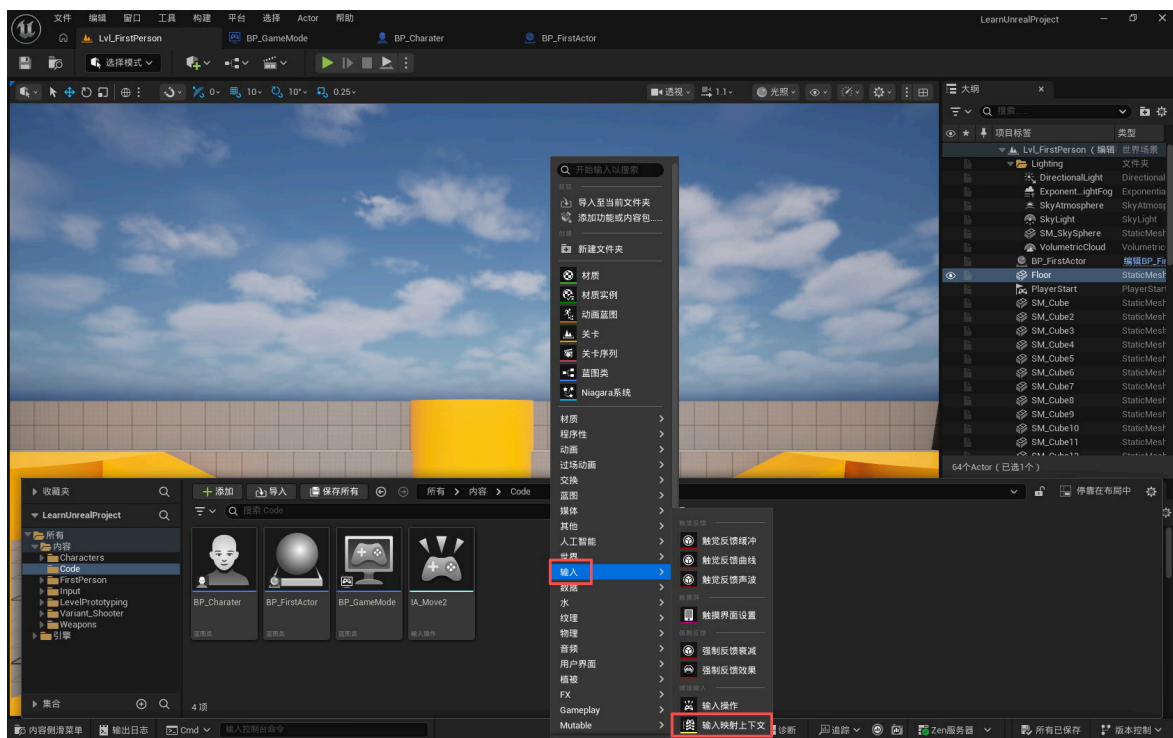
在内容侧滑菜单的Code目录下右键新增增强输入，为区分原有的移动输入命名，命名为IA_Move2（Input Action 输入操作/输入动作）



双击IA_Move2进入细节界面，修改值类型为Axis2D，因为前后左右移动为二维向量操作，修改后点击保存或Ctrl S

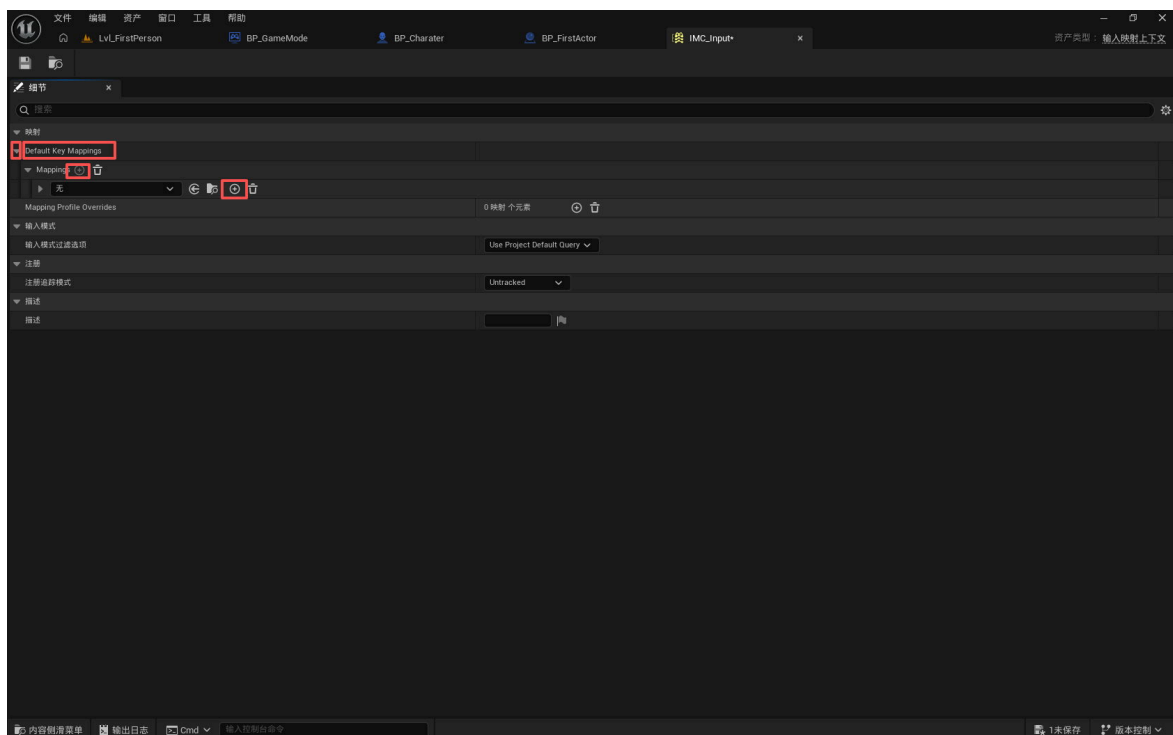


右键新增 **输入映射上下文**，命名为**IMC_Input**（Input Mapping Contexts 输入映射上下文）

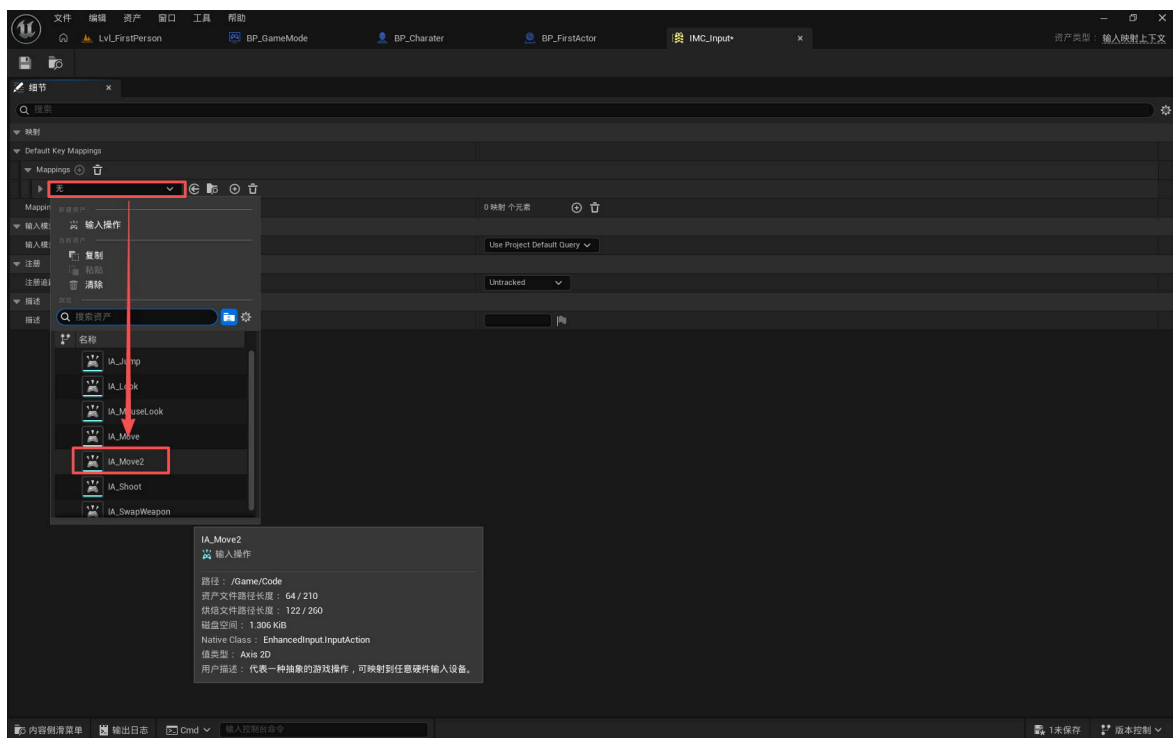


2. 设置映射上下文

双击IMC_Input进入细节界面，展开Default Key Mappings，在Mappings栏点击新增映射操作（+号）

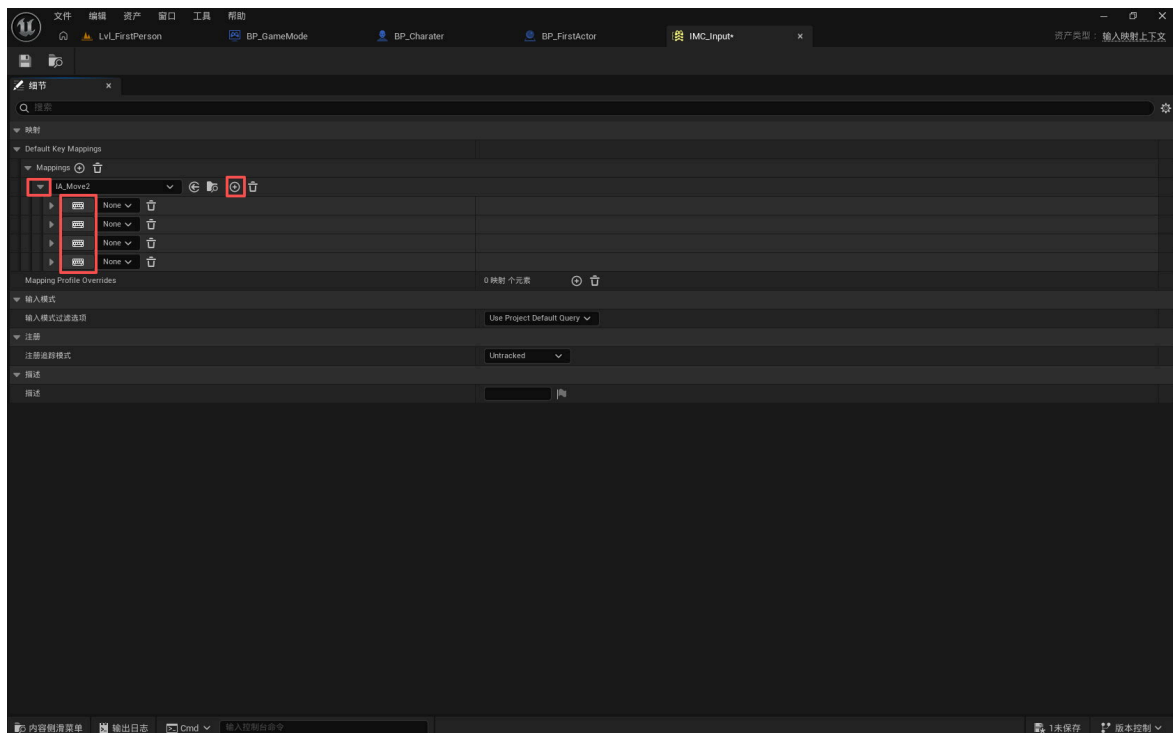


展开Mappings栏，选择新增的IA_Move2输入

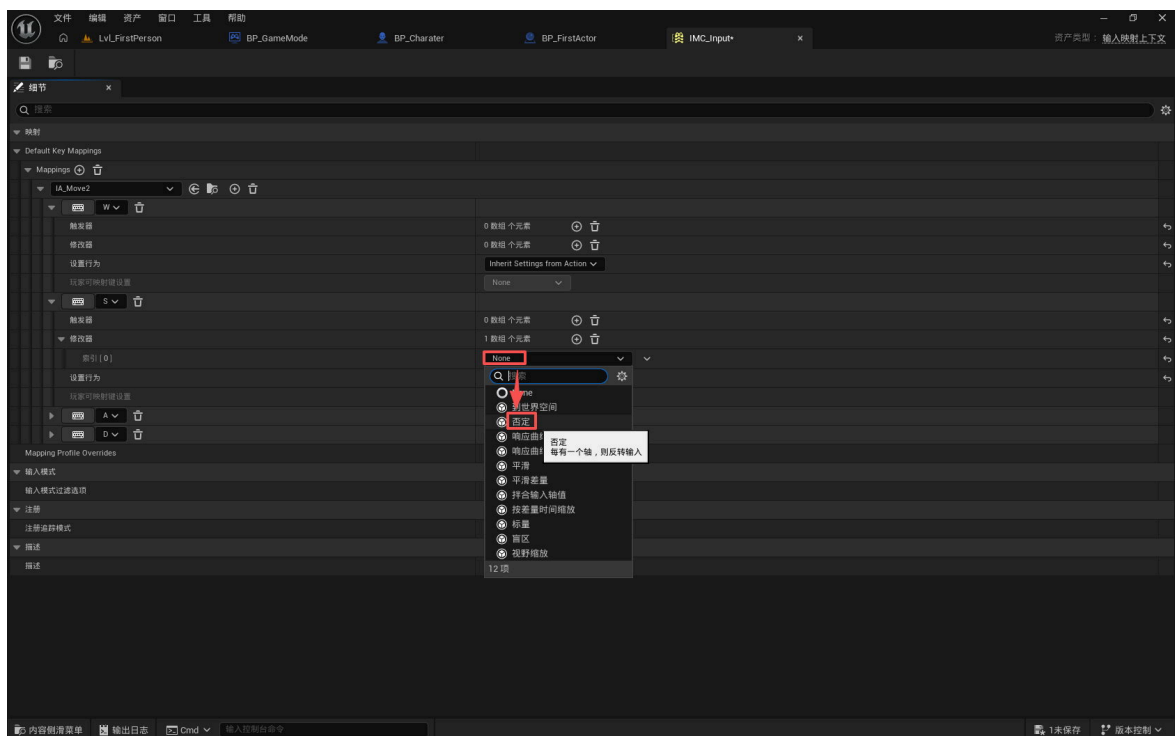
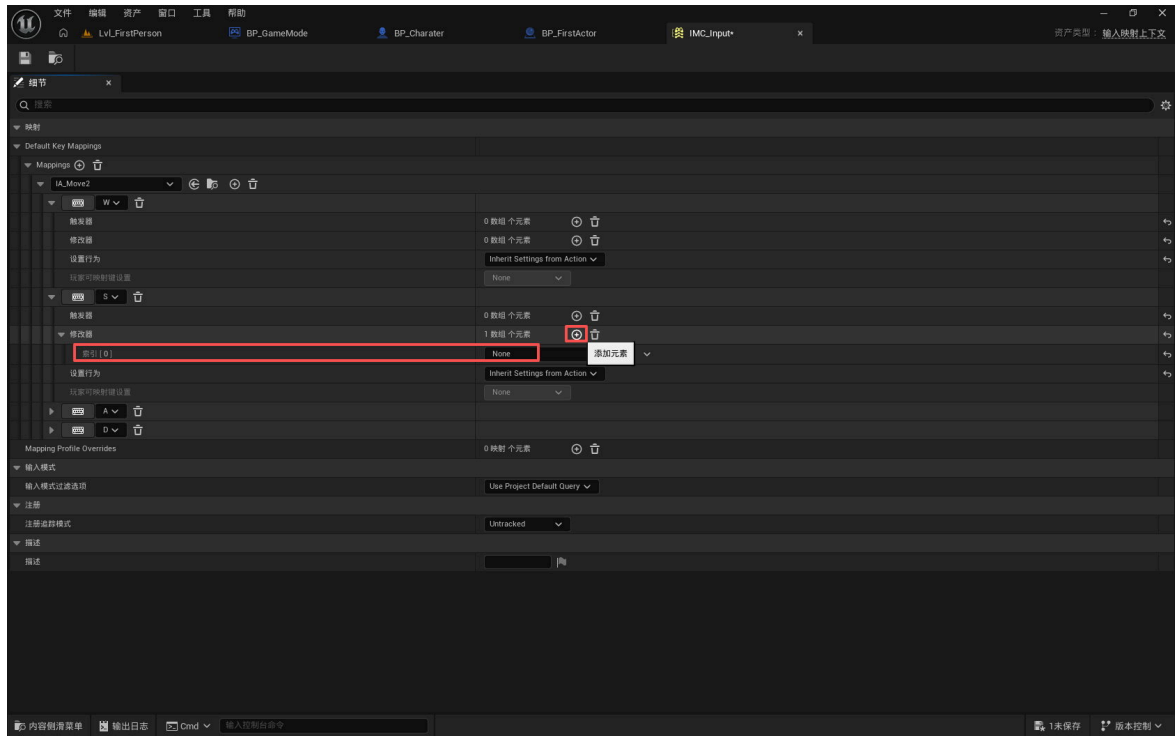
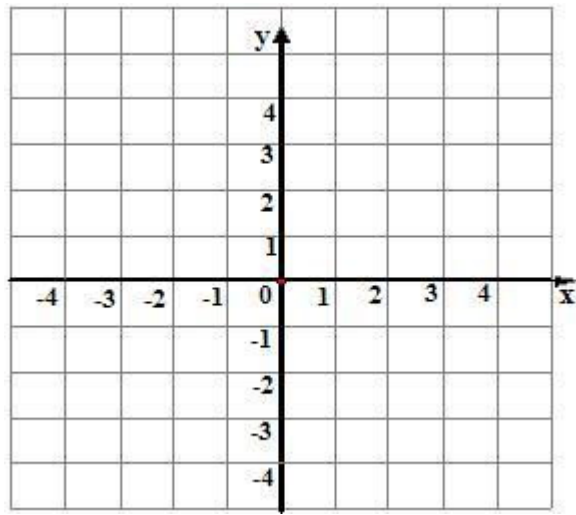


因为要设置**WASD**进行移动，所以点击四次Mappings子栏中的**将一个控制绑定到映射操作（+号）**

将设置好的子栏展开，在新增的四个映射栏中点击**选择键值（键盘按钮）**，依次按键盘上的**WASD**，完成设置

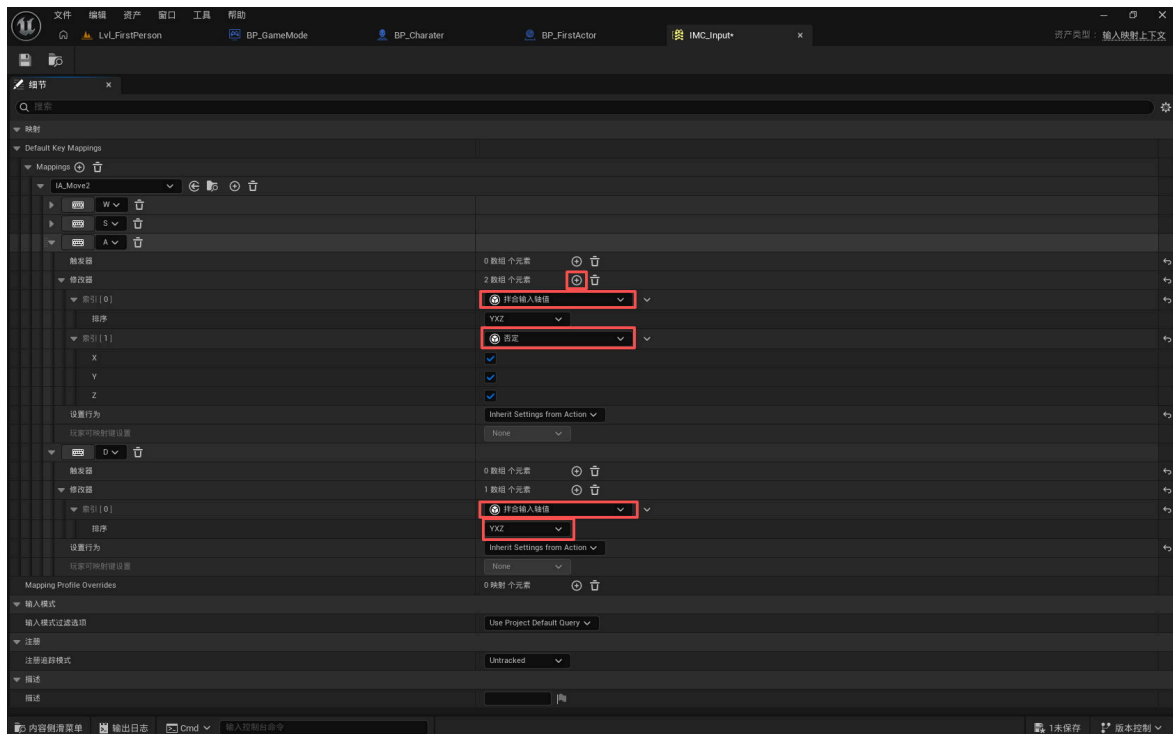


再次展开**WASD**键值栏，依据平面坐标系，当角色后退（向下移动）或者向左移动时，坐标所在位置为负数



因为键盘映射操作默认为X轴，AD与WS移动不在一个轴，所以需要额外设置拌合输入轴值，将AD操作的默认轴改为Y轴

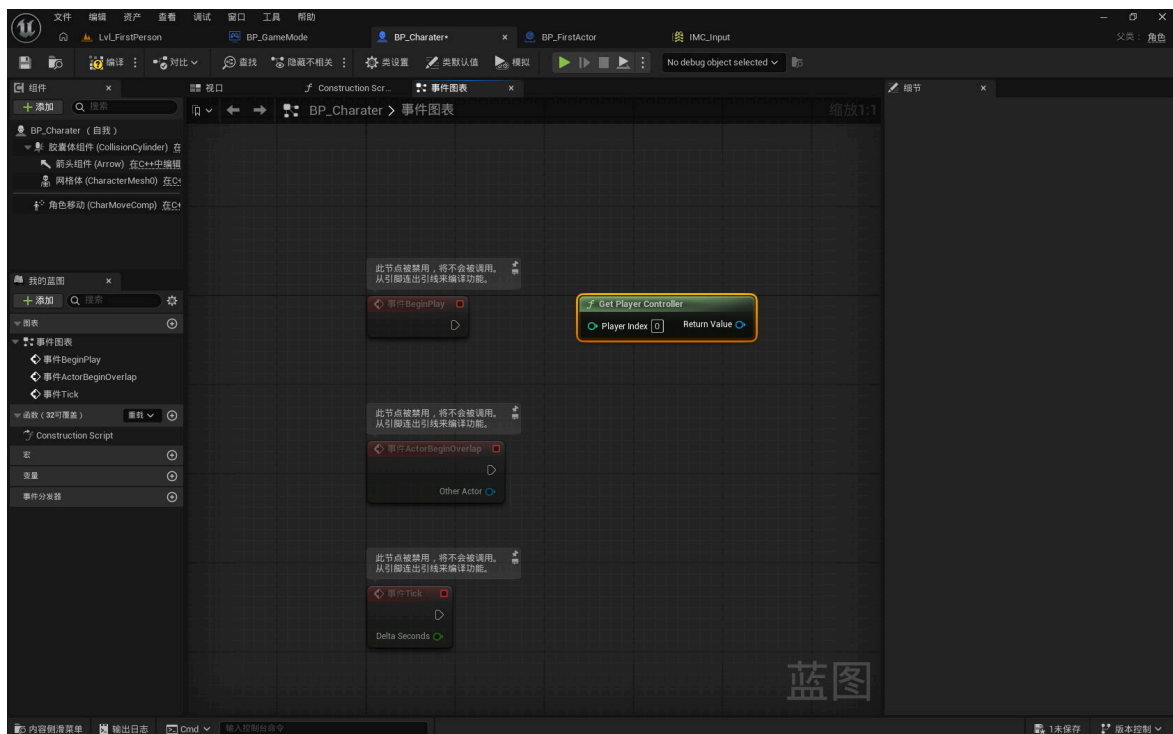
同时因为A向左移动，也要设置S同样的否定元素，因此需要新增两个元素



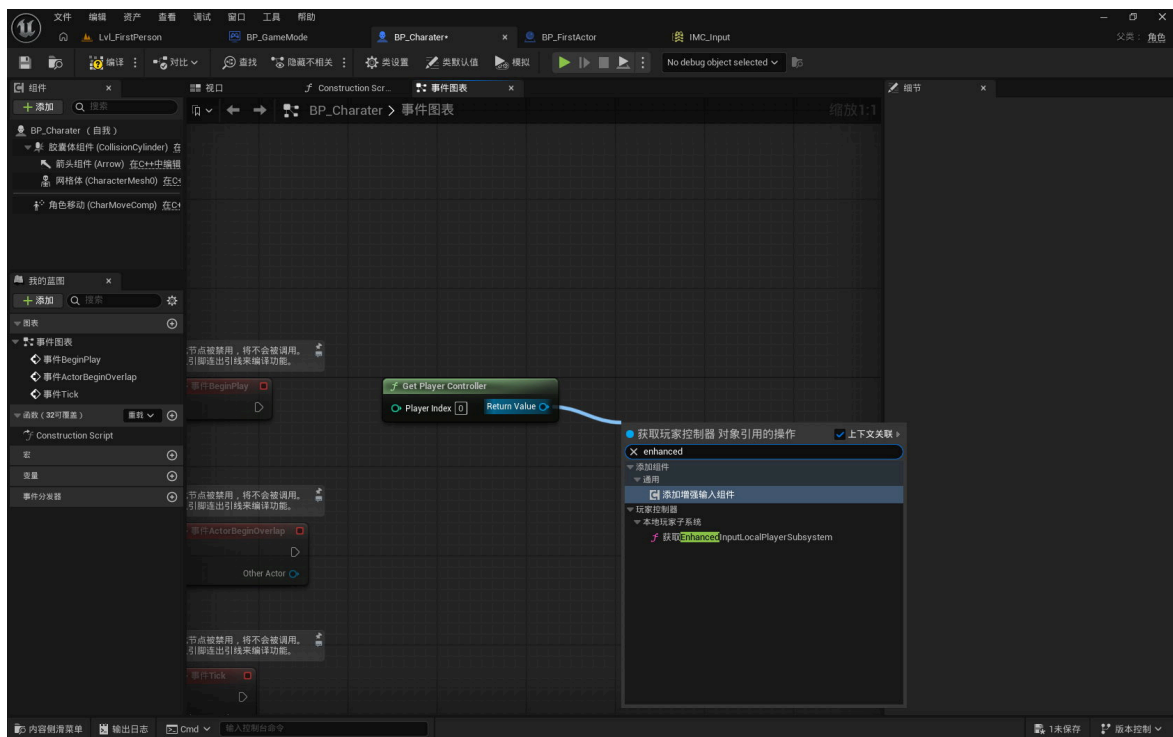
3. 激活映射上下文

点击保存或Ctrl S，返回BP_Character，进入事件图表界面，将设置好的增强输入激活

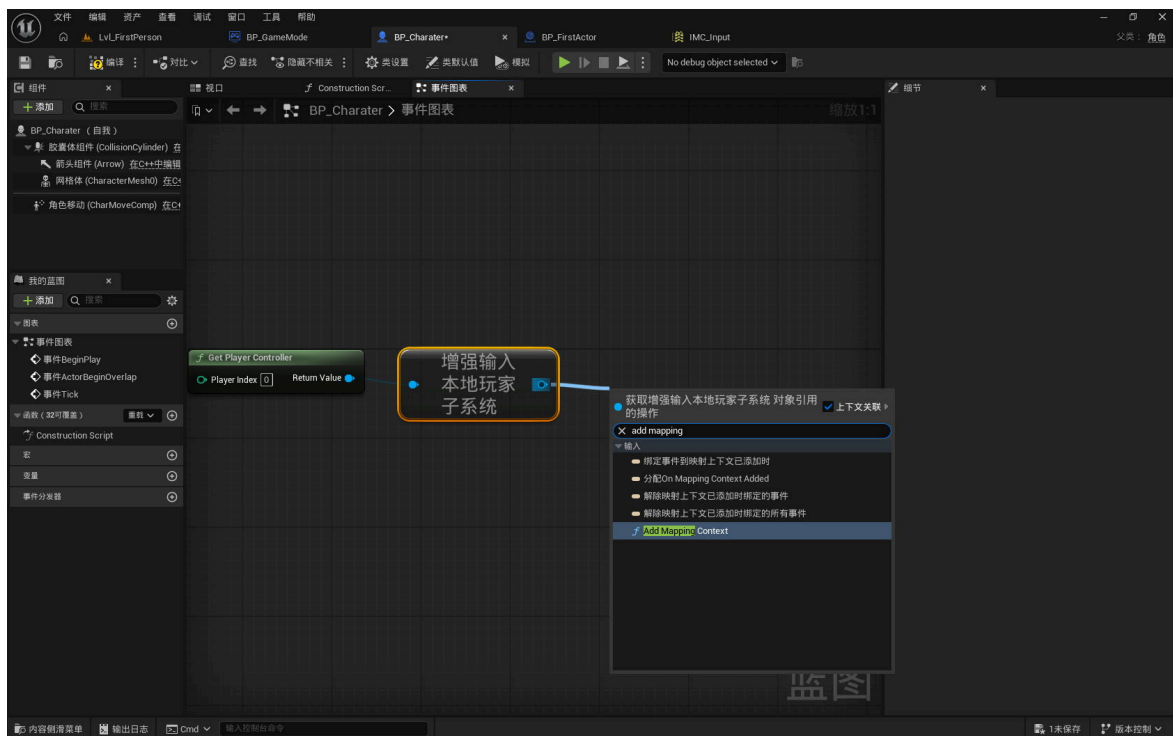
在事件图表界面右键搜索输入get player controller，点击新增 玩家控制器节点



左键长按玩家控制器右侧蓝色引脚拉出，在搜索输入框输入enhanced，获取 增强输入本地玩家子系统

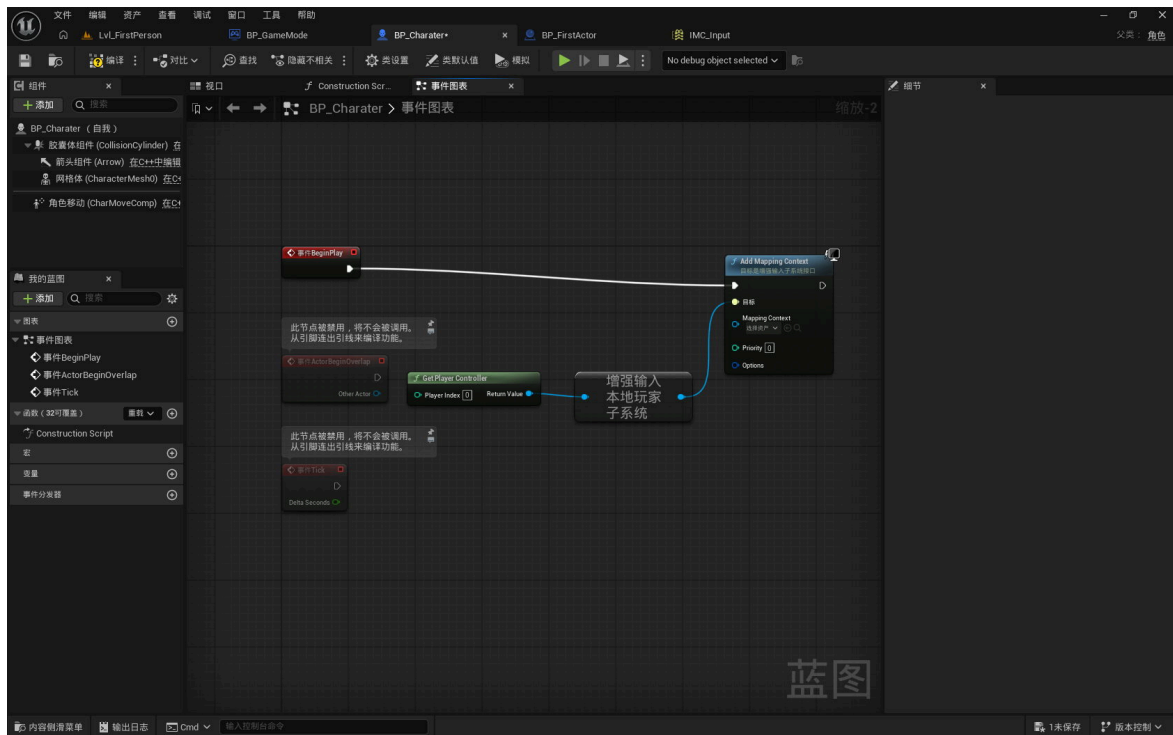


左键增强输入本地玩家子系统右侧蓝色引脚，在搜索输入框中输入**add mapping context**，获取 **映射上下文**

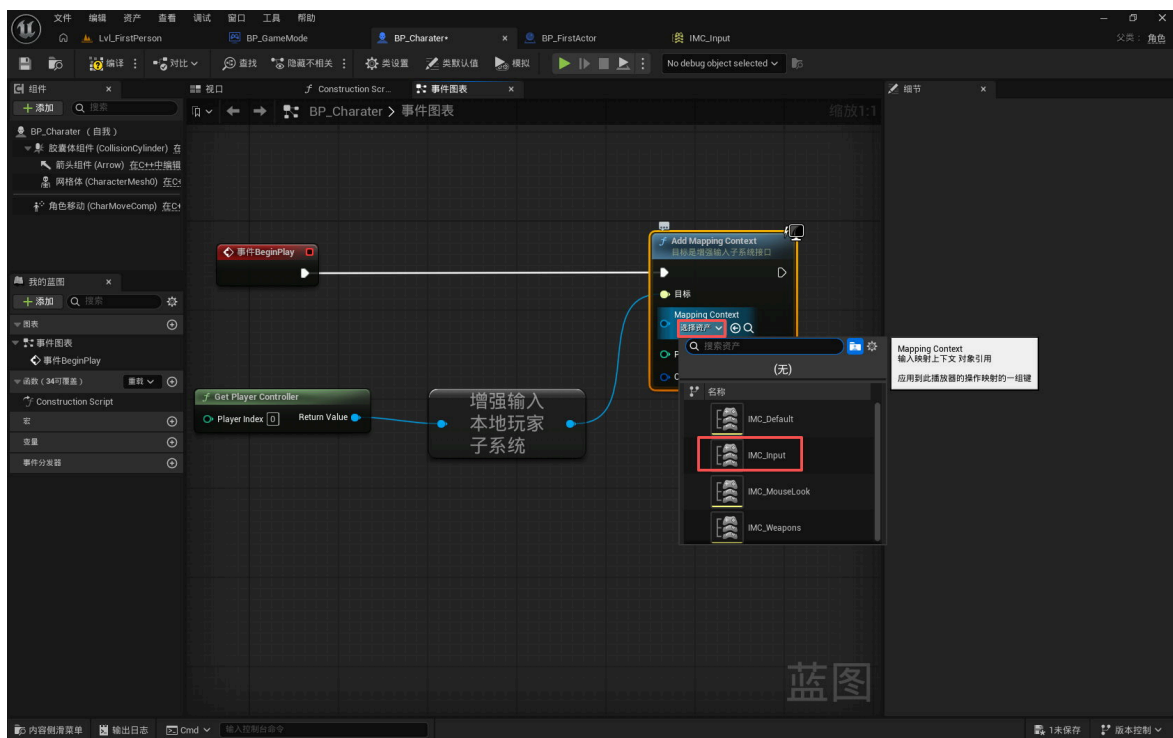


左键长按事件开始运行的左侧执行引脚，连接到映射上下文

如果觉得界面拥挤，可以按del键或右键删除其他两个初始节点，目前用不到

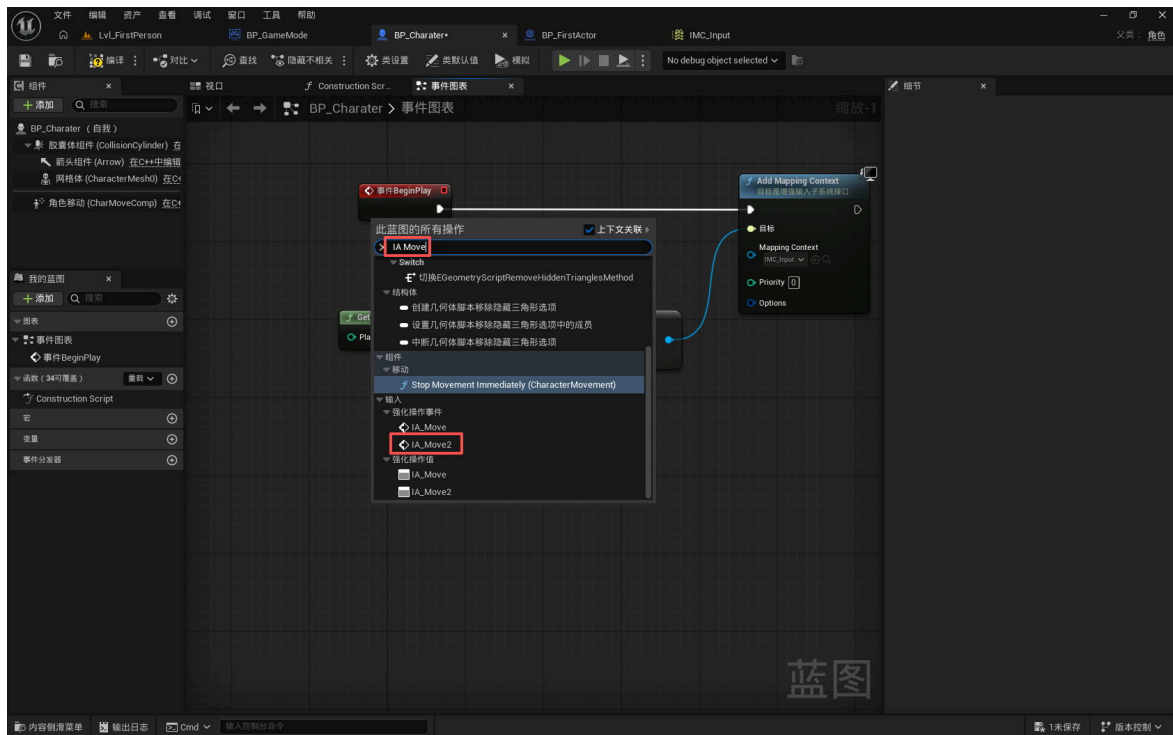


连接之后，在映射上下文节点中的**选择资产**中，选择已设置的**IMC_Input**



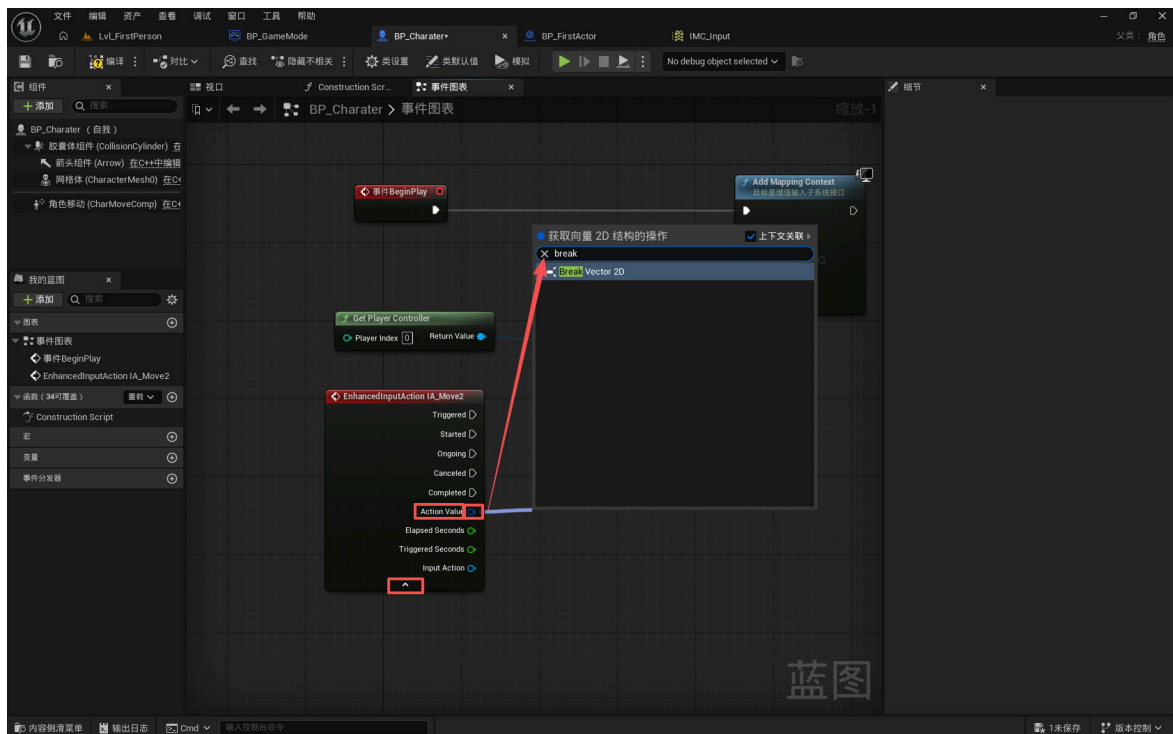
4. 编写角色移动

在事件图表界面右键，在搜索框中输入**IA Move**，选择强化操作事件中的**IA Move2**



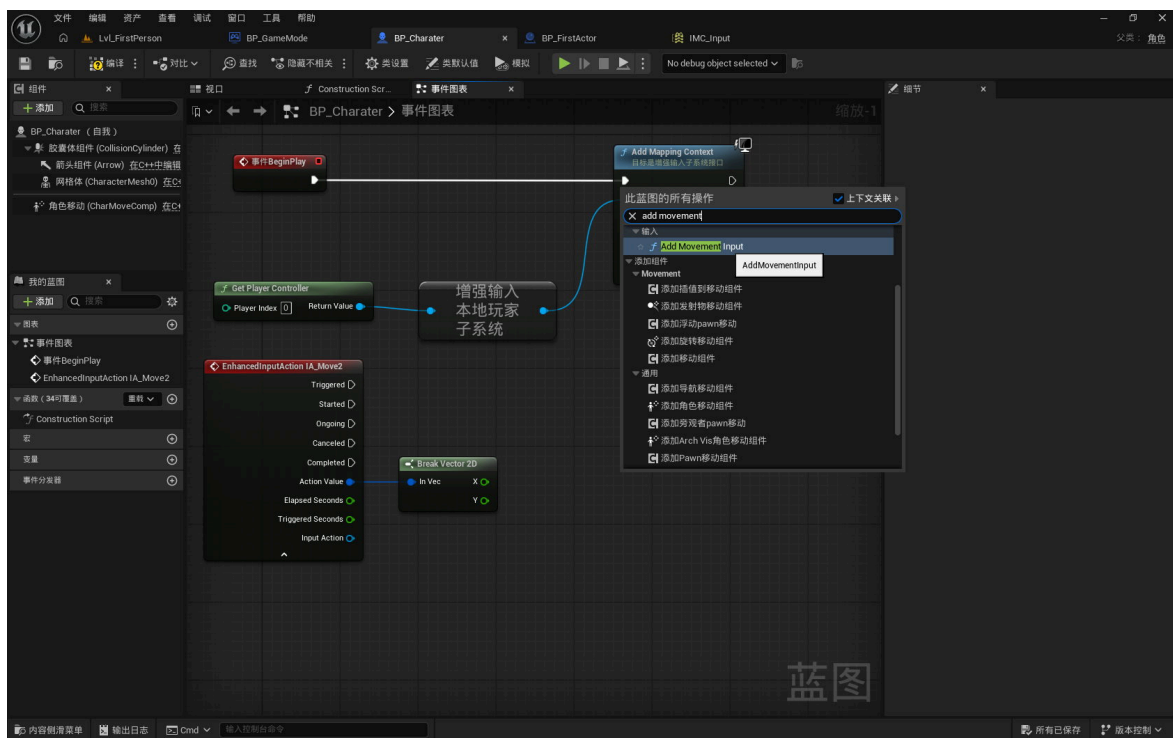
在新增的节点中，点击**展开**，左键长按Action Value右侧引脚，搜索输入**break**，选择 **Break Vector 2D**

Action Value为向量2D结构，拆分向量2D可以将2D向量拆分为**X**和**Y**的两个浮点数

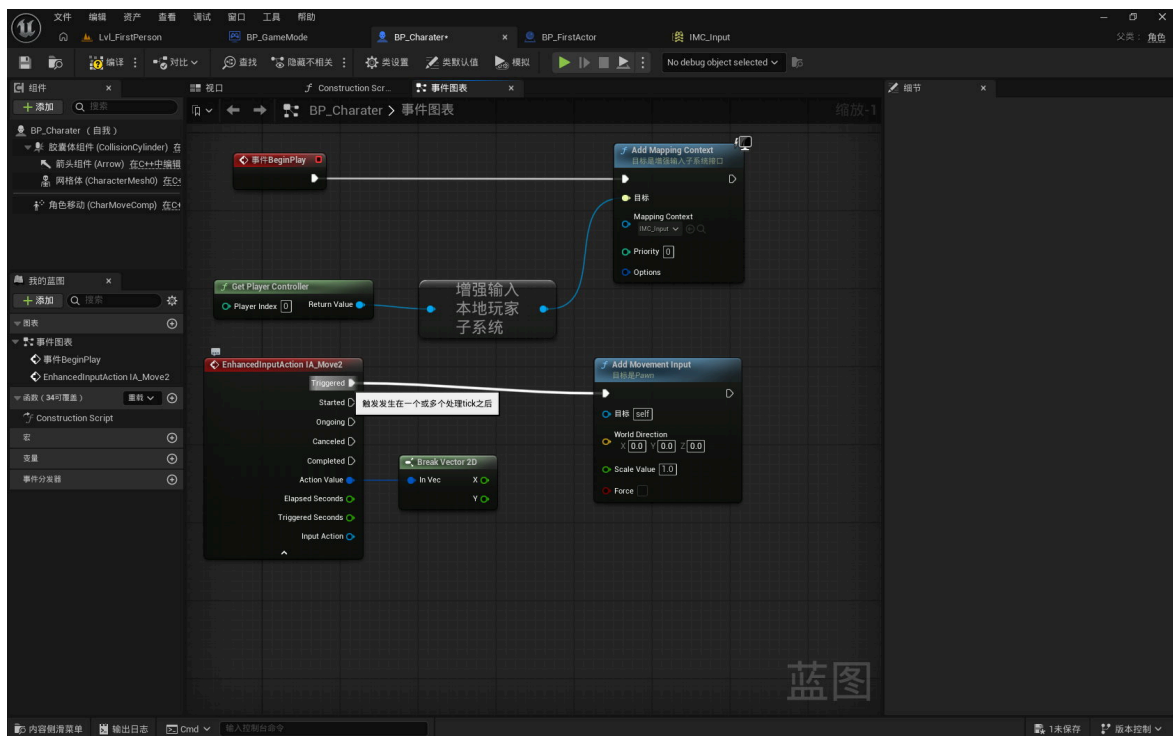


在事件图表界面右键，在搜索框中输入add movement input，点击添加 **Add Movement Input**

将鼠标悬停在新增节点处，按键盘 **alt** 键，即可查看节点原英文名称

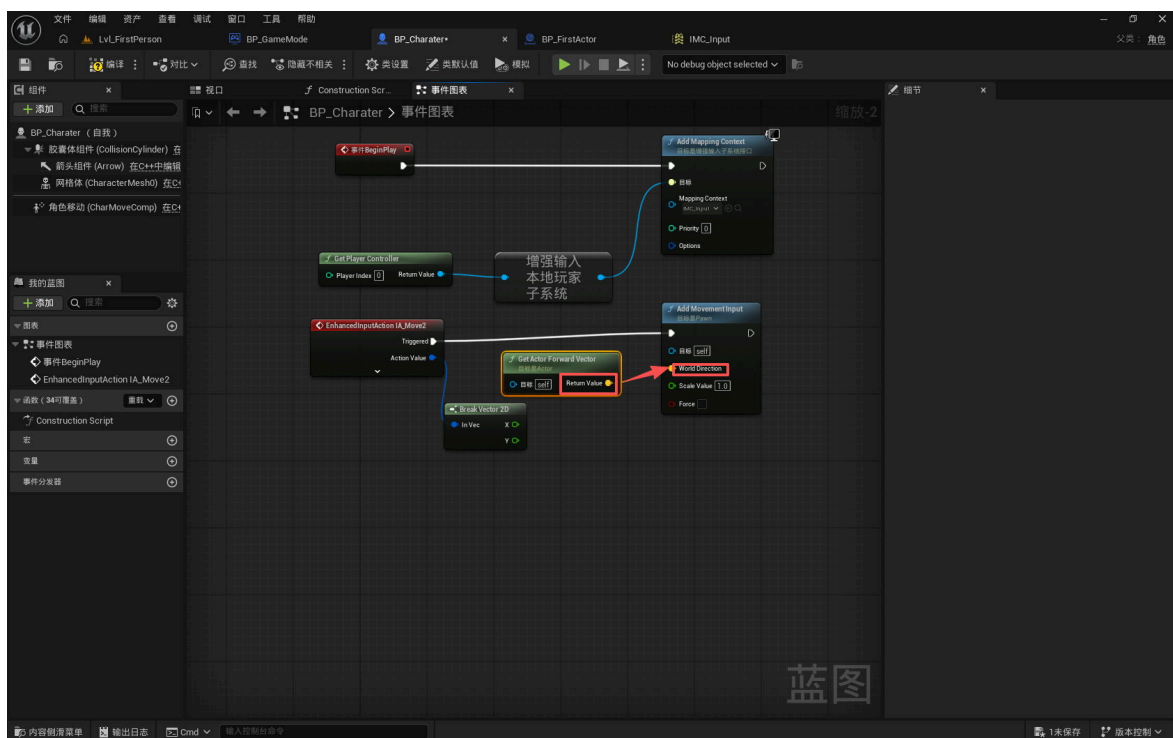
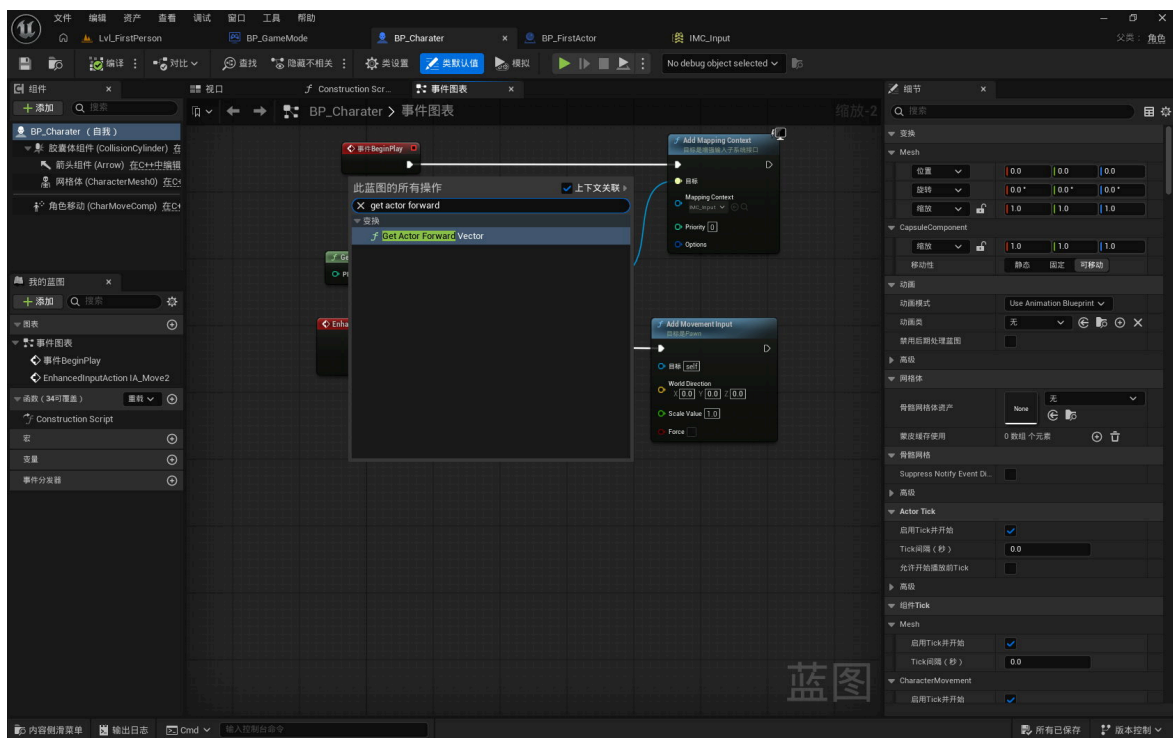


IA Move节点中存在多个引脚，其中**Triggered**引脚设置持续移动，因此需要将**Triggered**引脚连接到新增的移动输入节点



因为一般游戏中角色移动的方向取决于角色当前**视角正前方**的方向，因此还需要新增**获取Actor向前向量**

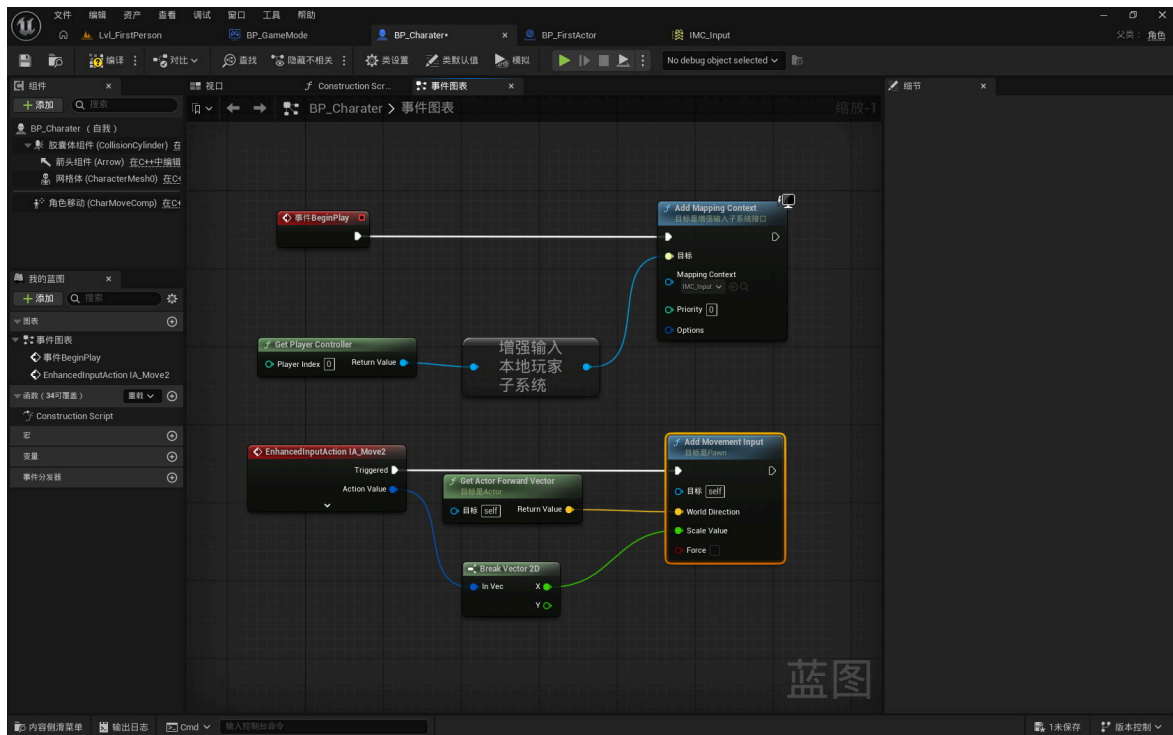
右键搜索输入**get actor forward**，选择 **Get Actor Forward Vector**，左键长按右侧黄色引脚，连接到移动输入的**World Direction**引脚处



5. 实现角色移动

Scala Value数值为正时，角色朝对应方向移动，数值为负时，角色朝向反方向移动

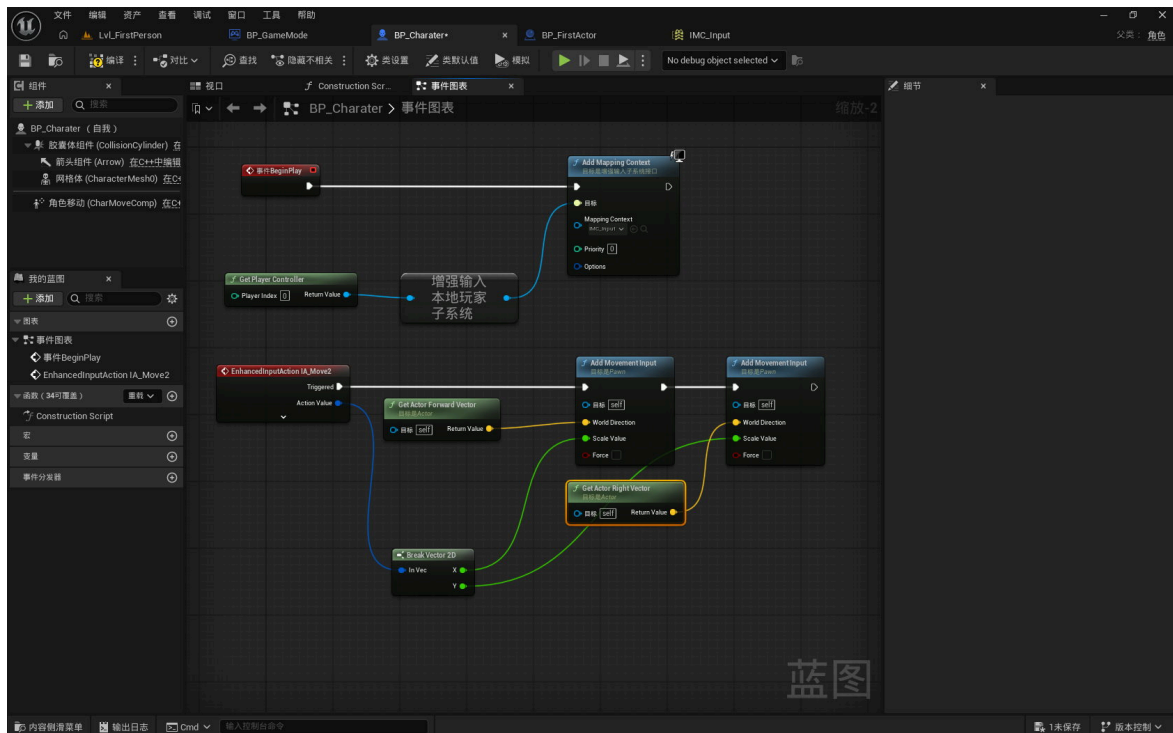
因此需要将拆分向量2D的X数值连接到**Scala Value**引脚处，此时WS的向前向后移动在完成编译之后即可生效



在已有的add movement input节点的执行引脚处，长按左键新增新的add movement input

因为需要实现的是左右移动，因此不能使用向前向量，右键搜索输入get actor right，新增获取向右向量

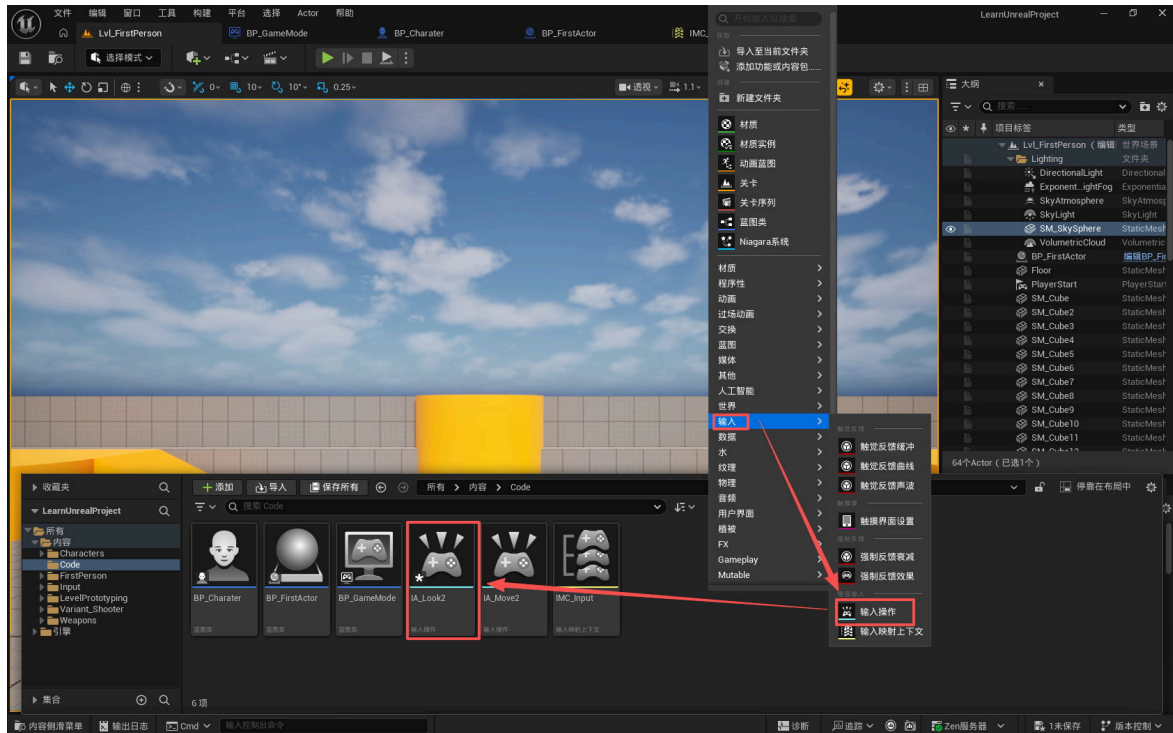
将新增的获取向右向量节点和拆分向量2D的Y数值连接到新增的add movement input的World Direction和Scala Value引脚处，点击编译后即可实现角色的左右移动



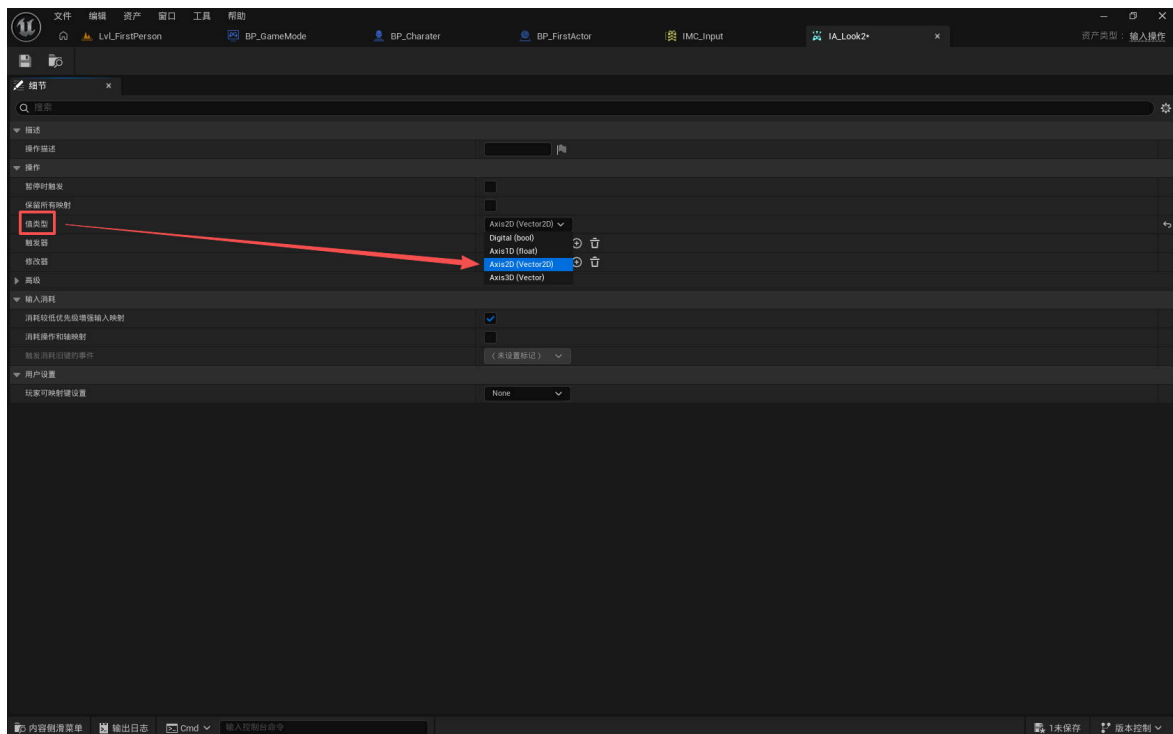
三、实现视角移动

1. 新增增强输入

在内容侧滑菜单的code目录下，新增输入操作，命名为IA_Look2

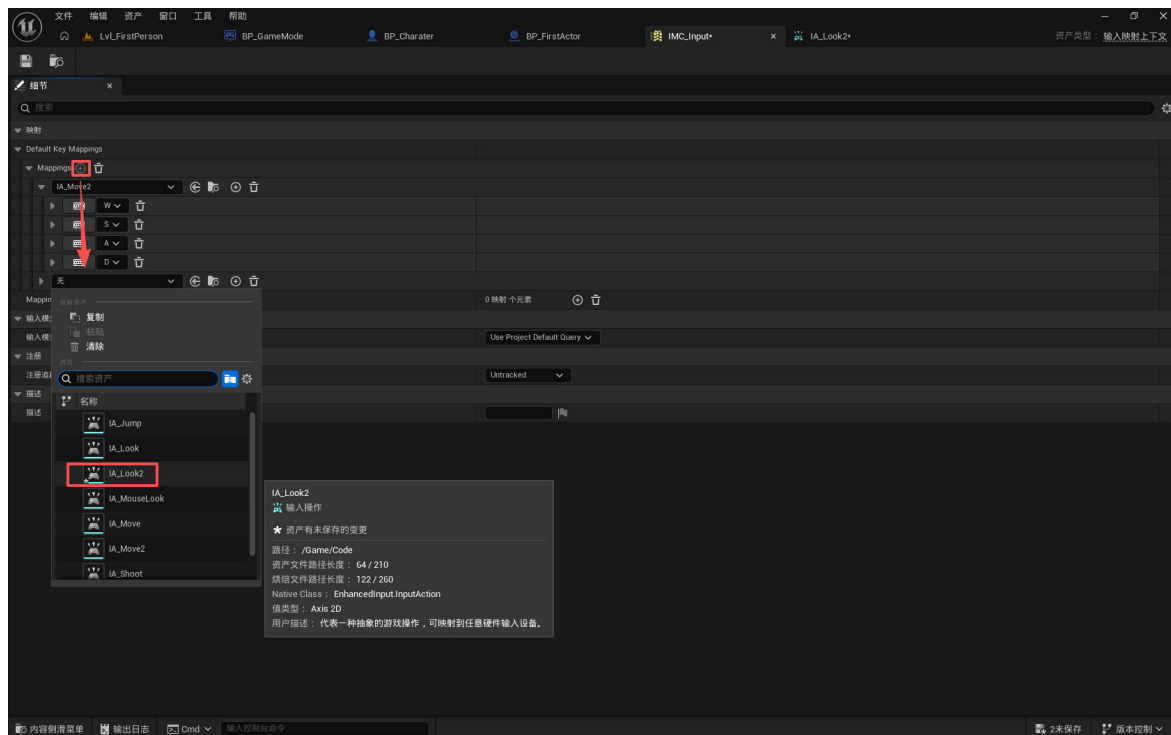


双击IA_Look2，进入细节界面，因为视角的上下左右旋转也是二维操作，所以修改值类型为Axis2D

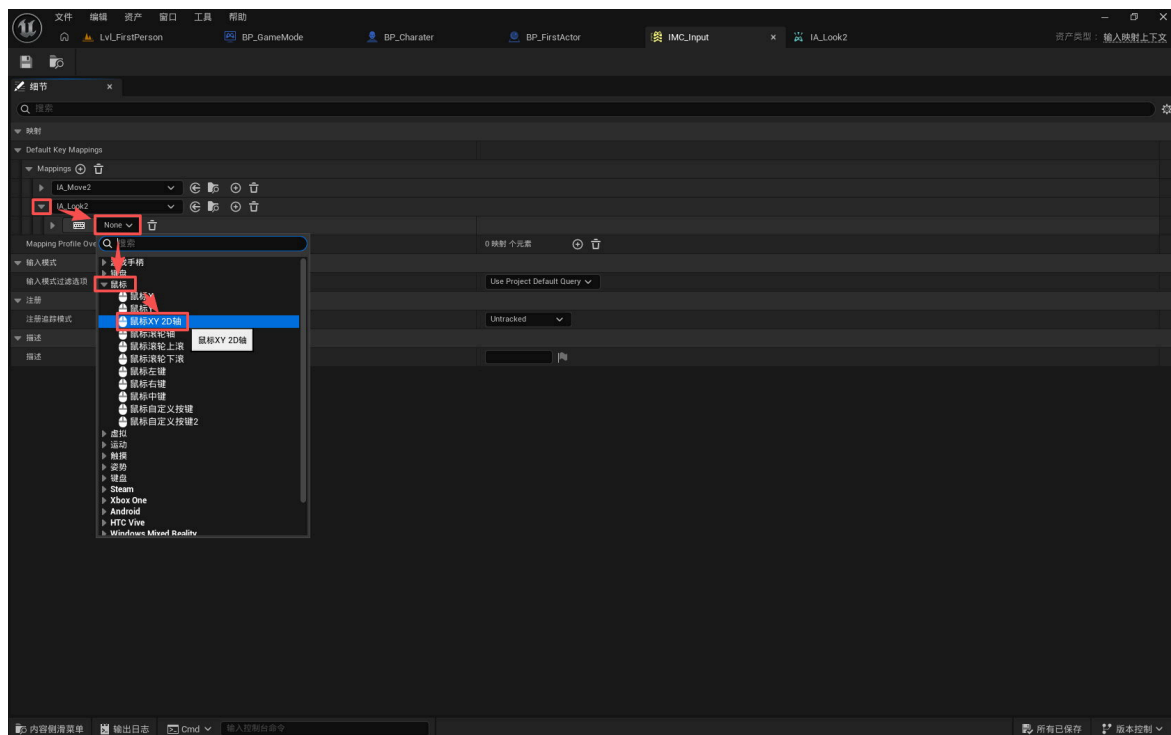


2. 设置增强输入

在映射上下文**IMC_Input**中添加操作映射，选择新增的**IA_Look2**

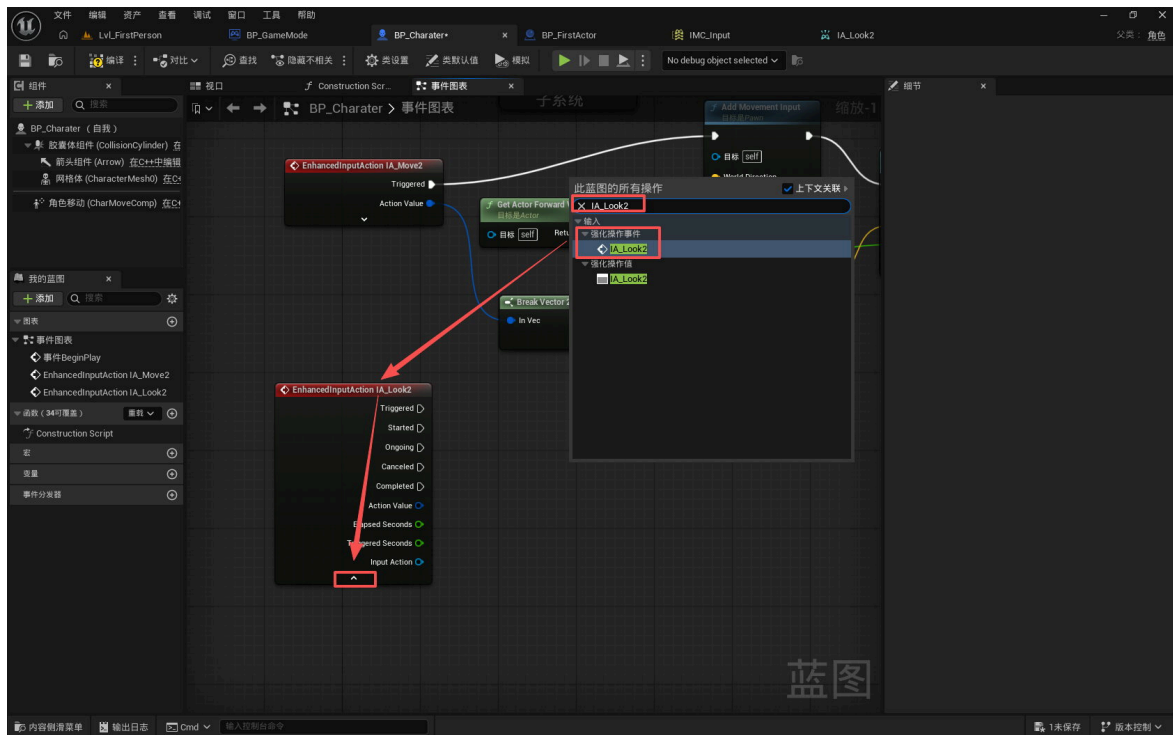


展开选好的**IA_Look2**，设置映射键值为鼠标的XY 2D轴，完成设置



返回BP_Character，将设置好的**IA_Look2**节点添加到事件图表中

右键搜索输入**IA_Look2**，选中强化操作事件中的**IA_Look2**



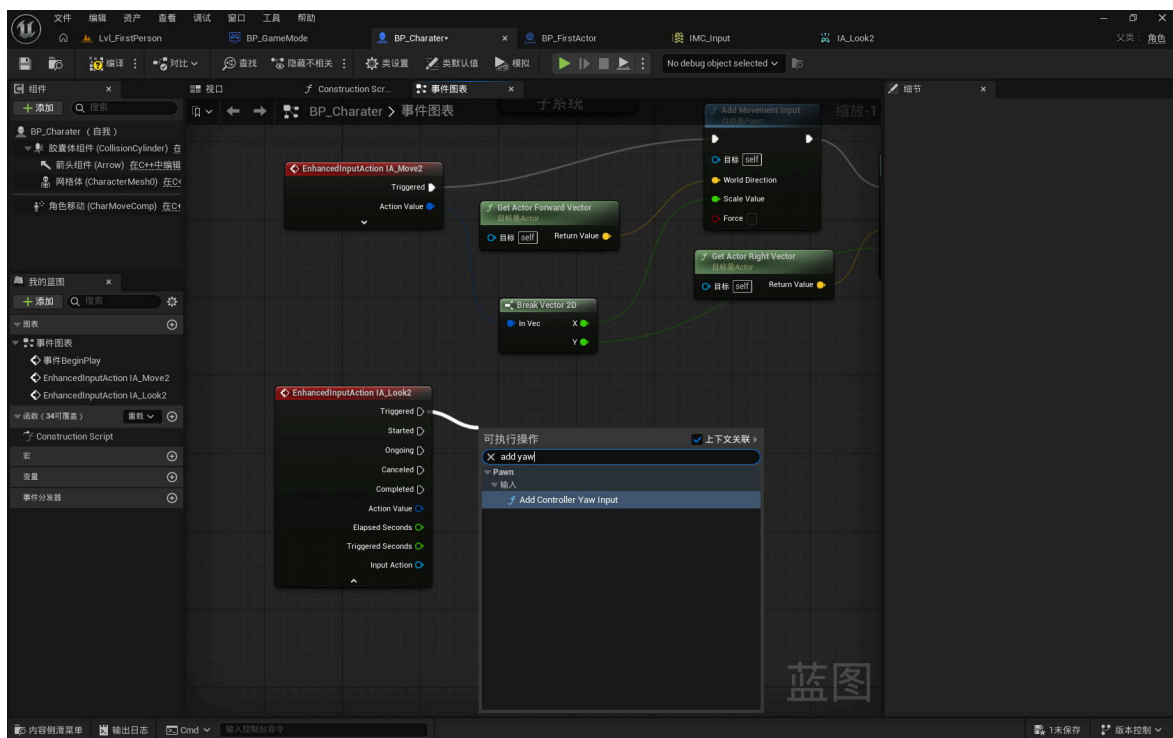
3.roll、yaw、pitch知识点

载体坐标系与载体坐标系的关系是三个Euler角:yaw, pitch, roll, 反应了载体相对基准面的姿态:

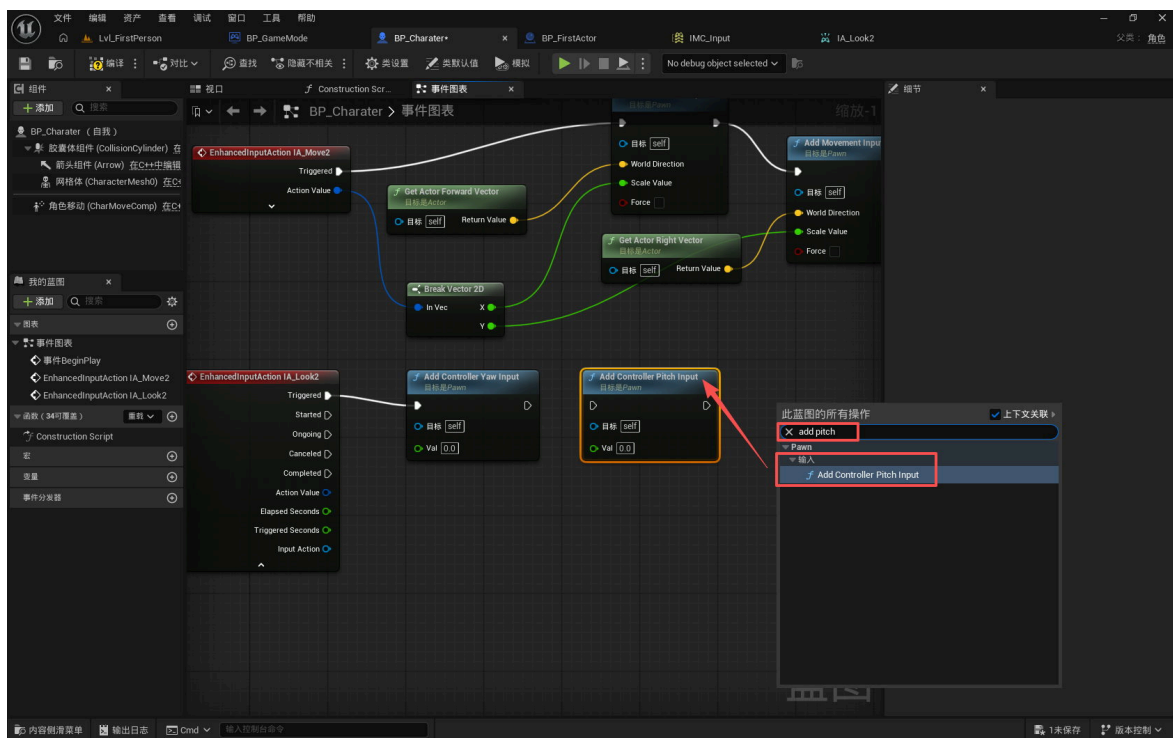
- pitch是围绕Y轴旋转, 也叫做俯仰角(视角上下移动)。当X轴的正半轴位于过坐标原点的水平面之上(抬头)时, 俯仰角为正, 否则为负。
- yaw是围绕Z轴旋转, 也叫偏航角(视角左右移动)。即机头右偏航为正, 反之为负。
- roll是围绕X轴旋转, 也叫翻滚角(视角旋转)。机体向右滚为正, 反之为负。

4. 编写视角移动

视角移动也属于持续操作, 左键长按Triggerd的执行引脚, 在搜索输入add yaw, 选择 **Add Controller Yaw Input**

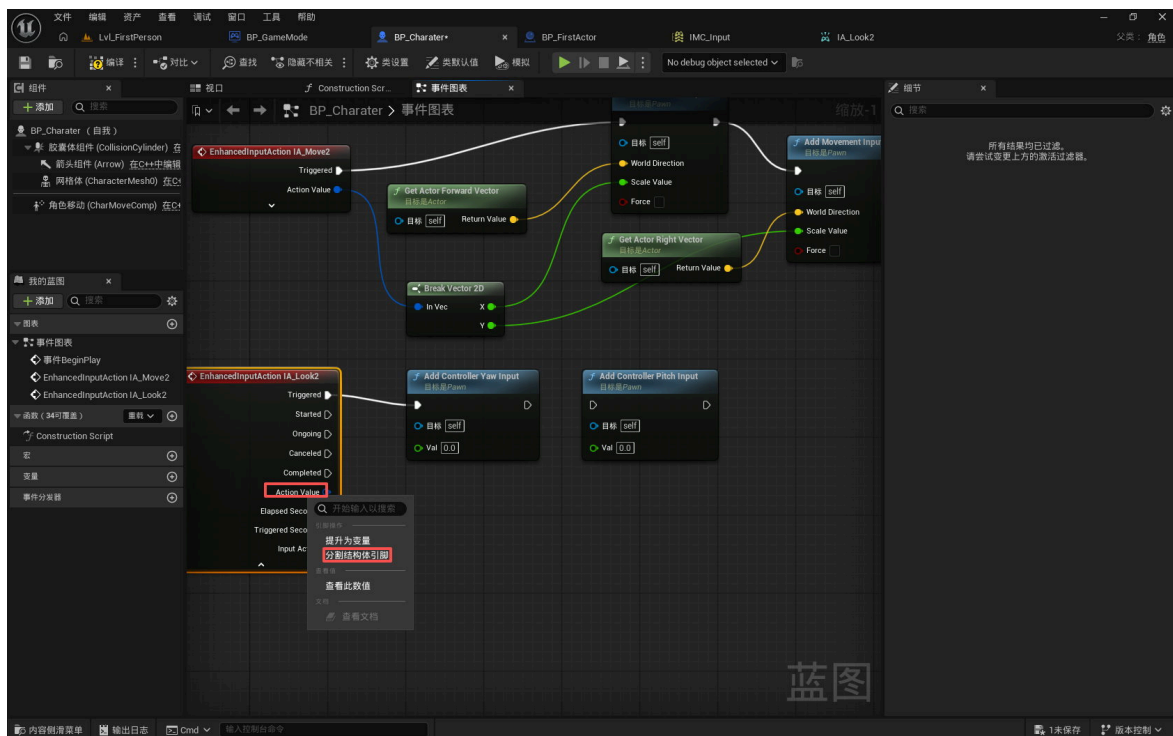
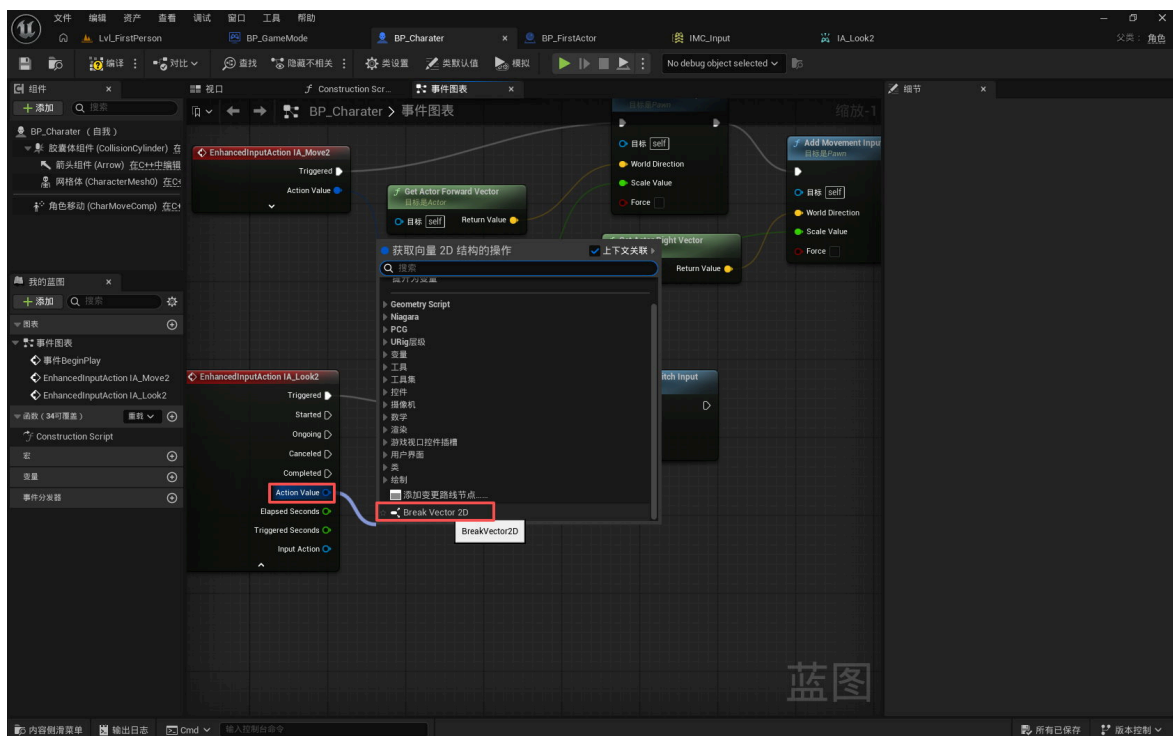


右键搜索输入add pitch，选择 **Add Controller Pitch Input**



左键长按IA_Look2节点的Action Value的引脚

快捷操作：将弹出的界面下滑到最底部，直接选择拆分向量2D，或直接右键Action Value文本，选择分割结构体引脚，Action Value会分裂成**Action Value X**和**Action Value Y**

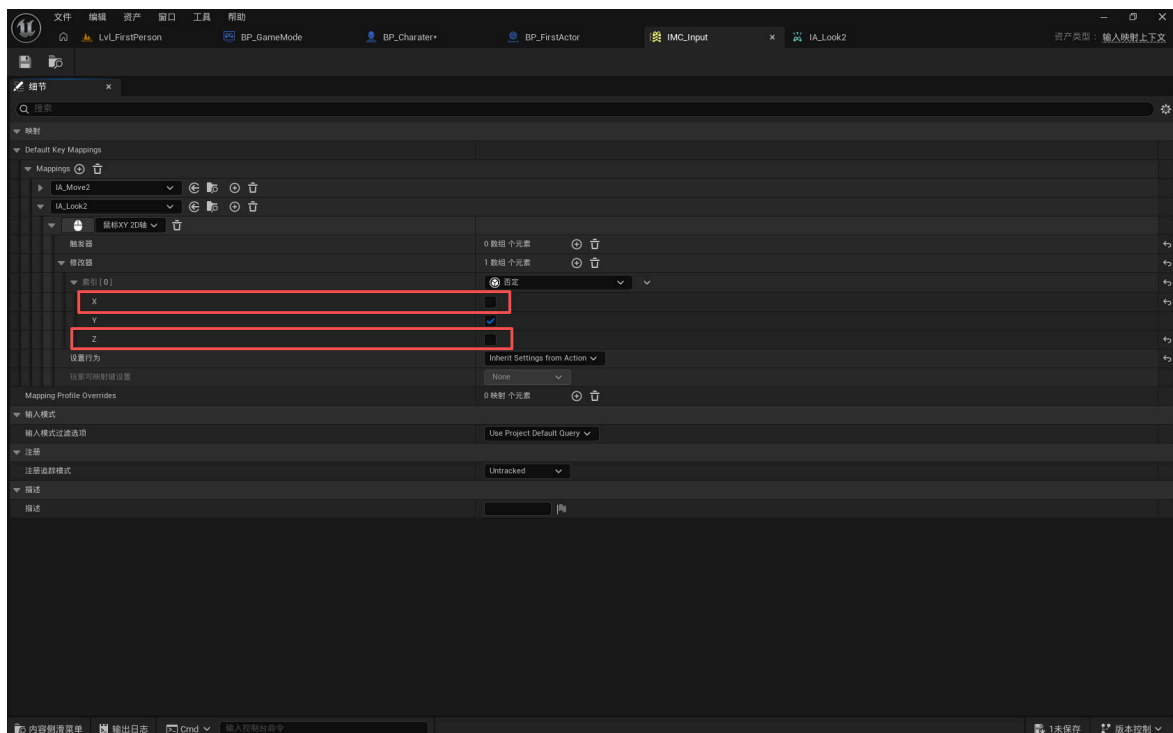
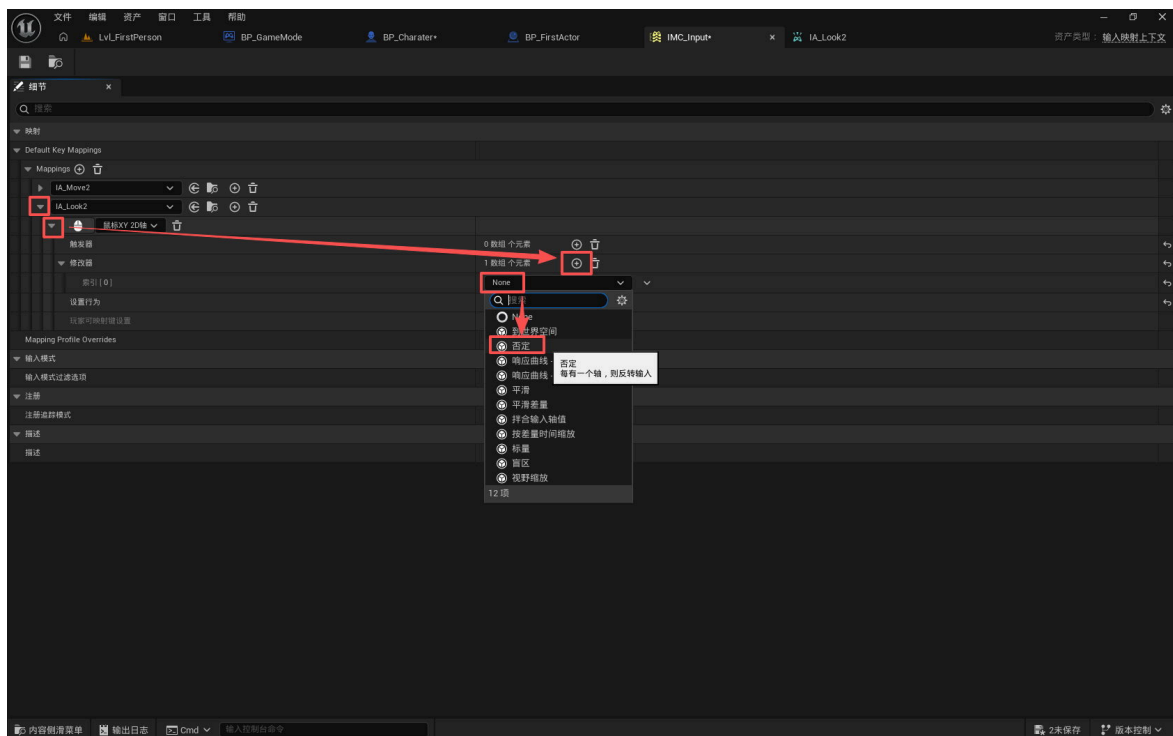


连接yaw输入和pitch输入的执行引脚

因为yaw负责左右视角移动，pitch负责上下视角移动，所以将2D向量的X连接到控制器Yaw输入的Val输入引脚，将2D向量的Y连接到控制器Pitch输入的Val输入引脚

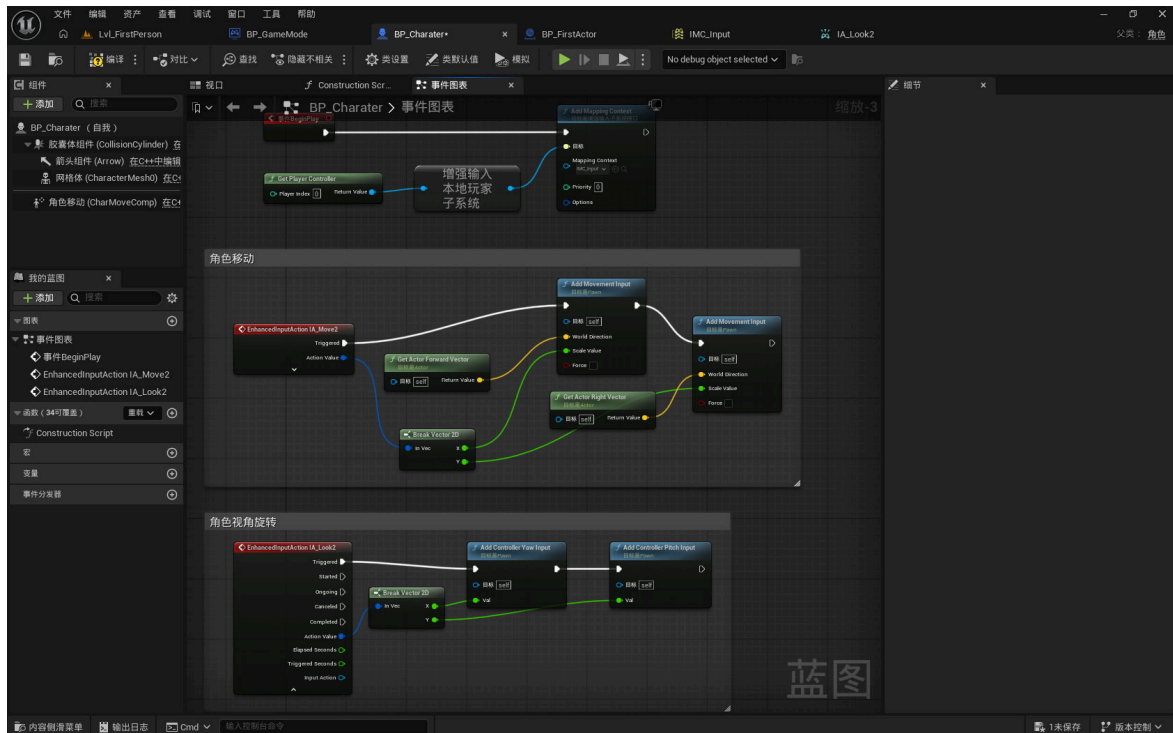
2.在映射上下文IMC_Input中继续修改IA_Look2:

- 展开IA_Look2的选择键值栏，在修改器栏选择添加元素
- 在新增的索引栏选择否定，取消XZ选项，仅保留Y的勾选
- 注意：此方案不需要更改原有事件图表中的连接



四、添加注释

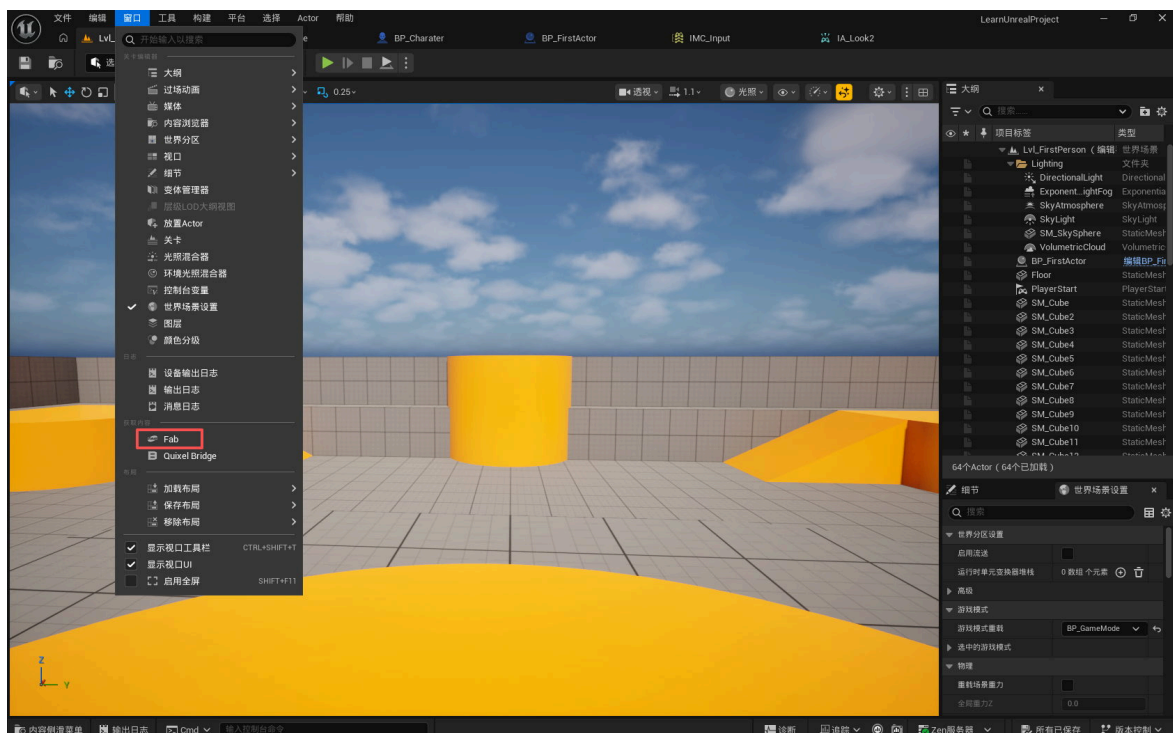
长按鼠标左键，框选住需要注释的节点群，松开后按**键盘C键**

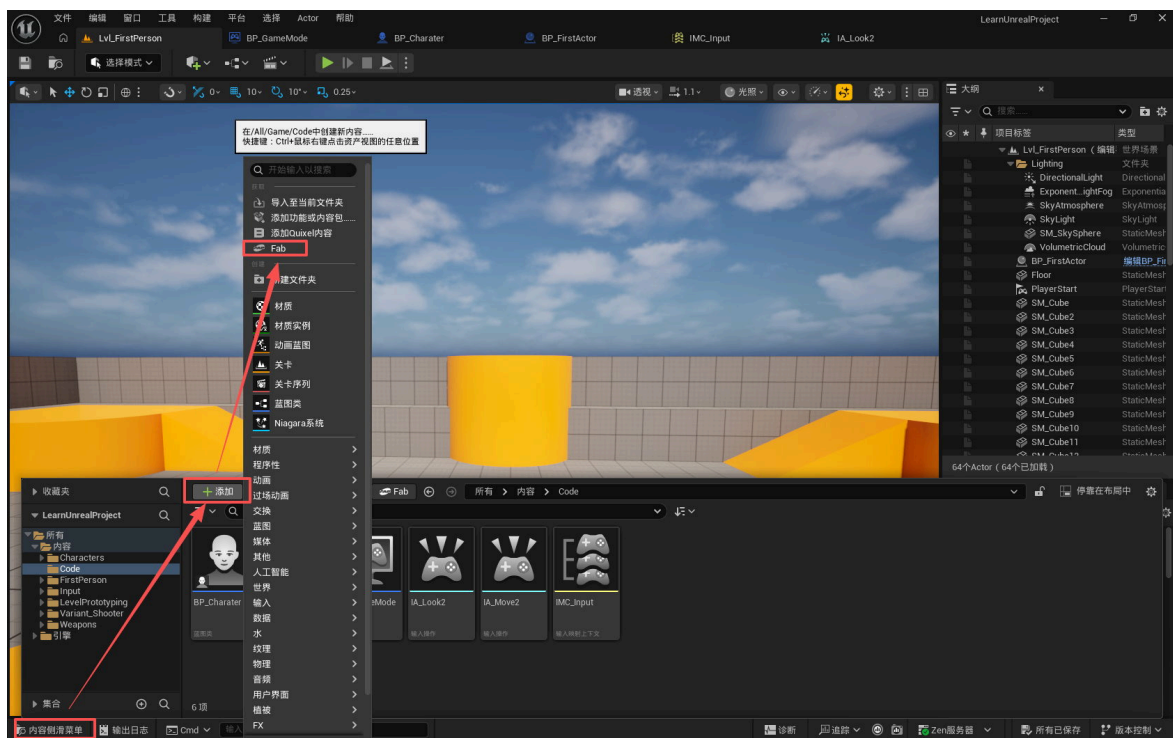


五、实现自动门功能

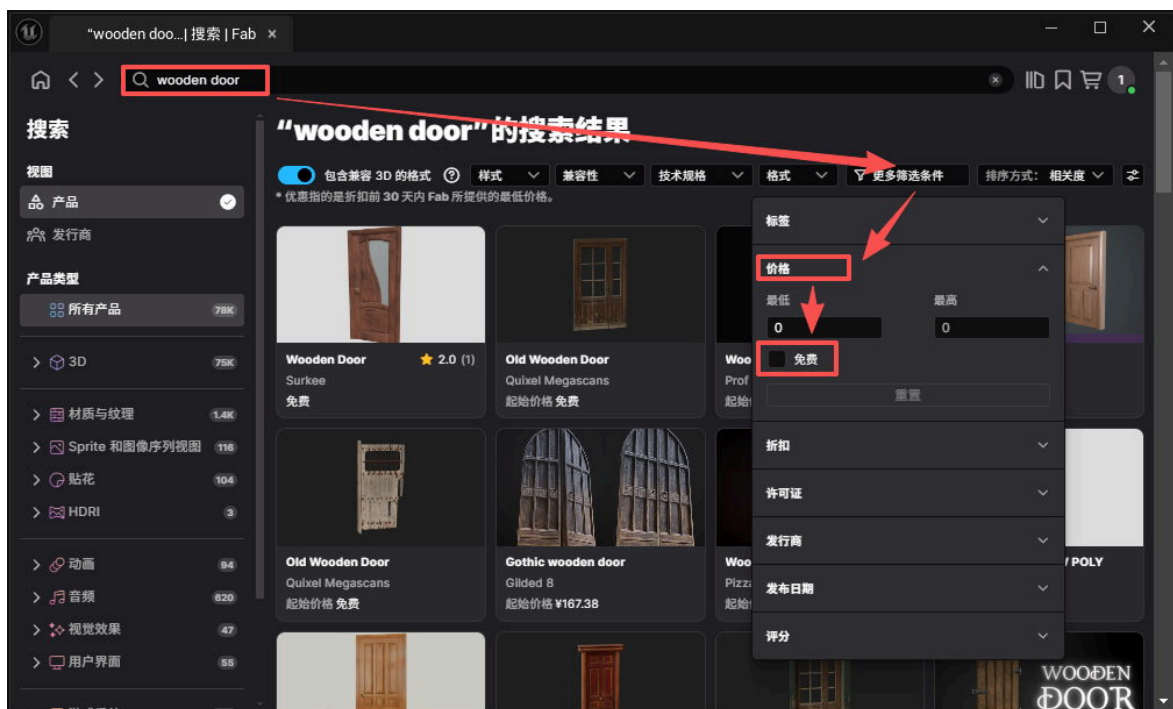
1. 导入素材

在虚幻引擎界面**已经安装Fab插件**之后，在项目界面内的窗口或内容侧滑菜单界面选择Fab商城

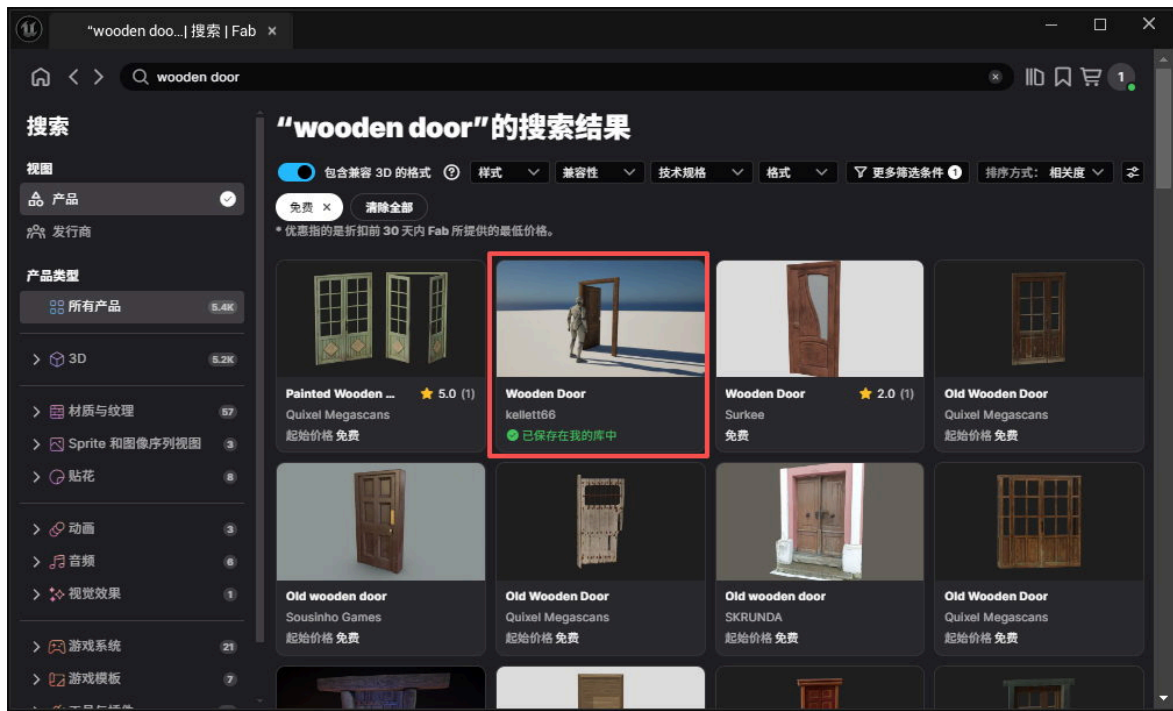




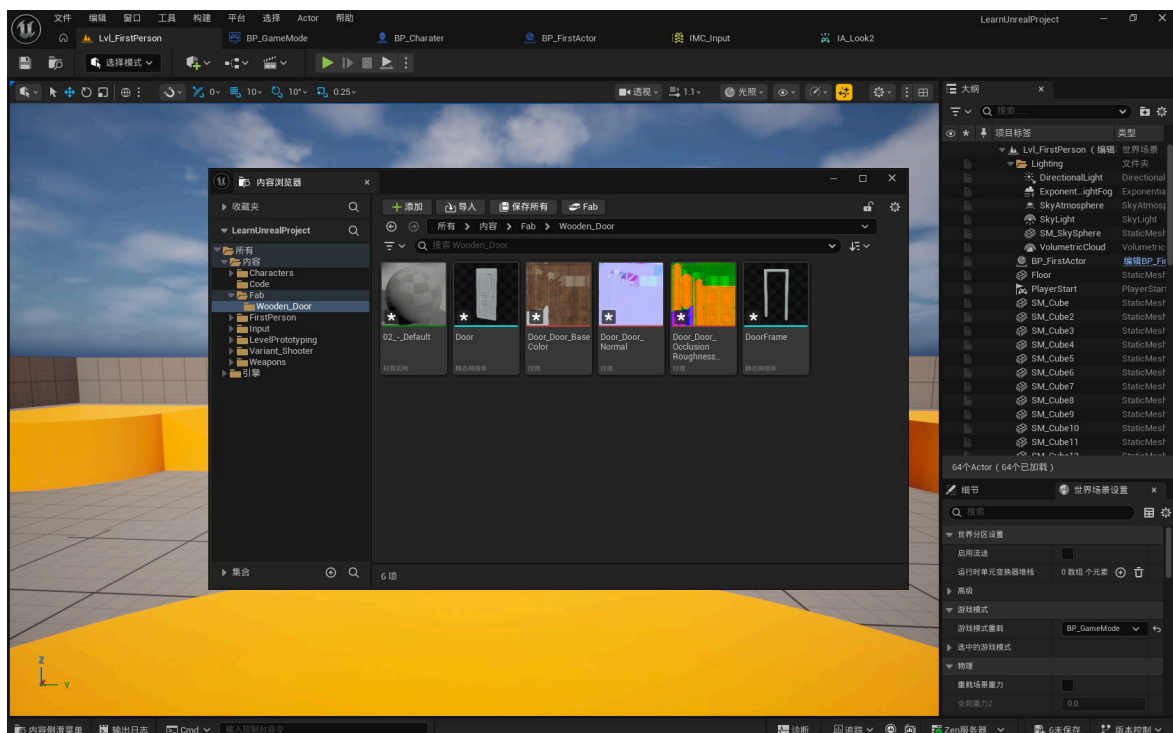
- 在弹出的Fab界面内的搜索框中输入**wooden door**，回车后在下方条件筛选栏的最后，选择**更多筛选条件**，之后选择价格，勾选免费
- Fab商场全屏展开的情况下，直接选择**价格**筛选条件
- 如果Fab商城长时间无法加载成功，检查网络或选择**加速器**和**梯子**



在筛选后的界面内选择如图木门资源，点击购买之后选择**添加到项目**



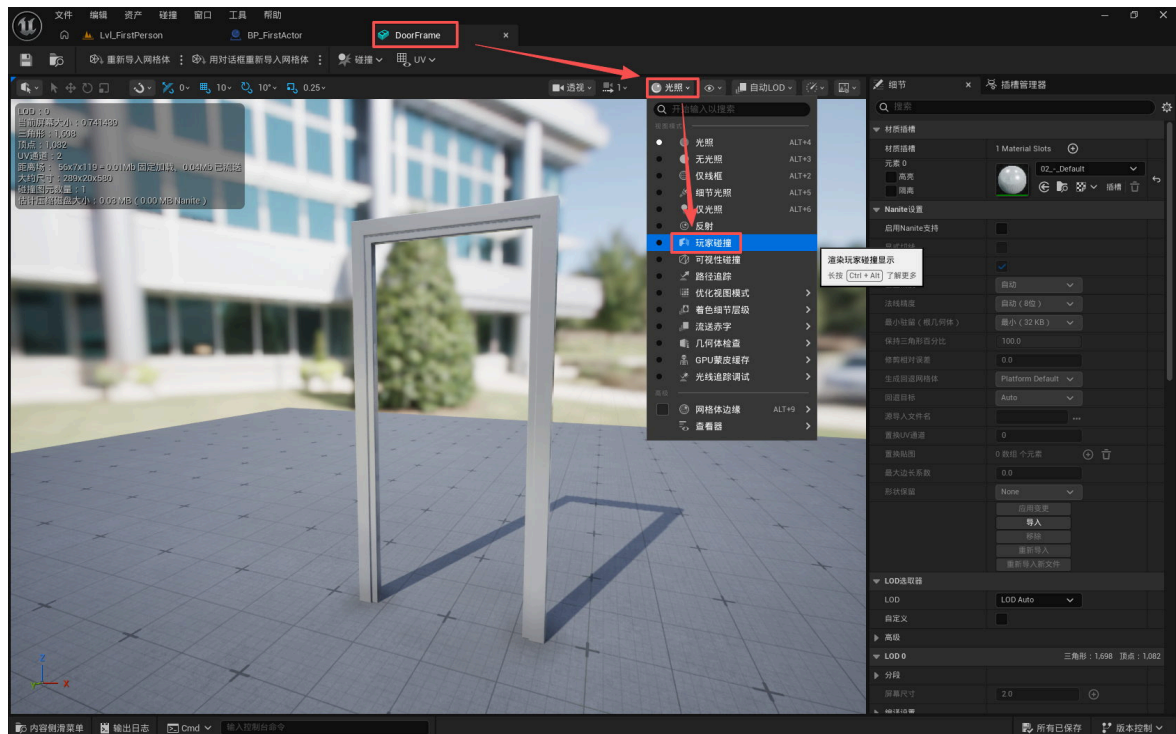
- 出现内容浏览器，代表素材导入成功，此时可关闭Fab商城
- 将内容浏览器显示的路径/Game/Fab/Wooden_Door进行复制，之后关闭内容浏览器
- 打开内容侧滑菜单，将复制的路径覆盖到当前目录上，路径对应根目录下Fab目录的Wooden_Door目录
- 目录中包含六个文件，第一个02_defalut为材质，是第二个素材Door门和第六个素材DoorFrame门框的材质，决定了门的外观，后续三个Door_Door_*是纹理，类似于贴图，可以使用在第一个素材材质上面
- 关键在于第二个素材和第六个素材静态网格体，可以理解为模型



2. 修改门框碰撞体

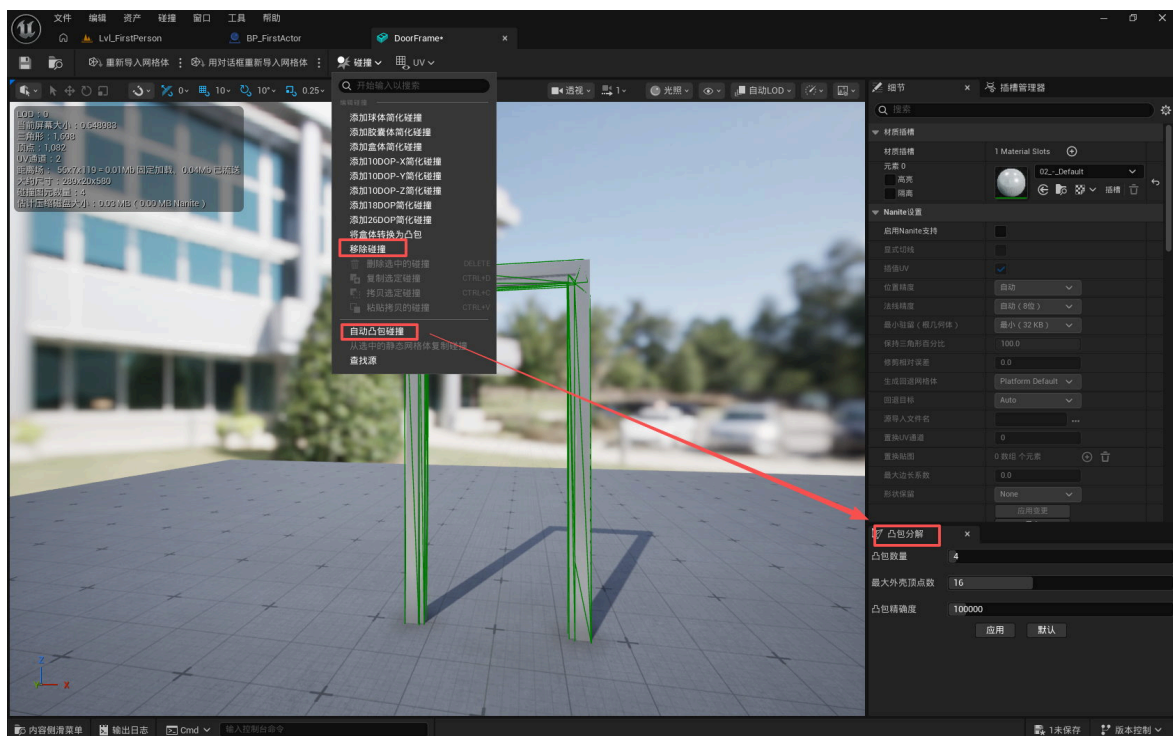
将DoorFrame门框拖拽到关卡内，在运行后移动角色穿过门框，会发现门框无法穿过

双击DoorFrame，点击右上角光照，选择玩家碰撞，之后可以发现门框中间是实心的，因此角色无法通过



选择标签页下方的碰撞选项：

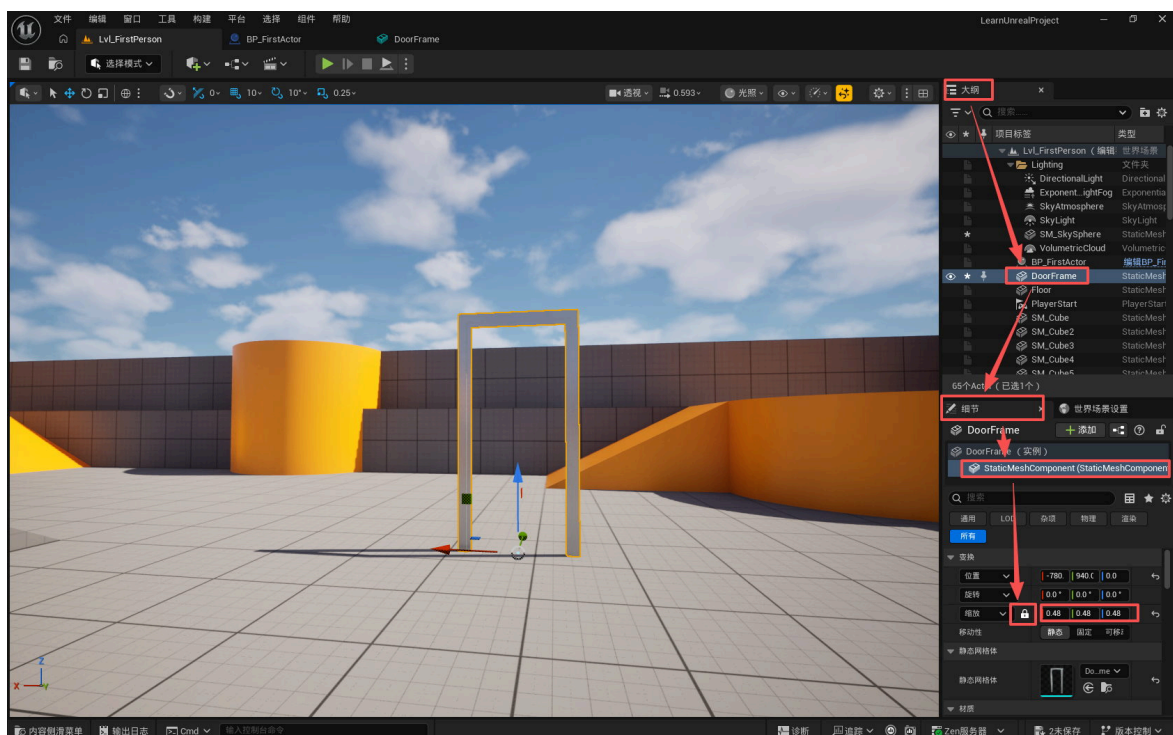
- 如果选择**移除碰撞**，再选择玩家碰撞模式观察，会发现在此模式中整个模型消失不见，说明整个门框都取消了碰撞体积
- 如果选择**自动凸包碰撞**，引擎会自动识别光照模式下可视模型范围，将对应模型转换为碰撞体
 - 选择自动凸包碰撞后，右下角会出现凸包分解窗口，如果在玩家碰撞模式下观察感觉碰撞体**存在瑕疵**，与观察模型相差较大，可以通过**增加凸包数量、凸包精准度**，来增加精确度
 - 增加对应设置之后会**增加游戏负荷**，不利于游戏优化，需要进行取舍



选择自动凸包碰撞，在凸包分解窗口选择应用之后，返回关卡**FirstPerson**标签，点击运行，可以发现此时门框内部角色可以穿过，但门框本体角色无法穿过

3. 修改门框体积

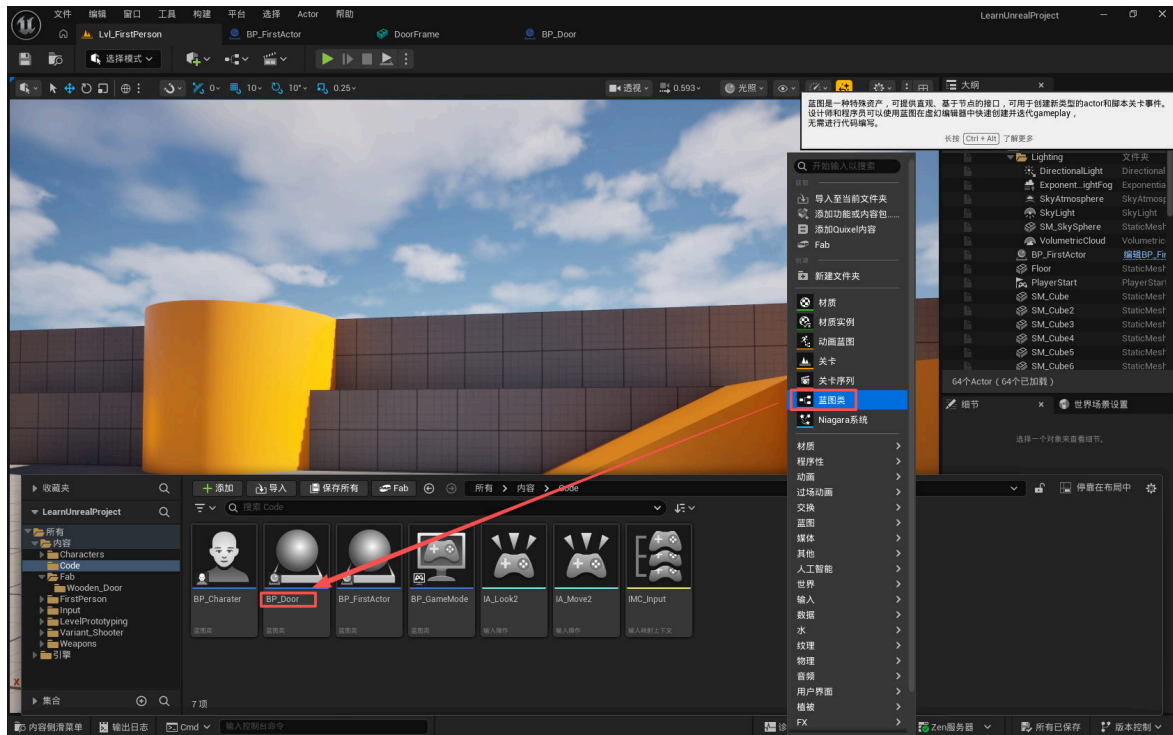
目前门框在关卡内体积过大，在关卡场景或大纲窗口中左键单击**DoorFrame**之后，选择右下角细节窗口，在缩放一栏点击锁定，之后在XYZ任意一个输入框中输入**0.48**



4. 绑定门与门框

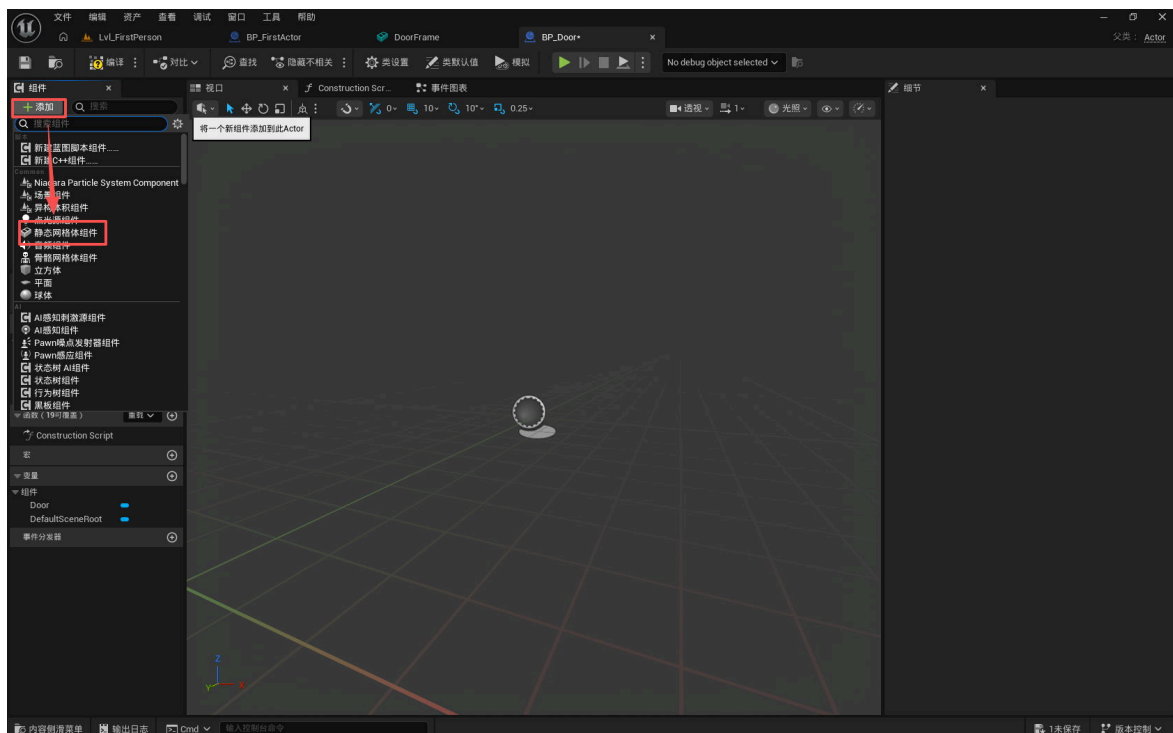
将门Door素材拖拽到关卡界面内，会发现门和门框都为静态资源，无法实现人物接近之后开关门的效果，因此需要将门和门框都放进蓝图内，作为编写蓝图类代码的一个资源

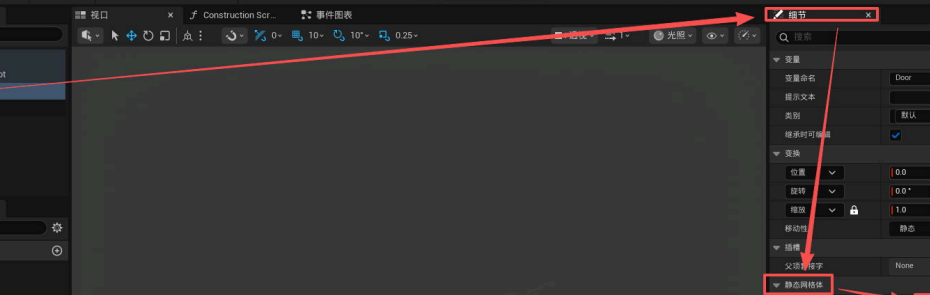
删除关卡界面内的门和门框，打开内容侧滑菜单，在code目录下新建蓝图类-**Actor**，命名为BP_Door



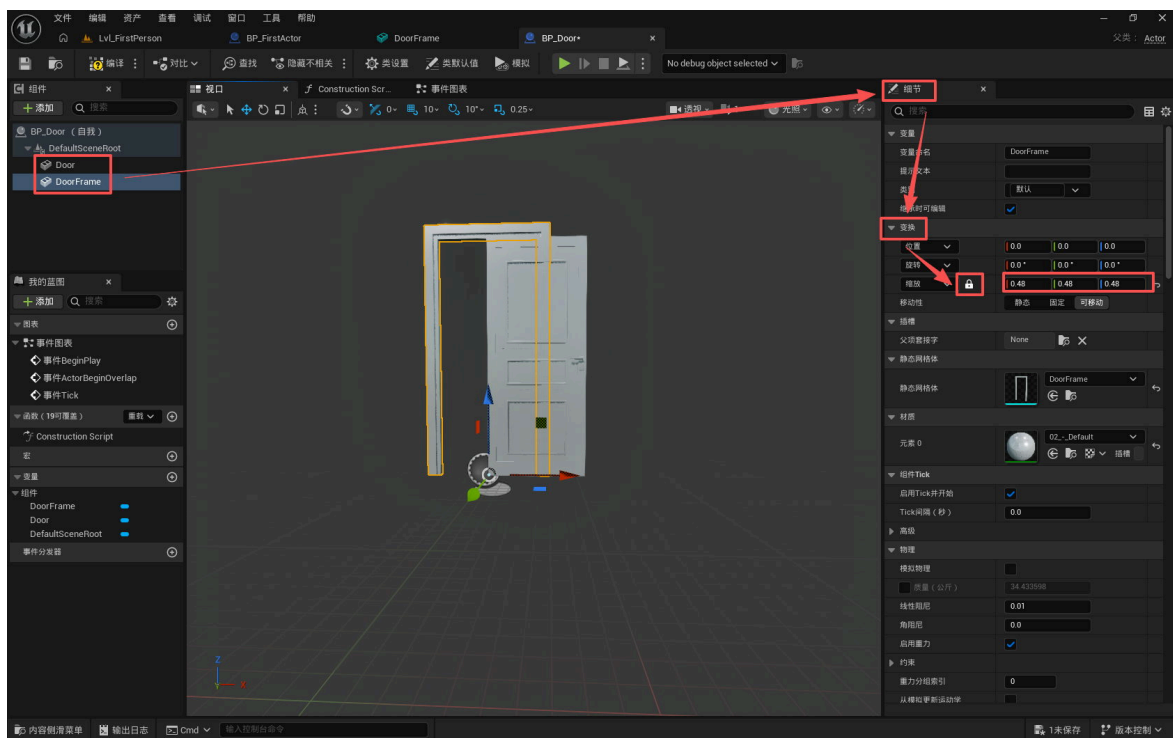
在内容侧滑菜单code目录下，左键双击新建的BP_Door，进入BP_Door的视口界面

左键单击组件窗口下方的新建两次，选择静态网格体组件，一个命名为Door，一个命名为DoorFrame



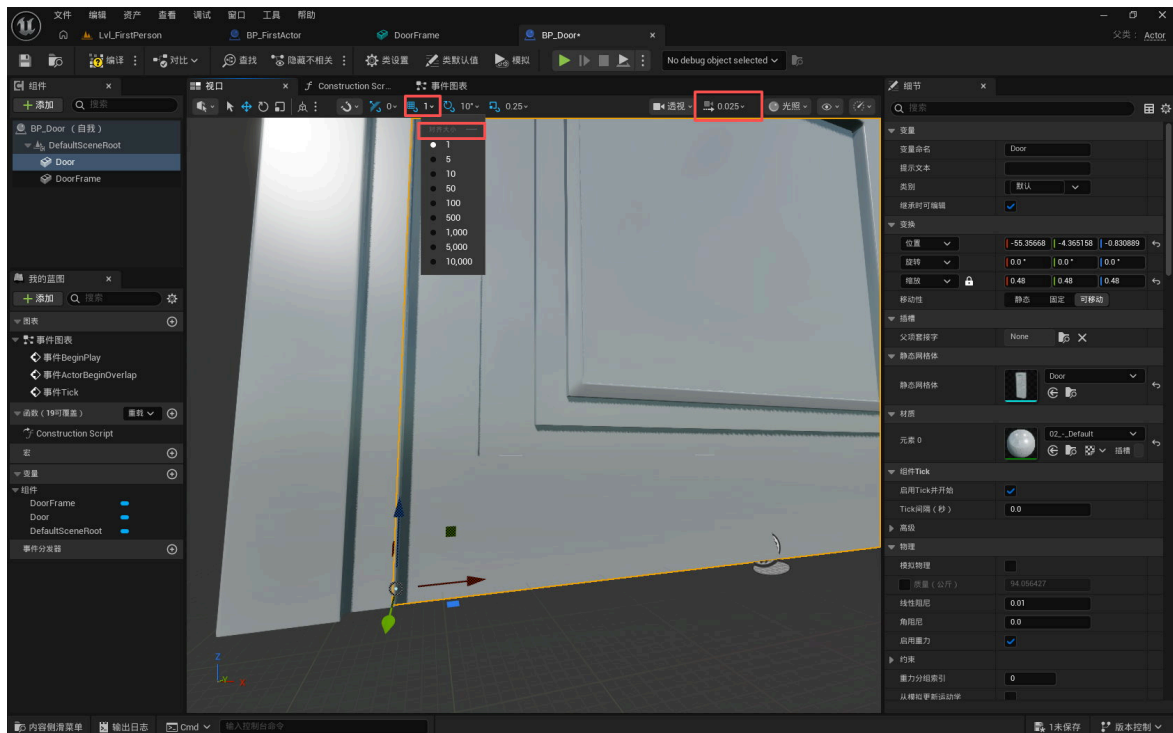


在变换栏中的缩放为**锁定状态**下，在后续任意栏中输入**0.48**，调整两个模型的体积大小



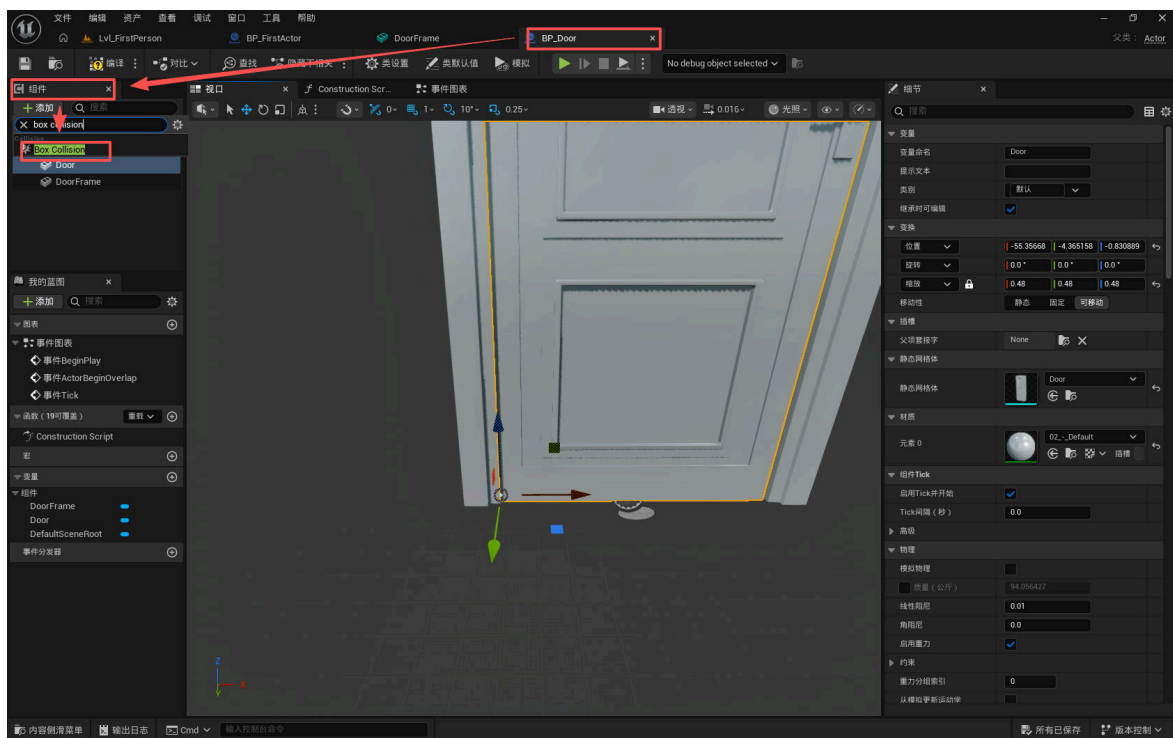
在Actor视口界面，手动调整门与门框的位置，可以在视口菜单的移动对齐数值设置按钮处设置移动模型的单位距离，**闭关时为无极调整**

可以在透视右侧的按钮处调整视角移动的速度，或者右键的同时移动滚轮来调整

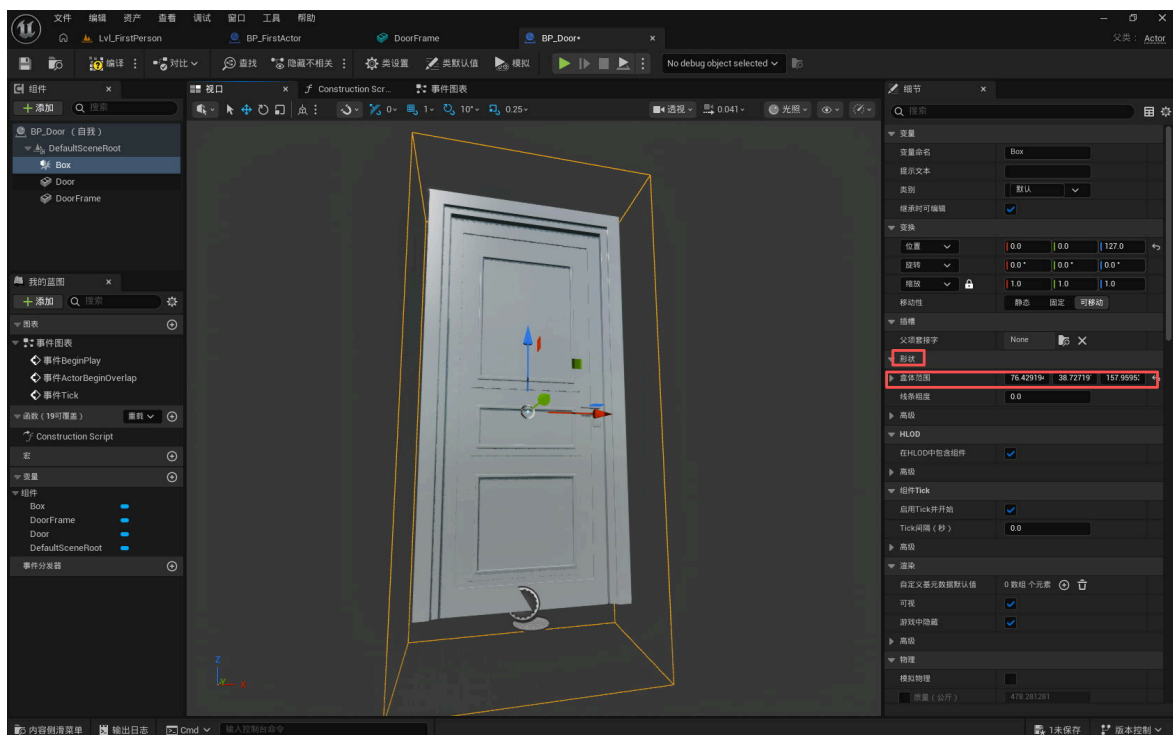


5. 编写单向自动门

为了实现在人物接近时门自动开关，需要进行碰撞检测，因此还需要在BP_Door中加入 **Box Collision**，在视口界面的组件页下方左键单击添加，搜索选择Box Collision

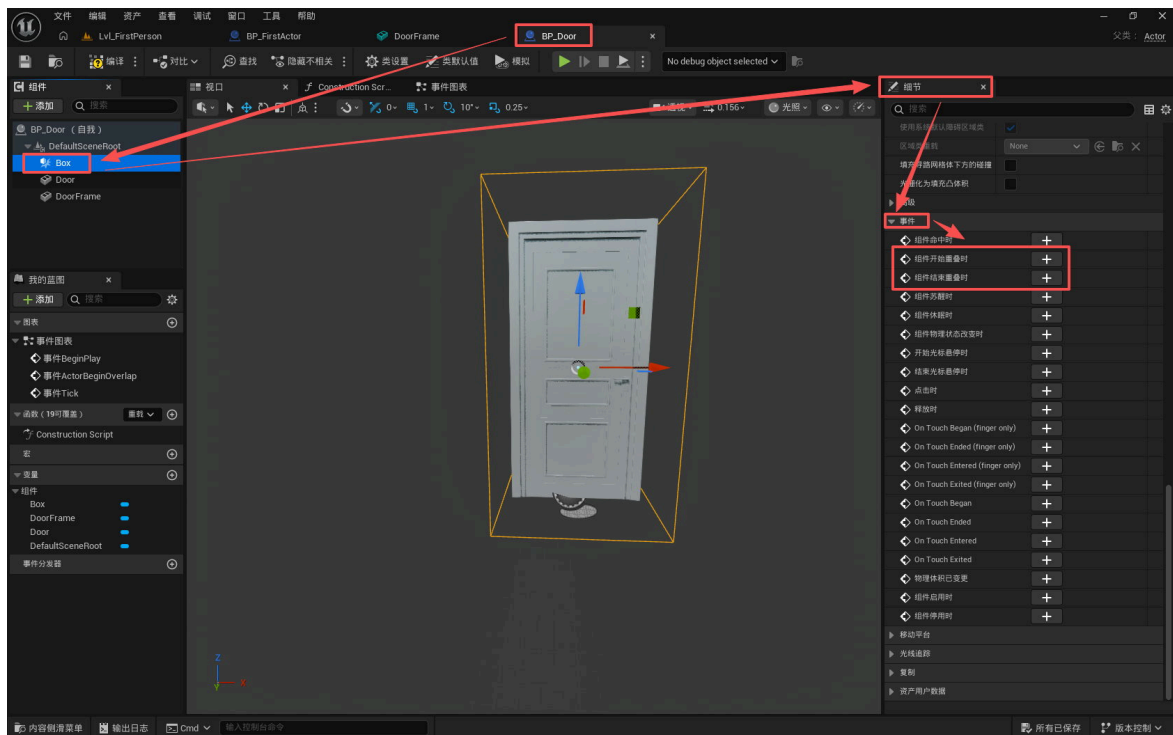


- 如图，盒型碰撞实际为一个方形，当角色或模型进入方形范围内就会发生某种响应，触发一些逻辑，因此需要将方形范围覆盖整个门
- 选中组件窗口中新增的盒型碰撞Box，在细节窗口的形状页中，调整盒体范围，虽然调整缩放也可以调整盒体范围，但**准标做法**就是调整盒体范围的数值
- **注意**：可以将鼠标放置在数值上，鼠标光标显示为左右箭头时，**左键长按**往左右**拉动**，往右为数值调大，往左为数值调小

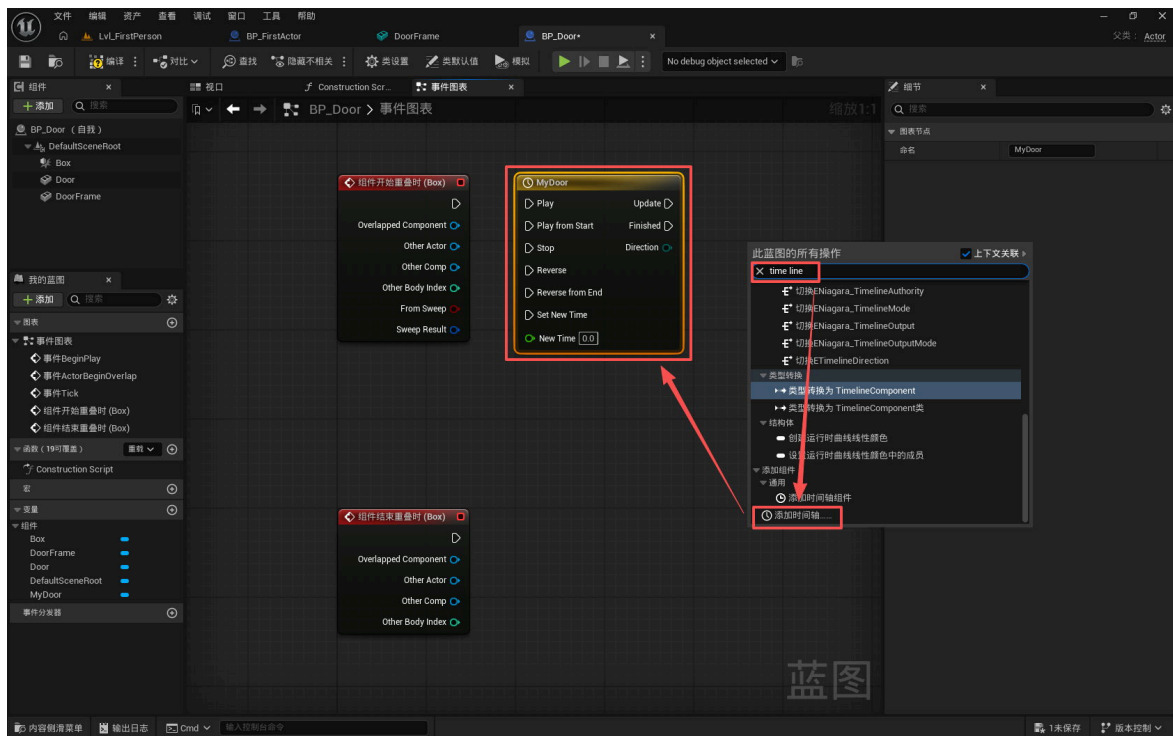


将盒型碰撞Box的细节页滑动到**最底部**，点击**组件开始重叠时**的右侧加号以及**组件结束重叠时**的右侧加号

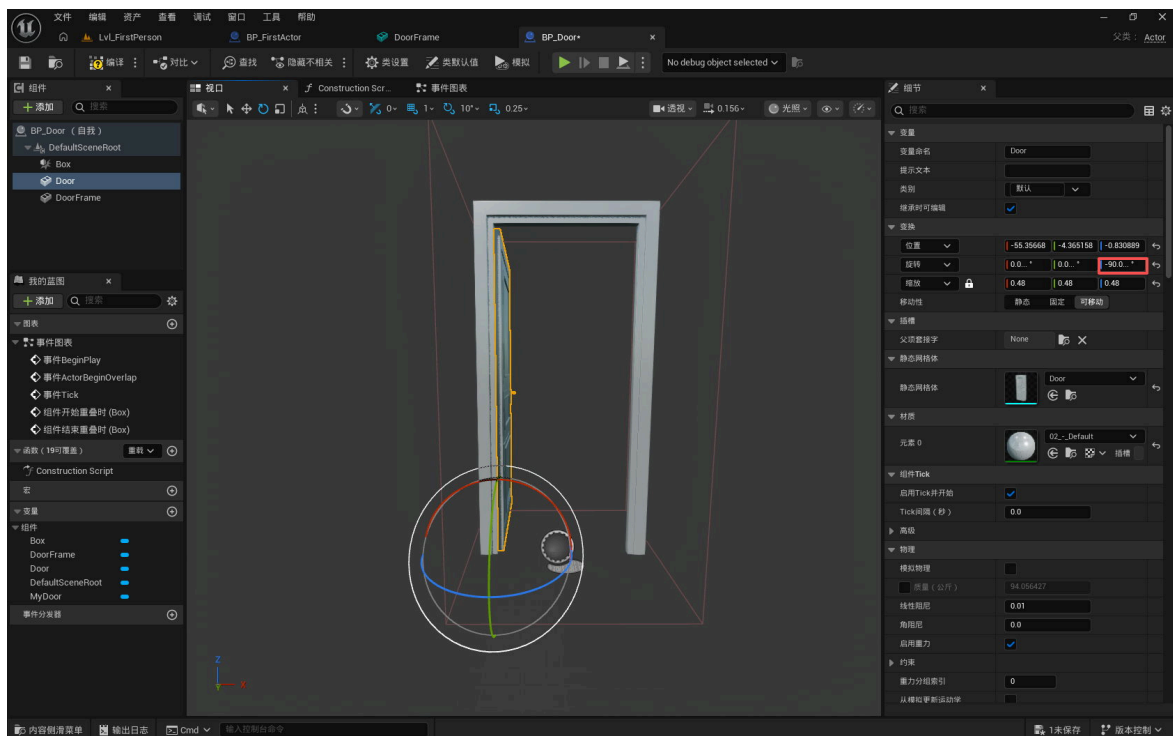
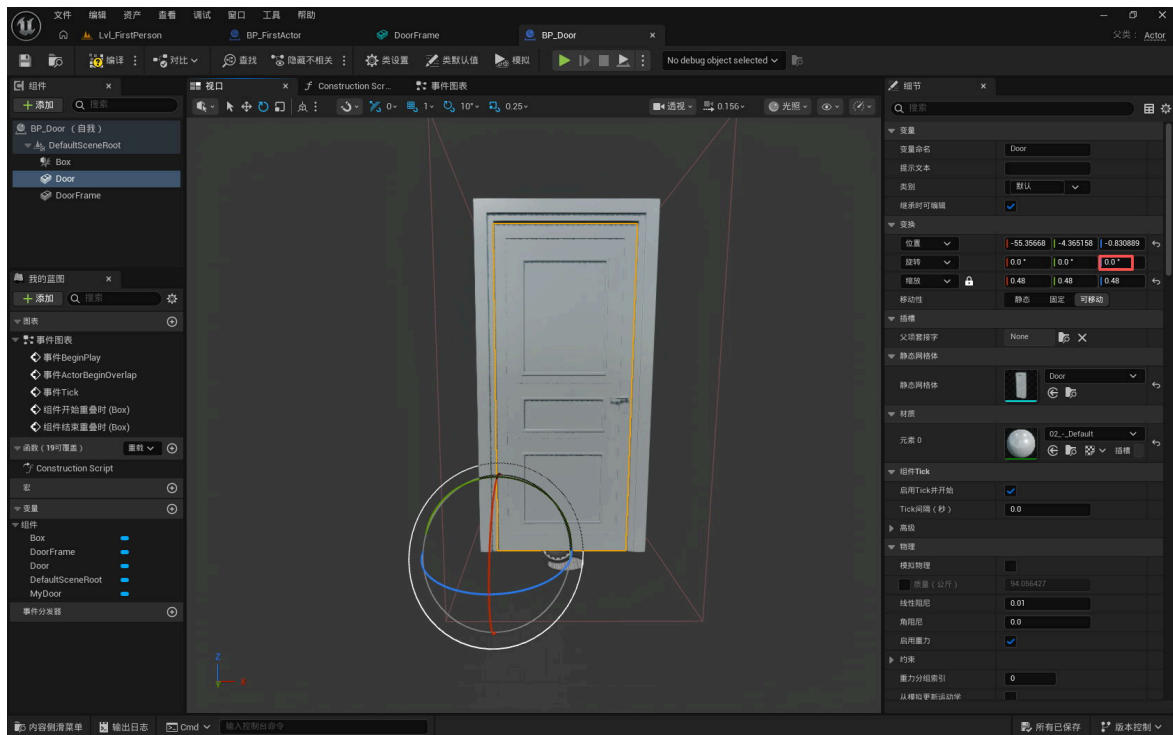
跳转到Box的事件图表，在事件图表中会新增**组件开始重叠时**和**组件结束重叠时**的节点，代表只要有模型**进入和离开**盒体范围，就会执行节点后续代码



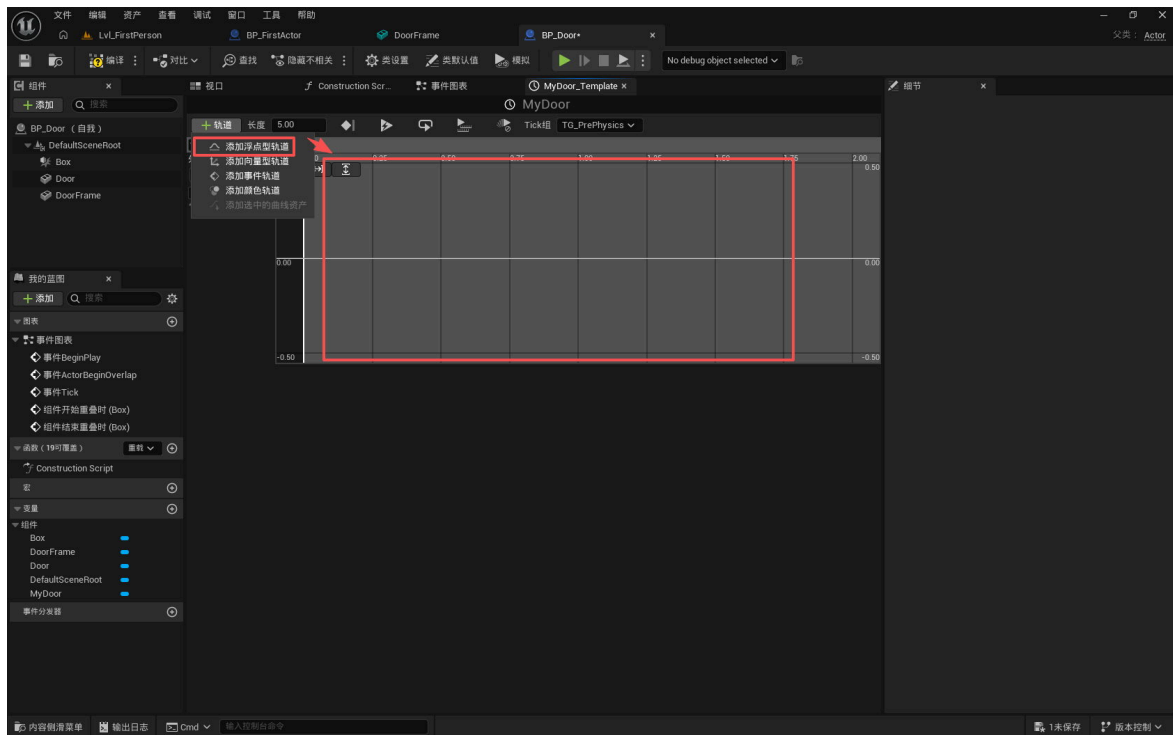
先将组件结束重叠时的节点移到一旁，在组件开始重叠时的节点右侧右键搜索输入time line，点击添加最底部的添加时间轴，命名为MyDoor



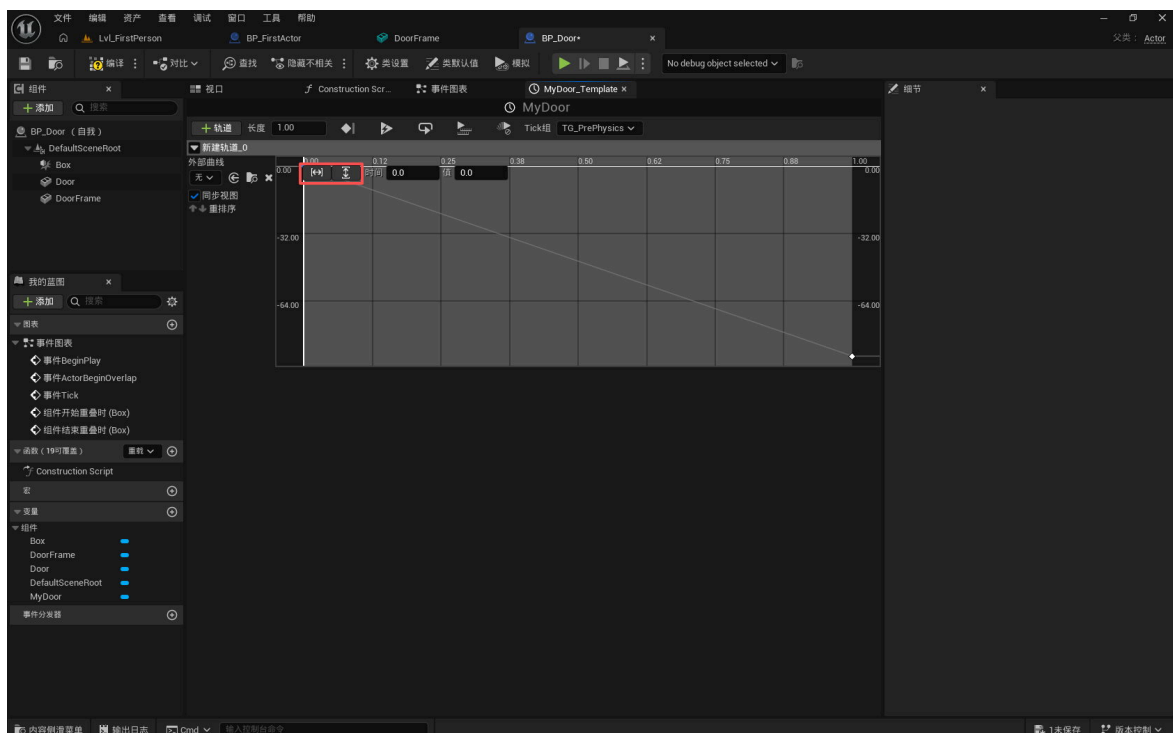
时间轴节点的主要作用是为了实现数值的线性增长与减小，而门的开与关，可以视为门沿z轴旋转，即关门状态代表z轴旋转0度，关门状态代表z轴旋转±90度（视实际情况为准）



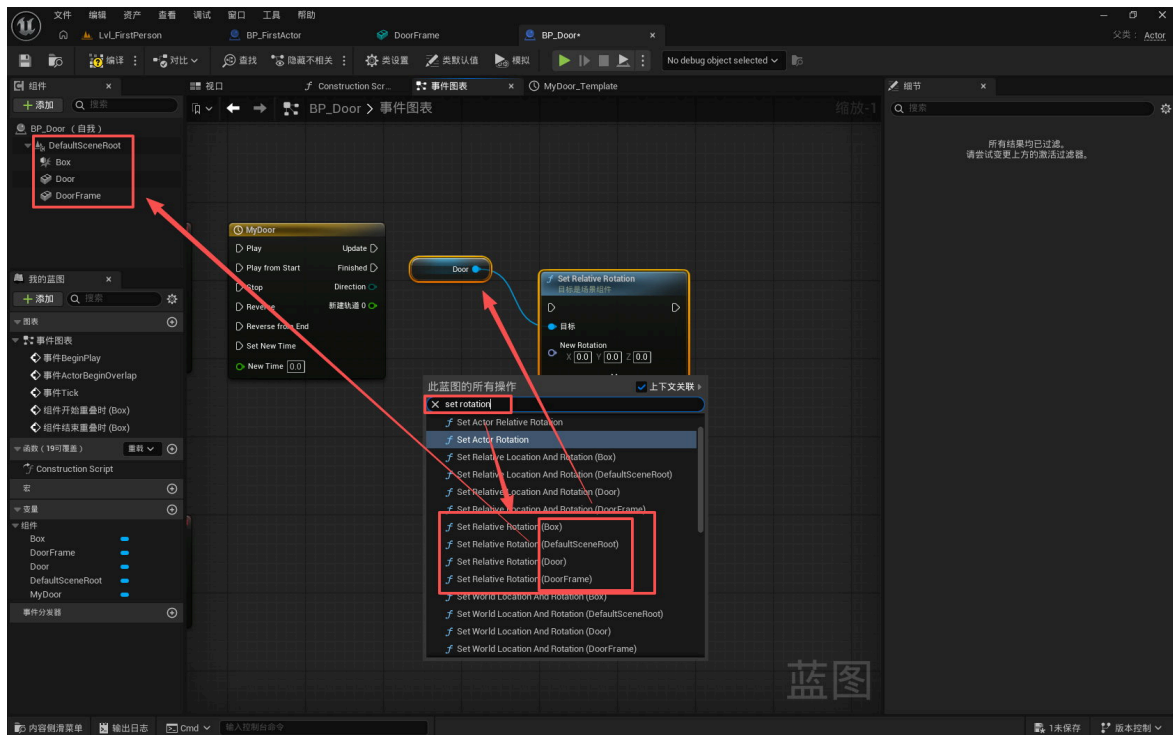
打开BP_Door的事件图表界面，左键双击新建的时间轴MyDoor节点，在进入的详情界面内左键单击+轨道



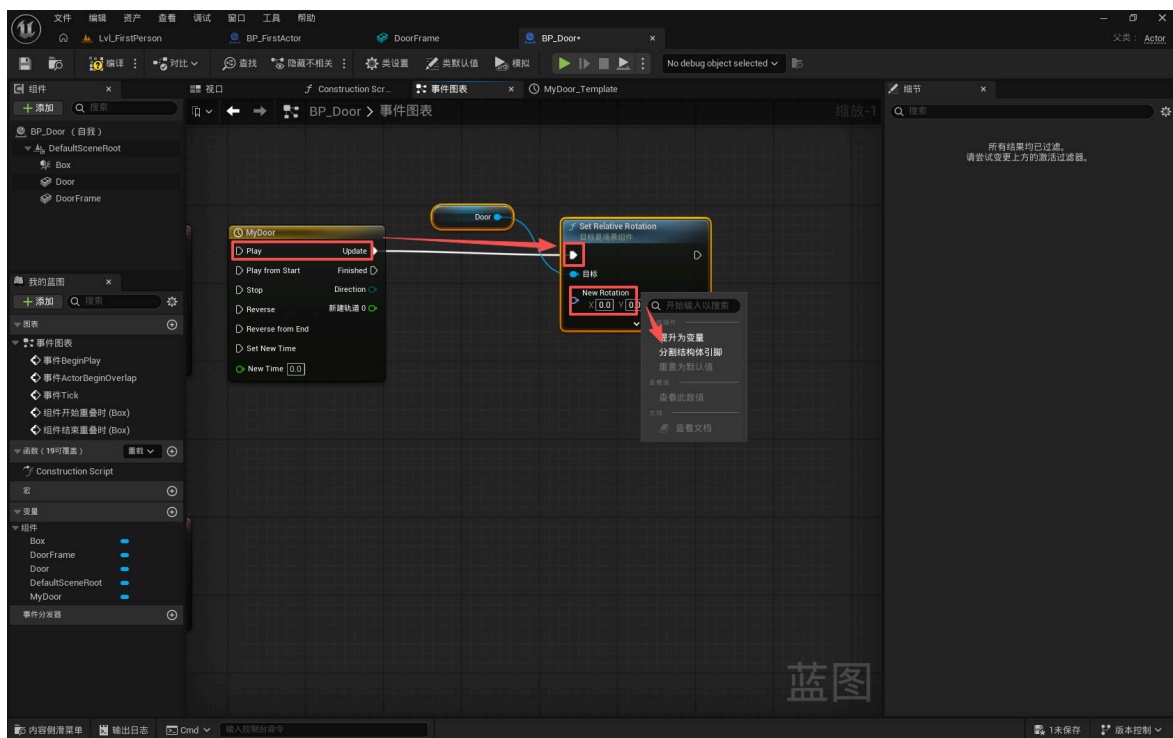
- 在新建的轨道界面内右键单击新建关键帧**两次**，在第一个时间输入框内输入0，对应值也为0，第二个时间输入框内输入1，对应值为-90，代表从0s到1s时间内，数值从0到-90线性减少，同时修改轨道右侧**长度**输入框内数值为1，对应关键帧的总时间长度
- 输入之后默认会跳转到无法完整观察到两个关键帧的部分，**键盘F键**或**左键单击**轨道界面左上角**缩放进行水平和垂直匹配**两个按键，就可以快速定位到已有的关键帧的部分
- 鼠标**左键单击**首尾帧任意一个点，即可对对应关键帧进行编辑
- **注意**：在输入时间和值之后，需要**按键盘上的enter键**，将已输入的数进行保存，不可直接将鼠标点击到别处，不按enter，如果不按，输入的数将无法保存到对应输入框中



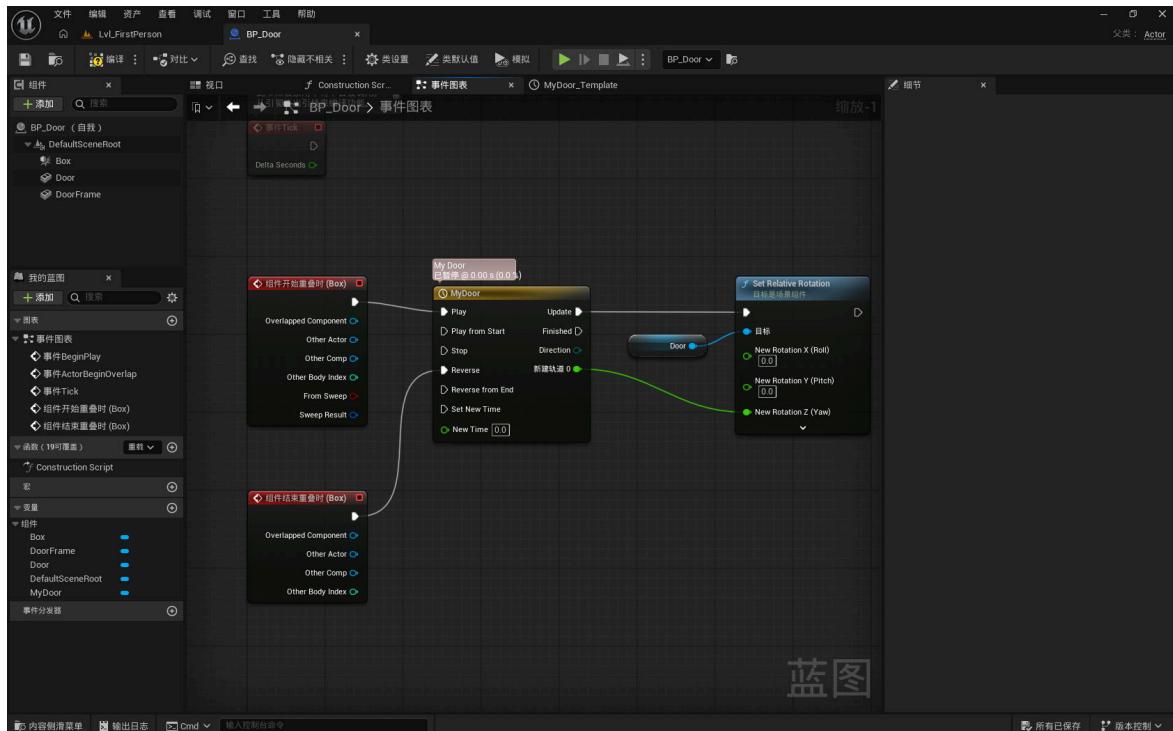
- 返回BP_Door的事件图表界面，在时间轴MyDoor节点旁右键搜索set rotation，选择 **Set Relative Rotation(Door)**，设置相对旋转（Door）
- 其他设置相对旋转与左侧组件内资产一一对应，新增的节点区别仅在**初始连接的节点**不同
- 如果选错了，可以将初始连接的左侧节点删除，从**组件处**左键拖动对应节点到事件图表内，将右侧引脚左键长按连接到Set Relative Rotation节点的目标引脚；或者直接将**组件处**左键拖动对应节点到Set Relative Rotation节点的目标引脚，可以直接连接



将时间轴MyDoor节点的update引脚左键长按连接到Set Relative Rotation节点的左侧执行引脚处，因为仅需要对Z轴进行旋转，所以右键New Rotation，选择分割结构体引脚，将Z轴独立出来



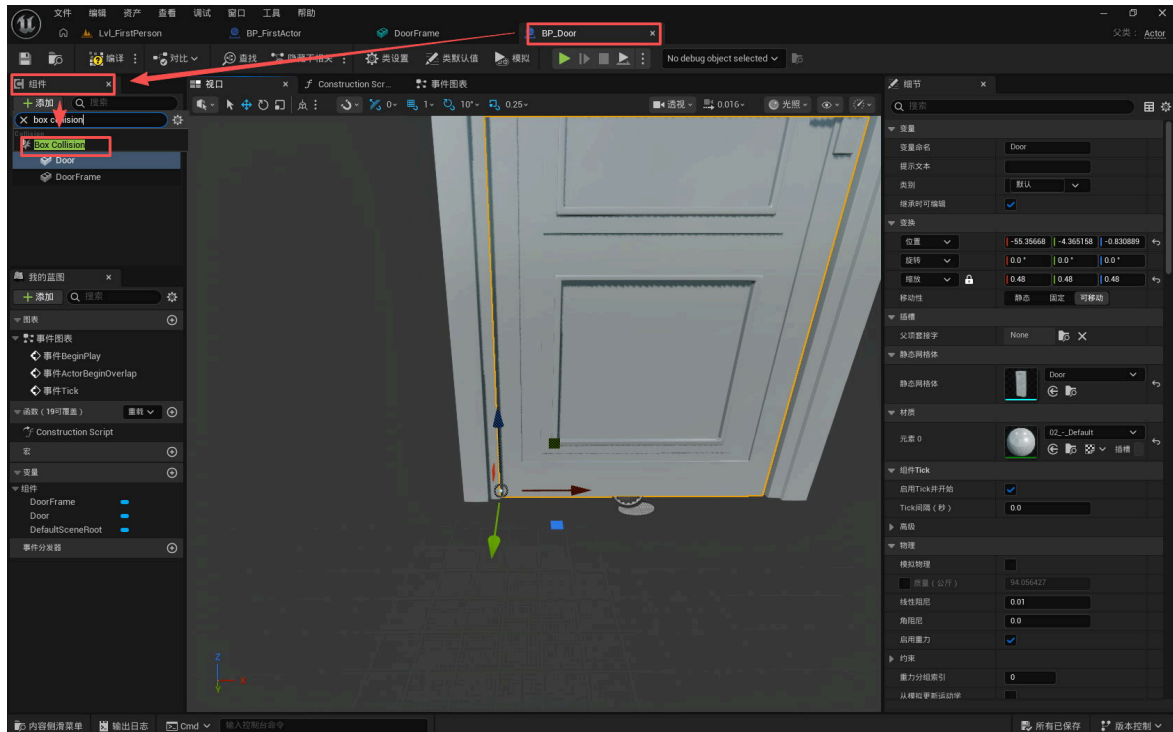
- 将时间轴MyDoor的Reverse 新建轨道0引脚左键长按连接到刚刚分割出的New Rotation Z引脚处
- 将组件开始重叠时的执行引脚左键长按连接到时间轴MyDoor的执行引脚
- 将组件结束重叠时的执行引脚左键长按连接到时间轴MyDoor的Reverse引脚，代表相反执行，会从时间轴的1s逆向执行到0s
- 左键单击左上角编译和保存，返回关卡界面，将BP_Door左键长按拖动到关卡界面内，单击运行或Alt P，检查运行效果



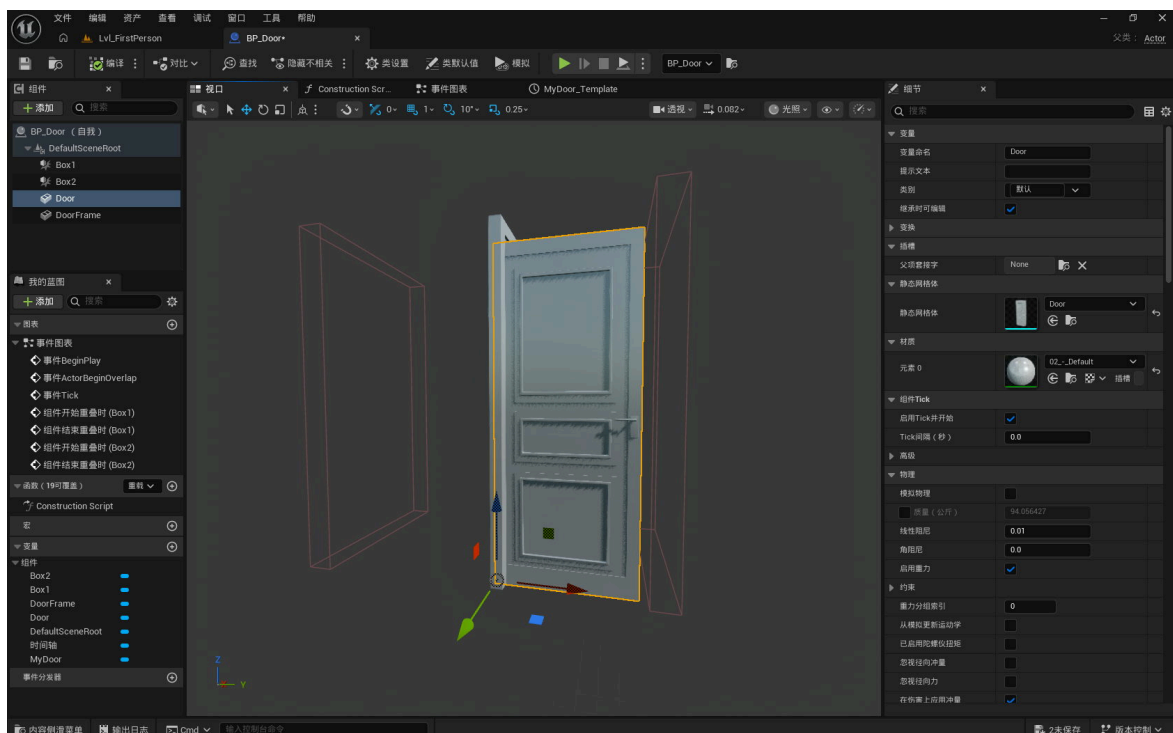
6. 编写双向自动门

单向门自动开合会存在角色反向进门会被开合的门模型卡住等问题，因此可以编写双向穿过门时，门都向外开关的双向自动门

首先，需要在BP_Door的视口界面额外再添加一个盒体碰撞，命名为Box2，为了区分，可以将原来的盒体碰撞重命名（按F2或右键）为Box1

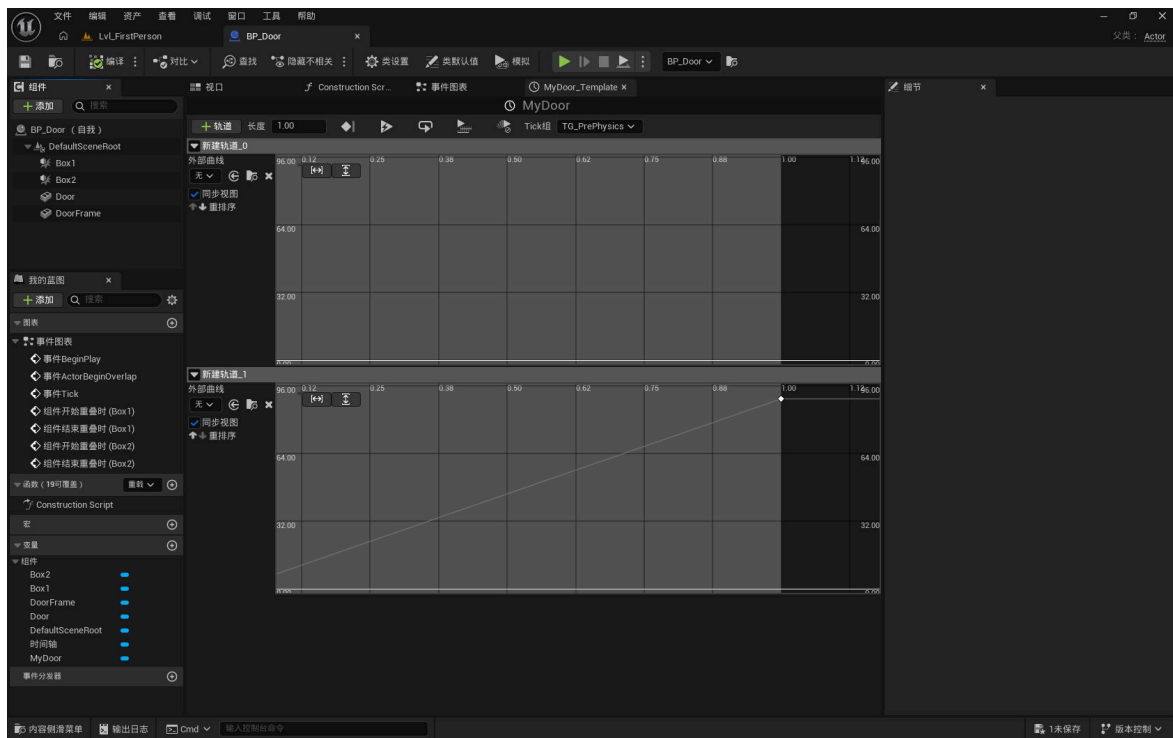


如图，可以将门的旋转角度设置为 ± 90 度，将两个碰撞体的位置设置在门开关的模型之外，因为本教程设置门是向角色反方向开关，如果碰撞体在门旋转 ± 90 度模型以内，会出现时间轴被打断，门开关抽搐的情况



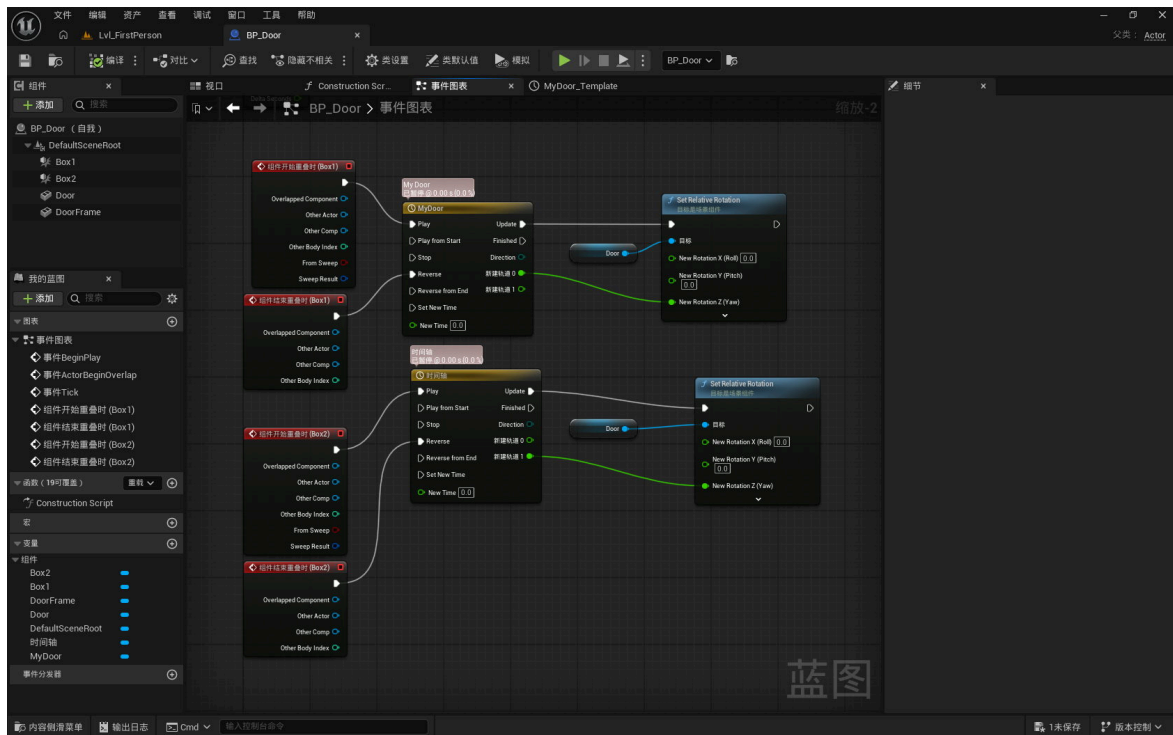
双击事件图表中的时间轴MyDoor节点，进入MyDoor_Template，如图，继续新增一条轨道，在新增的轨道中新增两个关键帧，分别为时间0，值0，时间1，值90

(与第一条轨道值相反)



返回事件图表界面：

1. 在**组件**窗口左键单击新增的Box2，在右侧**细节**窗口底部选择**组件开始重叠时**和**组件结束重叠时**
2. 右键搜索输入time line，新增时间轴，右侧右键搜索输入set rotation，新增 **Set Relative Rotation(Door)**，右键New Rotation项，选择分割结构体引脚
3. 将Box2的组件开始重叠时的执行引脚连接到新增的时间轴左侧执行引脚，将Box2的组件结束重叠时的执行引脚连接到新增的时间轴左侧Reverse引脚
4. 将新增的时间轴右侧update引脚连接到新增的Set Relative Rotation(Door)的左侧执行引脚，将新增的时间轴的新增轨道连接到Set Relative Rotation(Door)的New Rotation Z
5. 点击左上角**编译**和保存后，返回关卡界面，单击运行或者Alt P，检查运行结果，如果觉得开门触发的太早，可以进入BP_Door的视口界面，**调整Box1和Box2盒体范围的Y数值**



7. 时间轴引脚知识补充

- **Play**: 顺序播放时间轴设定的关键帧对应的数值，如果时间轴播放未完成，继续播放会延续上次未完成的时间轴继续播放
 - 举例：角色走到箱体碰撞范围时，门会从**当前角度**线性过渡到 ± 90 度之间，表现为门丝滑打开
- **Play from Start**: 顺序播放时间轴设定的关键帧对应的数值，如果时间轴播放未完成，继续播放从头开始播放
 - 举例：角色每次进入箱体碰撞范围时，门会闪现到完全关闭的状态再次打开
- **Stop**: 不管时间轴进行到哪里，立即停止
 - 举例：如果将组件结束重叠时，也就是角色离开碰撞范围连接到**Stop**，表现为角色远离门后，门不继续关闭，并且后续门不会执行关闭，只会继续打开
- **Reverse**: 倒序播放时间轴设定的关键帧对应的数值，如果时间轴播放未完成，继续播放会延续上次未完成的时间轴继续倒序播放
 - 举例：角色离开箱体碰撞范围时，门会从**已打开的角度**倒序过渡到0度，表现为门丝滑关闭
- **Reverse from End**: 倒序播放时间轴设定的关键帧对应的数值，如果时间轴播放未完成，继续播放从最后开始播放
 - 举例：角色每次离开箱体碰撞范围时，门会闪现到完全开门的状态再次关闭
- **Set New Time**: 设定时间轴上一个指定的播放时间，联动下方的**New Time**数值



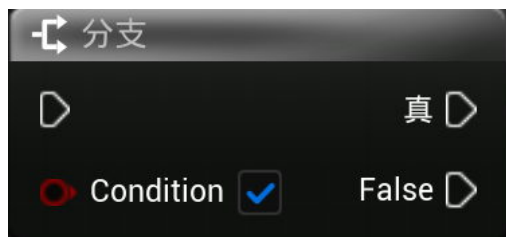
六、常用流程控制节点

1. 流程控制节点定义

控制蓝图执行顺序和逻辑分支的节点，类似于编程中的if-else和循环语句

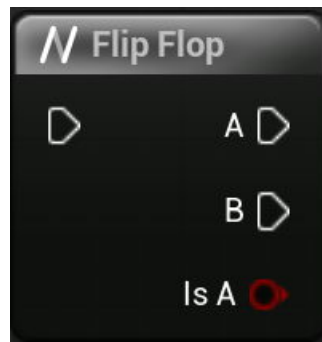
2. Branch

- 左下角Condition引脚为布尔输入值，左上角为执行引脚，类似于触发器
- 右侧真和False为输出引脚，分别对应输入值真假进行输出
- 类似于编程中的if-else



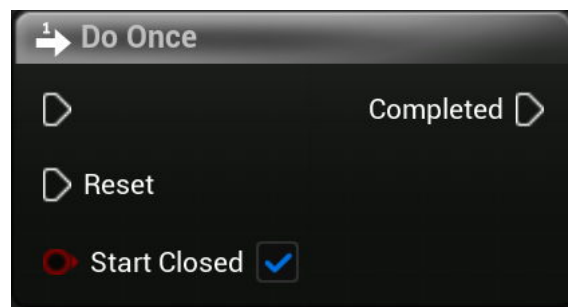
3. Flip Flop

- 左上角为执行引脚，右侧为三个输出引脚
- 输入执行，输出会在两个执行输出间切换：第一次被调用时，将会输出A，第二次被调用时，将会输出B，循环往复
- 右下角为布尔变量输出，当输出A节点时输出真



Do Once

- 左上角为执行引脚，右侧为输出引脚
- 在左上角执行引脚输入后，右侧输出**仅执行一次**，后续继续输入执行引脚，右侧不再输出
- 在触发左侧Reset引脚之后，左上角执行引脚被重置，可以再次触发右侧输出
- 左下角Start Closed勾选后，会默认关闭执行引脚的状态，表现为勾选Start Closed后，直接输入执行引脚没有输出，仅在触发Reset引脚后，执行引脚可正常执行一次



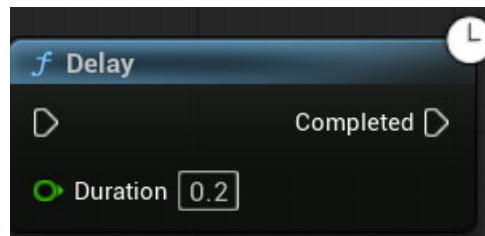
4. Sequence

- 左侧为执行引脚，右侧为输出引脚，点击添加引脚可添加输出引脚数量
- 在左侧执行引脚输入之后，会按顺序输出右侧输出引脚，类似于编程中的switch



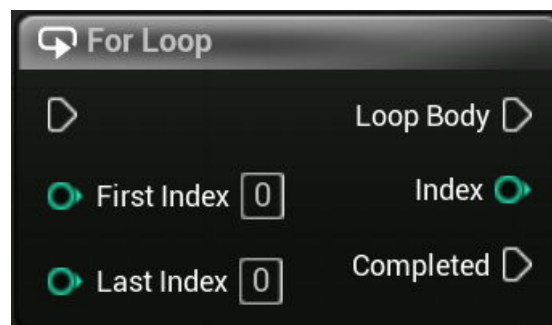
5. Delay

- 左上角为执行引脚，右侧为输出引脚
- 下方Duration输入的数值为右侧输出延迟的时间，单位为秒



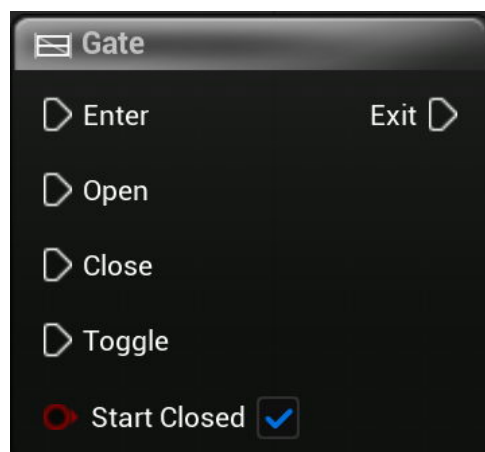
For Loop

- 左上角为执行引脚，右上角为输出引脚
- 左侧**First Index**代表循环起始数，数值类型为整数，代表不能设置浮点数，**Last Index**代表循环终止数，循环会从起始数一直持续到终止数，举例，**First Index**为0，**Last Index**为5，那么循环会**持续6次**，从0循环到5
- 右侧**Index**会输出当前循环所在的循环数，**Completed**引脚会在循环结束时执行后续节点



6. Gate

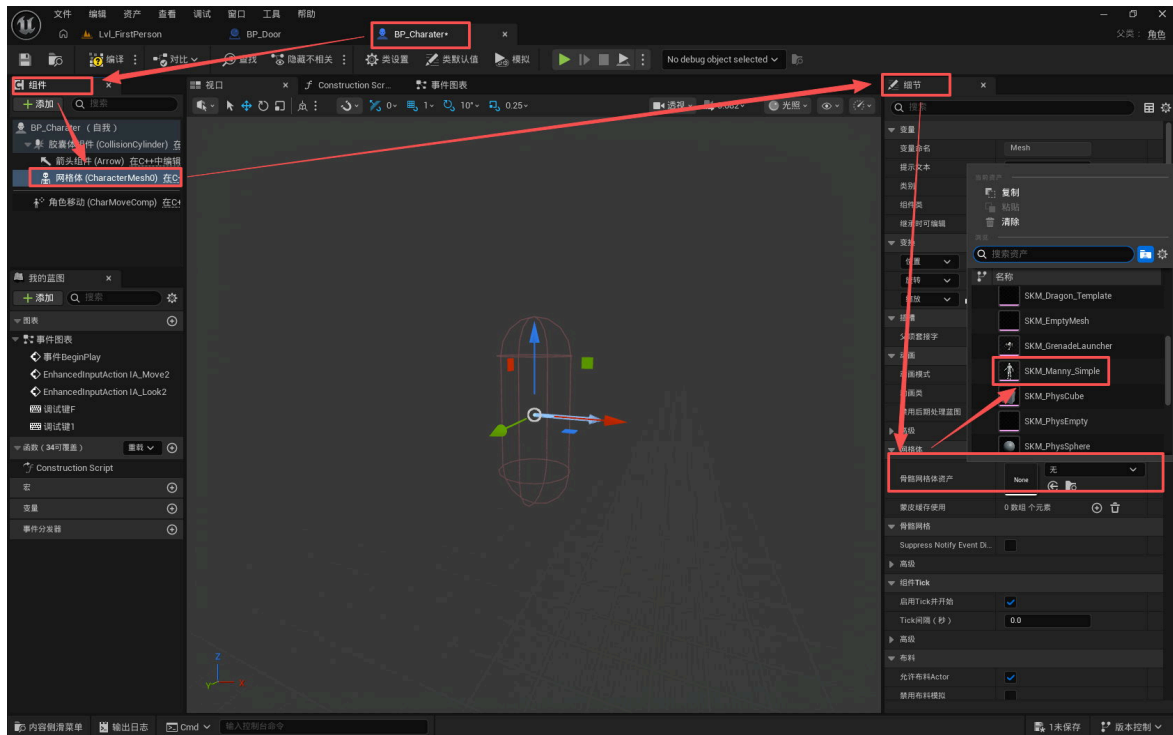
- 左侧**Enter**为执行引脚，右侧**Exit**为输出引脚
- 因为**Start Closed**默认勾选，所以输入执行引脚后，不会有输出，默认为关闭状态
- 在输入**Open**引脚后，输入**Enter**引脚会有正常输出
- 在输入**Close**引脚后，输入**Enter**引脚没有输出
- 输入**Toggle**引脚后，会在**Open**和**Close**两个状态之间进行切换



七、制作第三人称运动系统

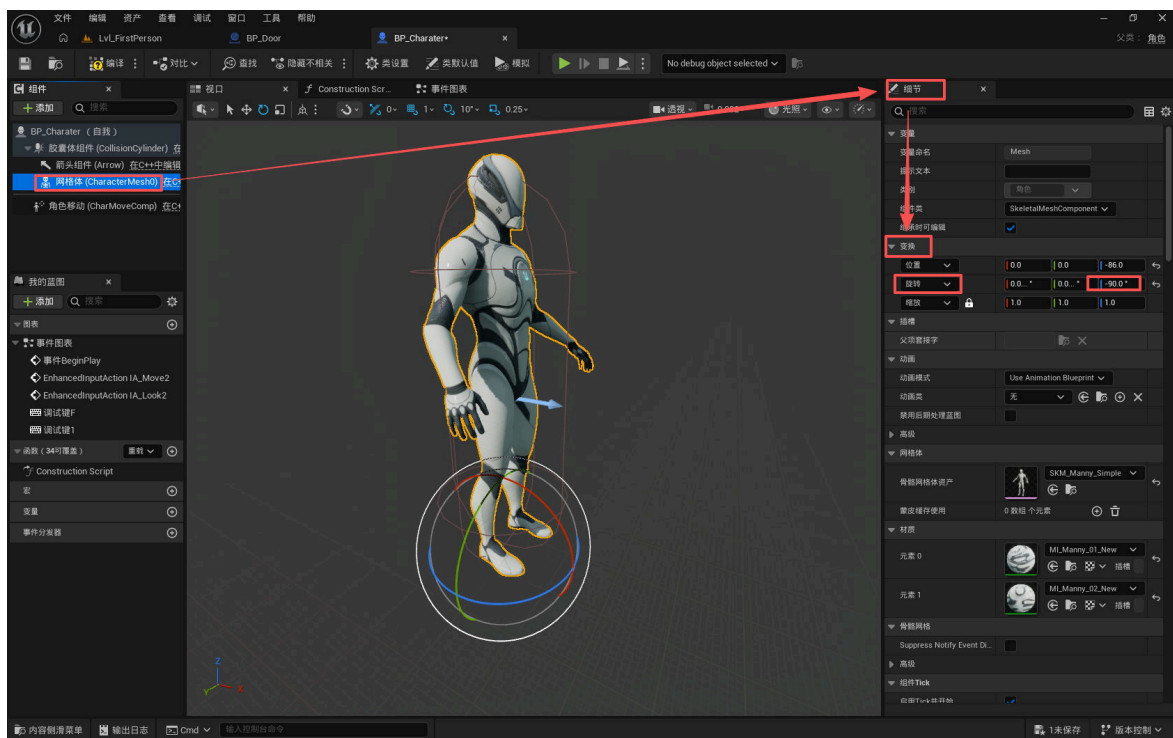
1. 添加角色模型

- 双击内容侧滑菜单的**BP_Character**，进入BP_Character的视口界面，此时胶囊体内为空，而第三人称相关需要角色模型
- 选中组件窗口的网格体，在右侧细节窗口内的**骨骼网络体资产**中选择SKM_Manny_Simple（Skeletal Mesh 骨骼网络体）
- 骨骼网络体可以理解为**带动画的模型**，静态网格体为不带动画的模型



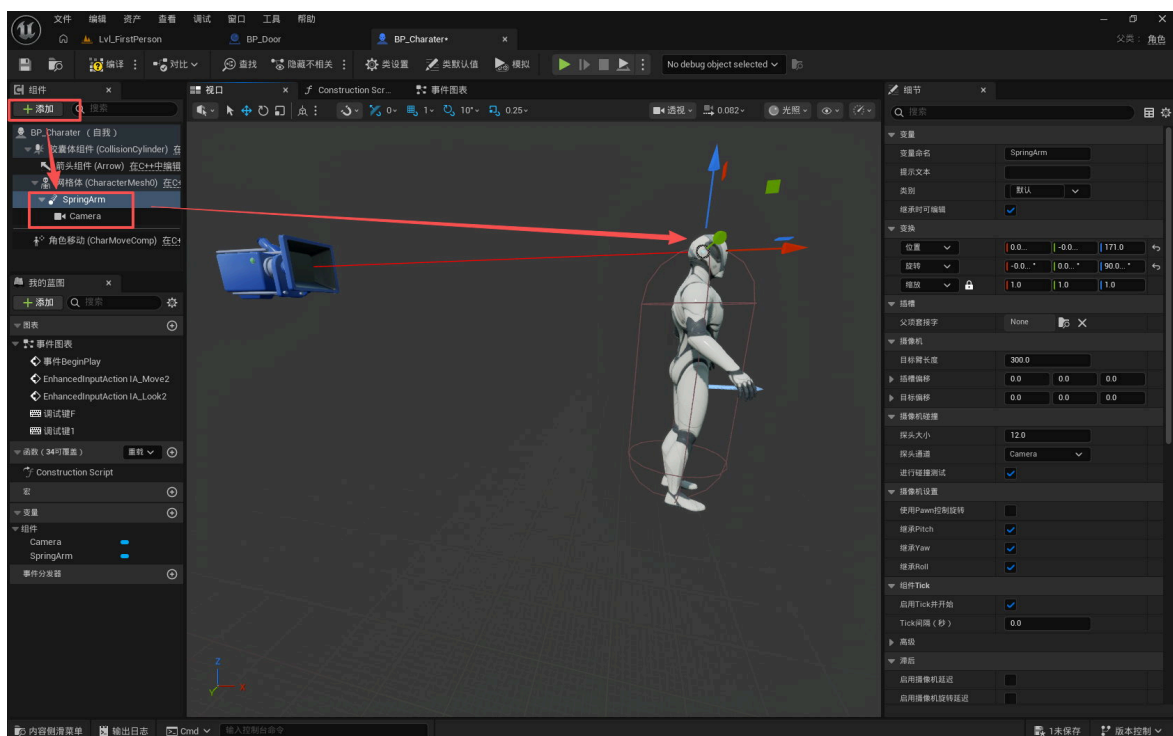
2. 调整角色模型

- 胶囊体为角色碰撞范围，选中后将模型向下拉，拉到胶囊体内
- 胶囊体内存在蓝色箭头，指向方向为角色面向的正前方，因此需要旋转模型到蓝色箭头方向
- 在网格体的细节窗口的变换中，直接在旋转的Z轴输入-90完成朝向调整
- 点击左上角**编译和保存**后完成调整



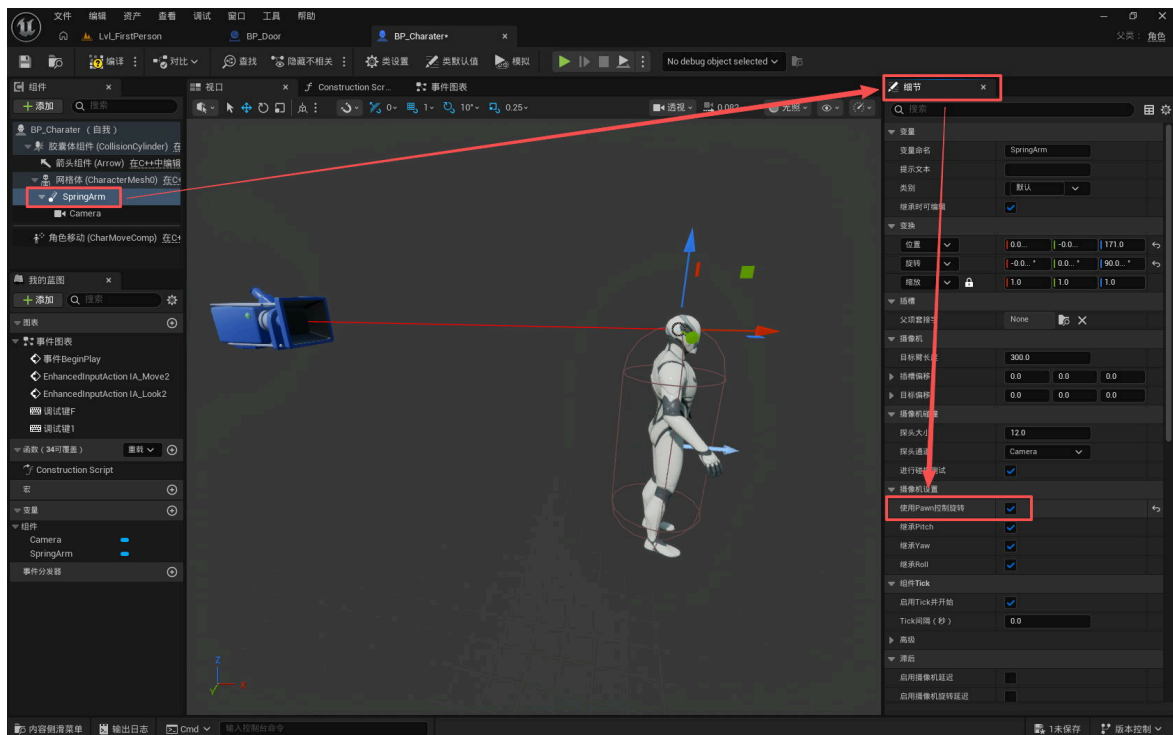
3. 设置第三人称视角

- 在组件窗口单击新增，搜索选择 **弹簧臂组件** 和 **摄像机组件**
- 将新增的摄像机组件左键长按到弹簧臂组件上，设置摄像机组件为弹簧臂组件的子类
 - 选中弹簧臂再新增摄像机组件，摄像机组件新增即为弹簧臂子类
- 在视口界面左键单击**弹簧臂组件**，按键盘w调整移动模式为平移，将弹簧臂组件平移到角色头部位置

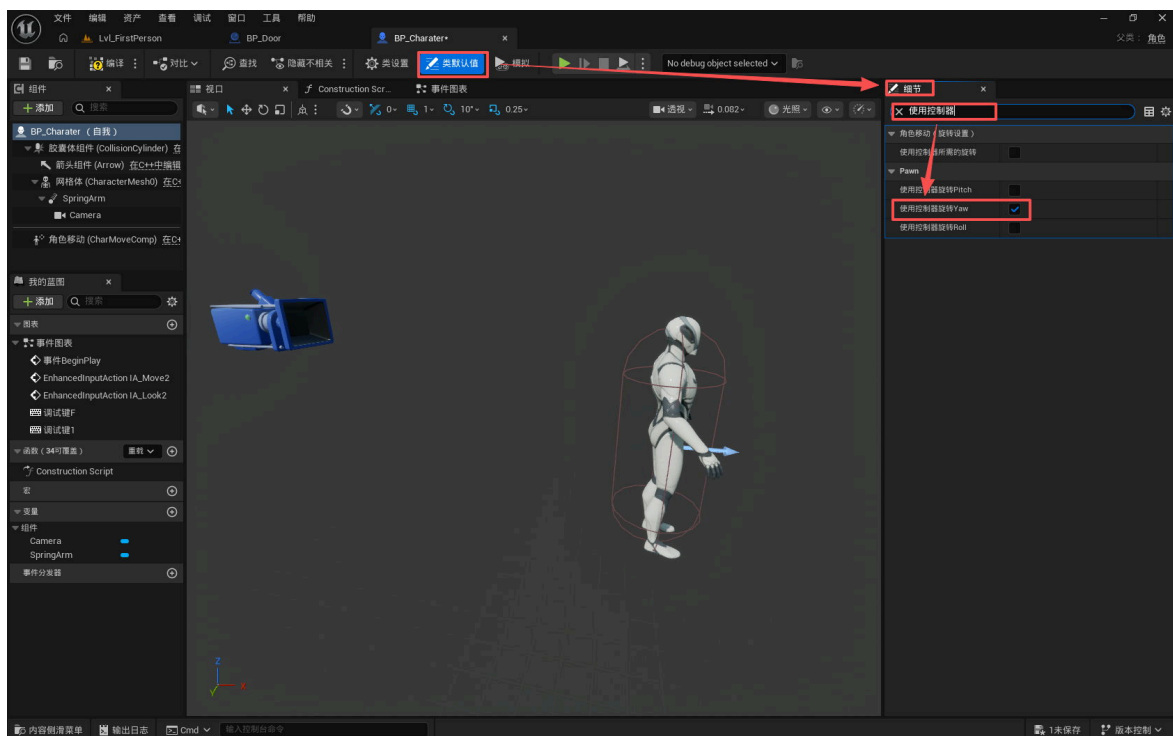


- 左键选择弹簧臂组件，在右侧细节窗口中的摄像机选择，勾选**使用Pawn控制旋转**

- 勾选后，可以实现使用鼠标自由移动上下纵向视角



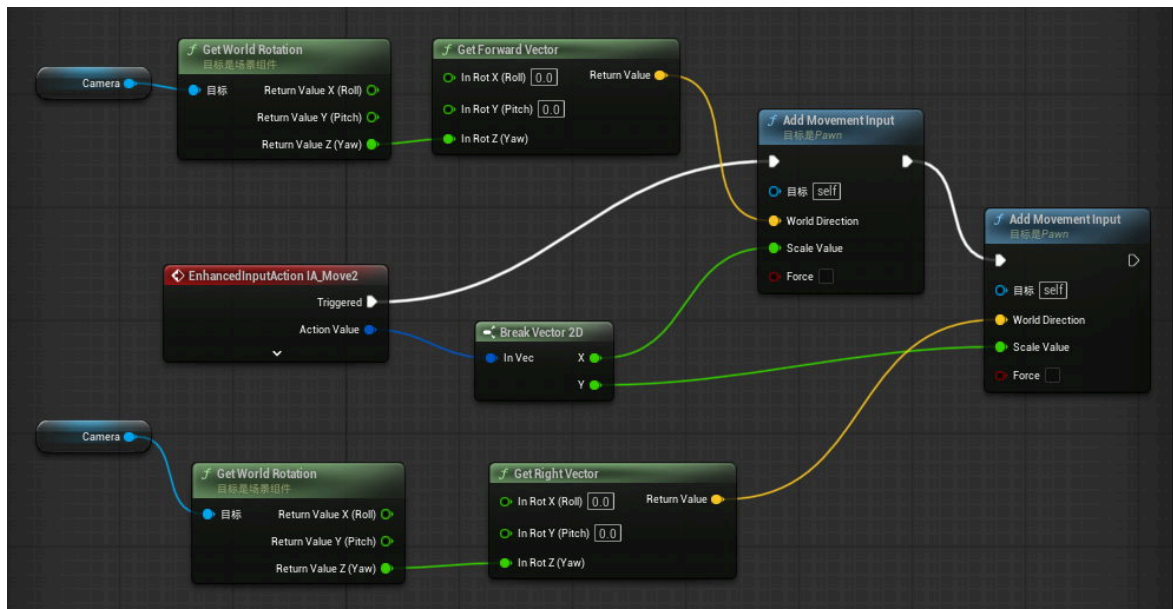
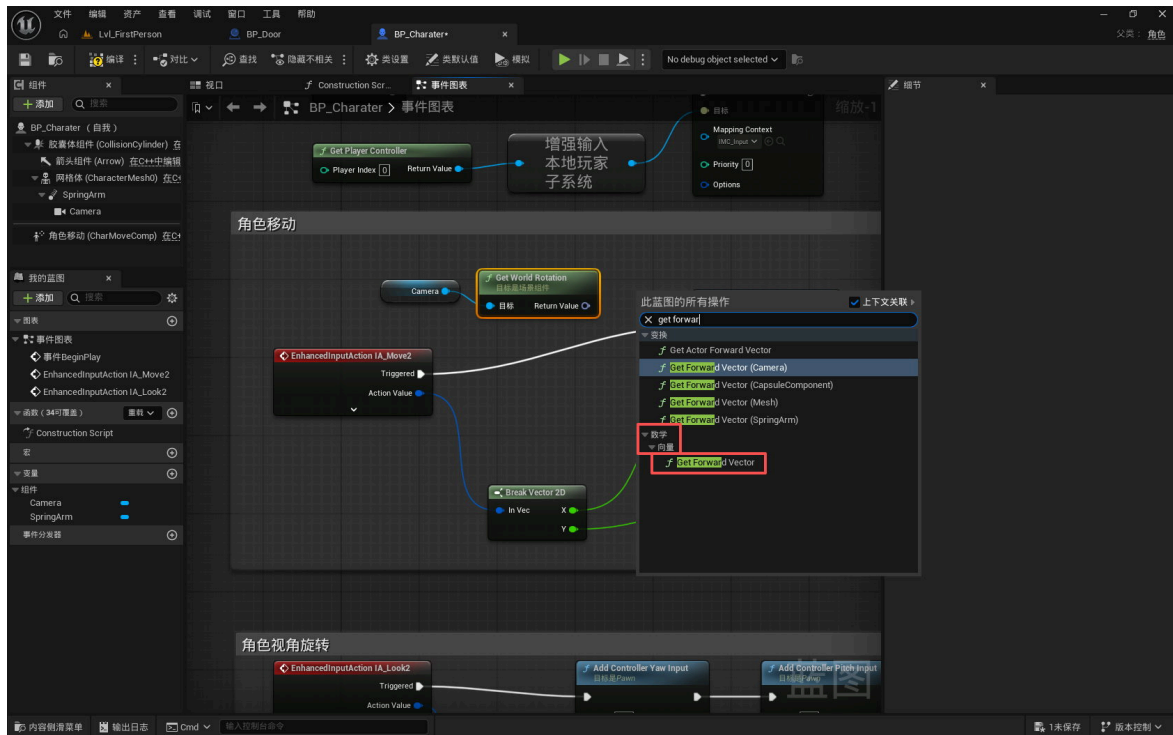
左键选择类默认值，在右侧细节窗口搜索输入使用控制器，取消勾选使用控制器旋转Yaw，此时鼠标可以自由移动左右横向视角，但是角色无法向视角朝向移动



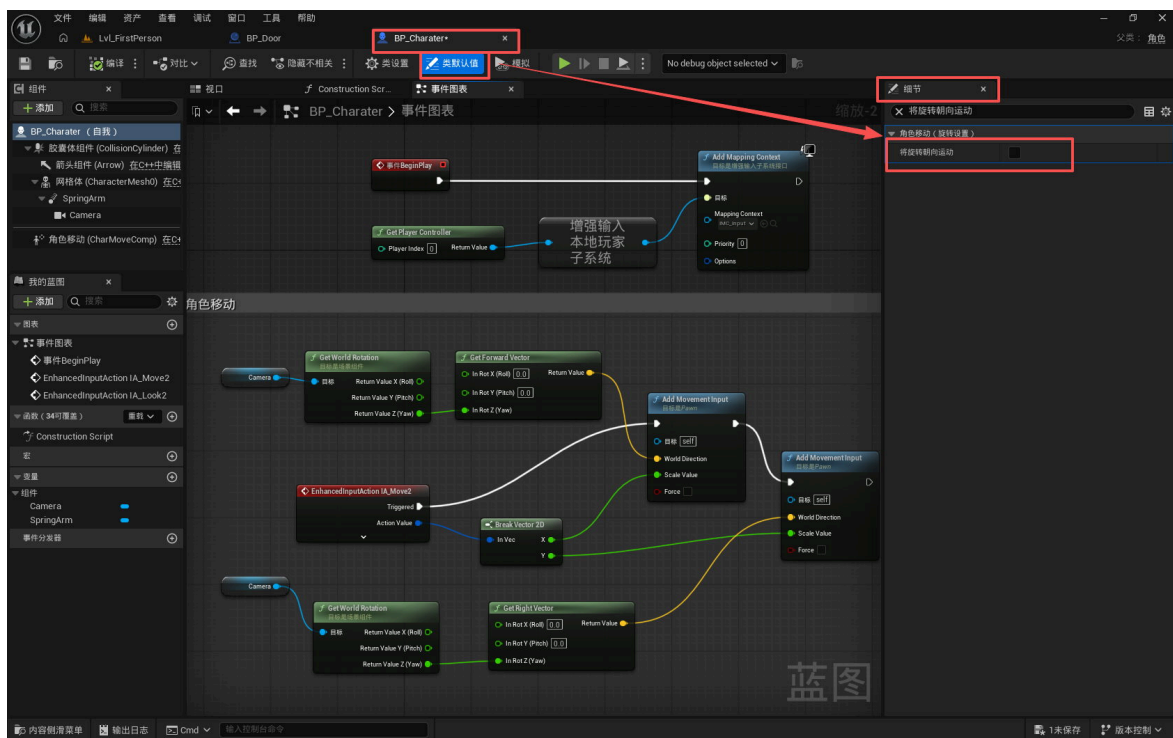
4. 编写第三人称视角

- 打开BP_Character的事件图表界面，观察原有的角色移动代码，编写逻辑为朝向角色面向的正前方移动，而现在的逻辑是向摄像机朝向移动
- 长按左侧组件窗口中的摄像机组件拖动到事件图表内，左键长按引脚搜索输入 **get world**，点击 **Get World Rotation**，右键获取世界旋转的 **Return Value**，选择分割结构体引脚

- 删除原有的获取向前向量，右键搜索输入**get forward**和**get right**，点击**数学-向量**中的 **Get Forward Vector** 和 **Get Right Vector**，右键in Rot选择分割结构体引脚
- 将获取世界旋转的Return value Z(Yaw)左键长按连接到分割后出现的in Rot Z(Yaw)，仅获取摄像机的左右转向，而当摄像机上下移动时不做处理，防止角色上天入地
- 点击**编译和保存**后完成摄像机朝向的移动蓝图编写



- 进入关卡界面点击运行或者Alt P，会发现此时角色已经向着摄像机朝向移动，但是角色本身没有进行转身或者旋转到朝向的动作
- 返回BP_Character，点击类默认值，搜索**将旋转朝向运动**，进行勾选
- 点击编译和保存，返回关卡界面，运行检查发现此时角色已经可以进行转向

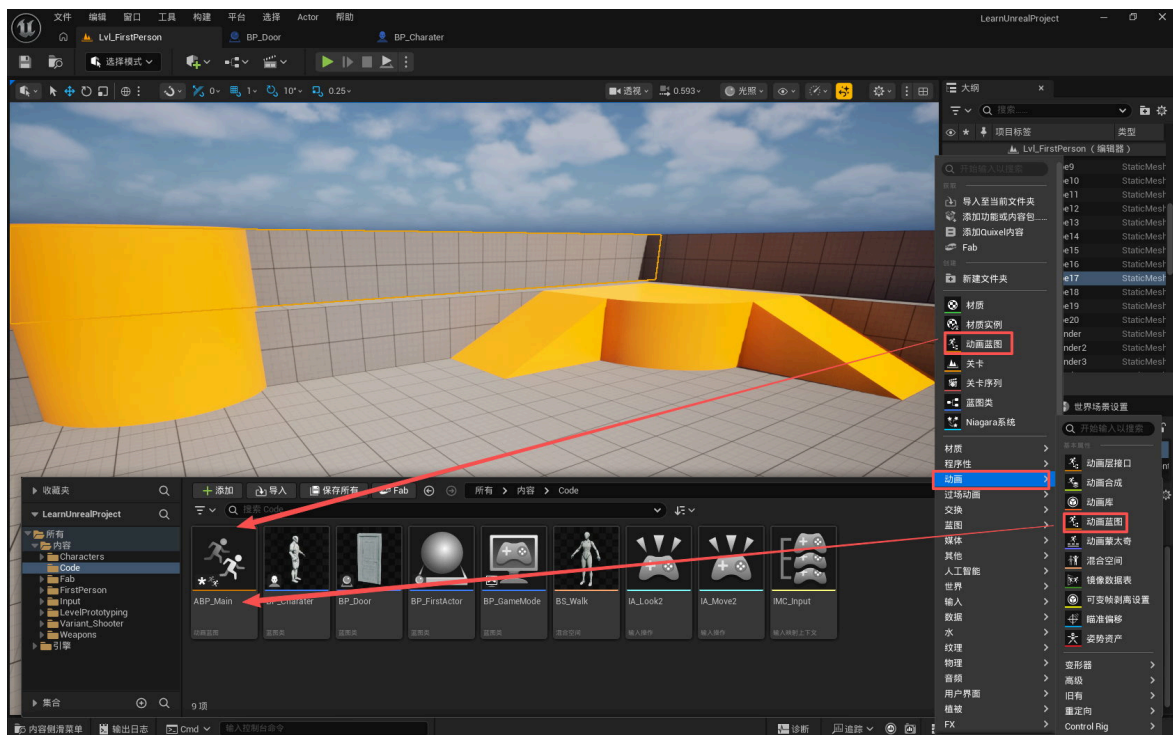


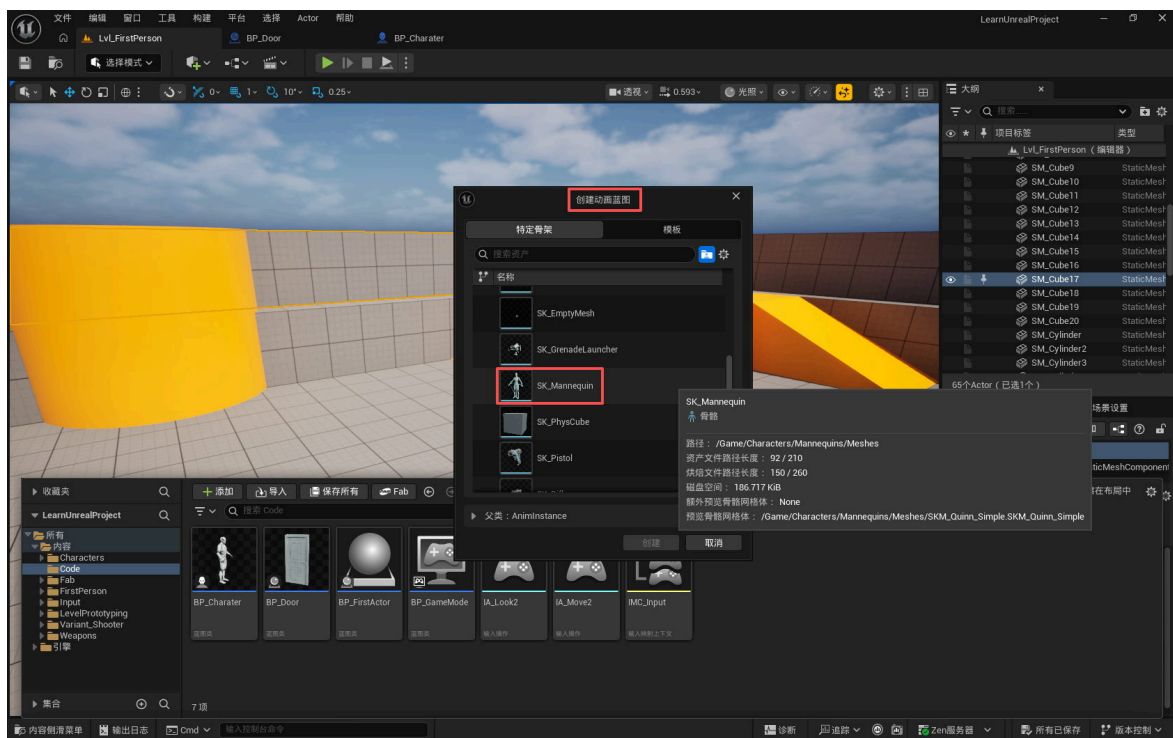
八、制作第三人称角色动画

1. 新增动画蓝图和混合空间

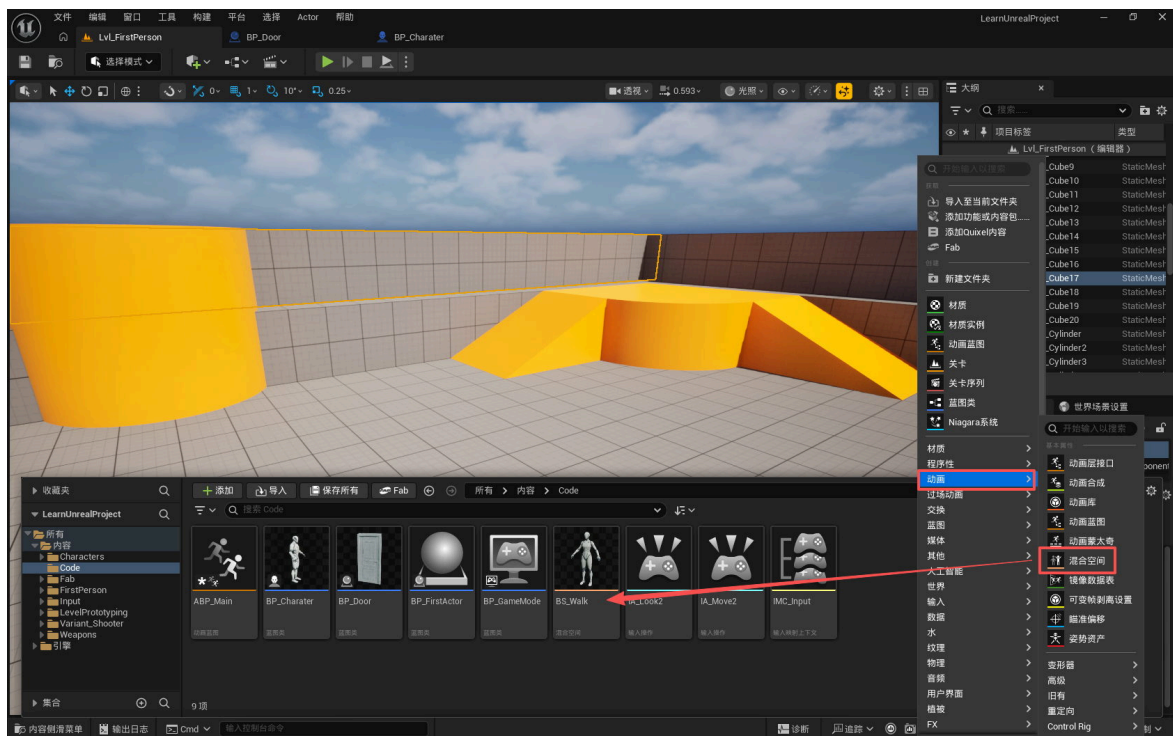
打开内容侧滑菜单，右键菜单中选择**动画蓝图**，或者选择动画-**动画蓝图**

在弹出的界面内搜索选择**SK_Mannequin**（Skeletal 骨骼的），命名为**ABP_Main**（Animation Blueprint 动画蓝图）



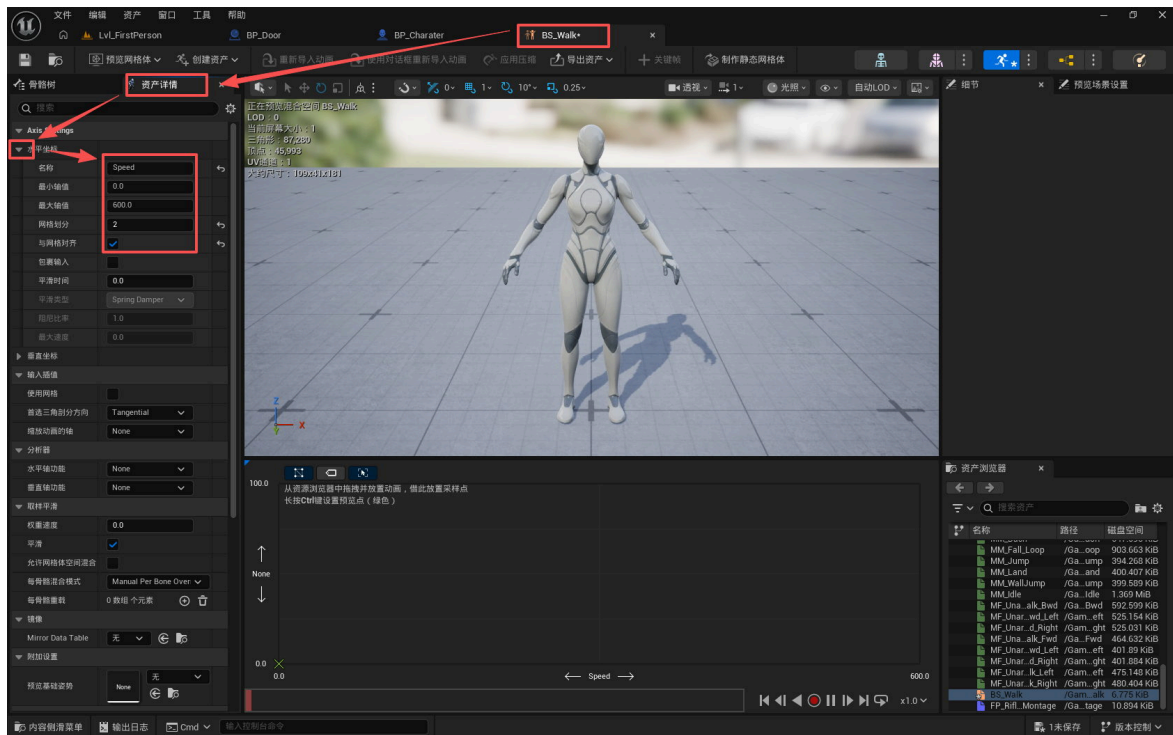


选择右键菜单-动画-混合空间，选择同样的骨骼SK_Mannequin，命名为BS_Walk（Blend Space 混合空间）

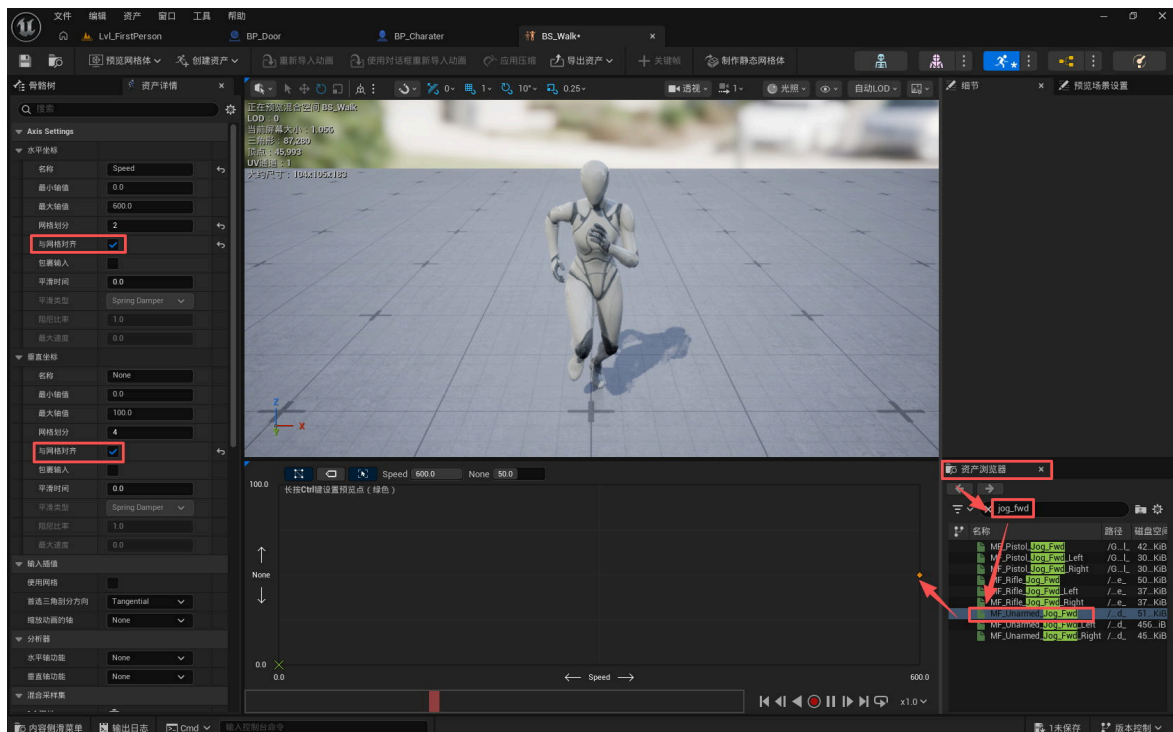


2. 设置混合空间

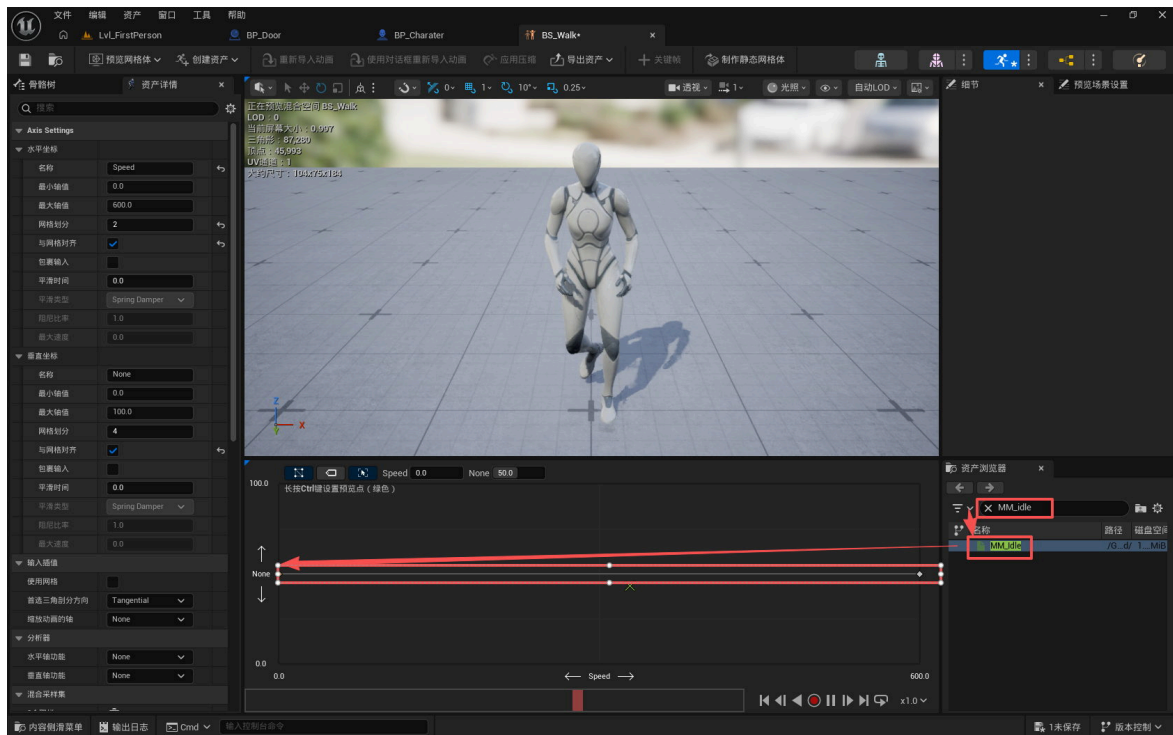
- 双击BS_Walk，在左侧资产详情窗口展开水平坐标与垂直坐标
- 将水平坐标的名称改为Speed，最大轴值改为600，因为角色最大移动速度为600，网格化算设置为2，勾选与网格对齐；勾选垂直坐标中的与网格对齐



- 在右下角资产浏览器中搜索输入**jog_fwd**，此处需要使用的是**MF_Unarmed_Joy_Fwd**动画，Unarmed代表是角色空手向前奔跑的动画，左键长按拖拽到中部采样点处**最右侧**
- 注意：**此处动画需要项目为一开始的模板项目，如果是自建项目，不会包含自带动画

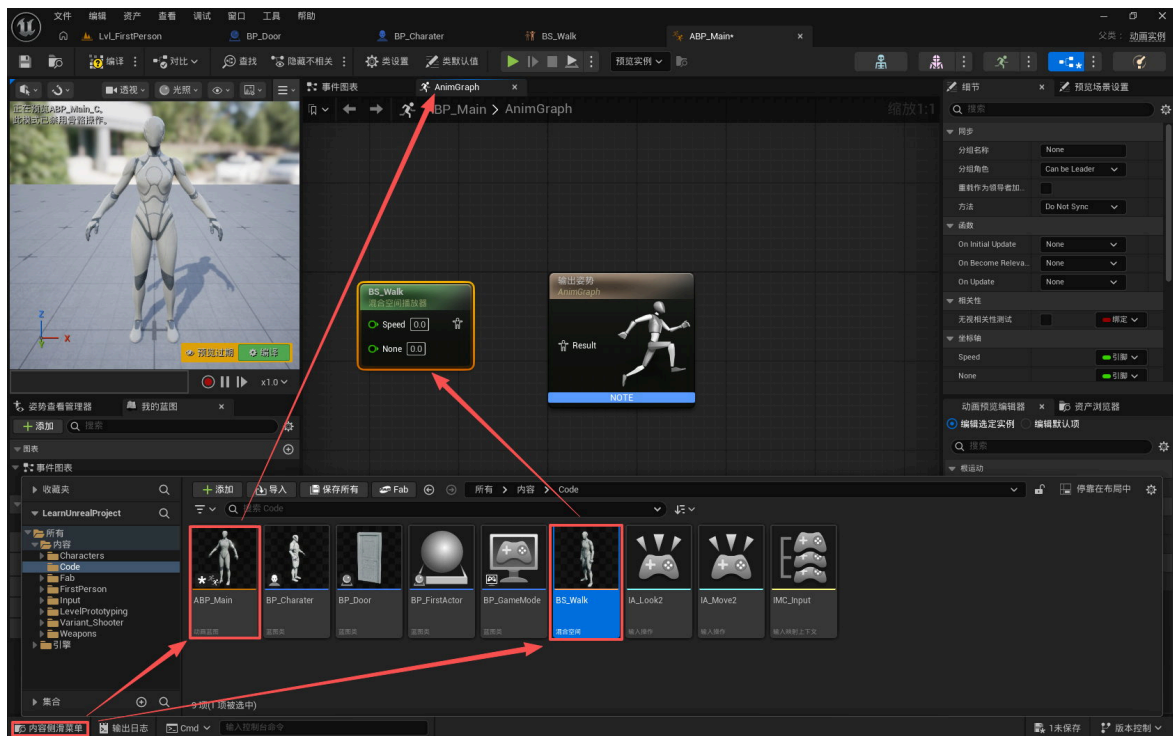


- 在右下角资产浏览器中搜索输入**MM_idle**，左键长按拖拽到中部采样点处**最左侧**，并且保持和MF_Unarmed_Joy_Fwd水平
- 按住Ctrl键，鼠标在采样点左右移动，可以看到两个动画的过渡，这就是**混合空间**的作用

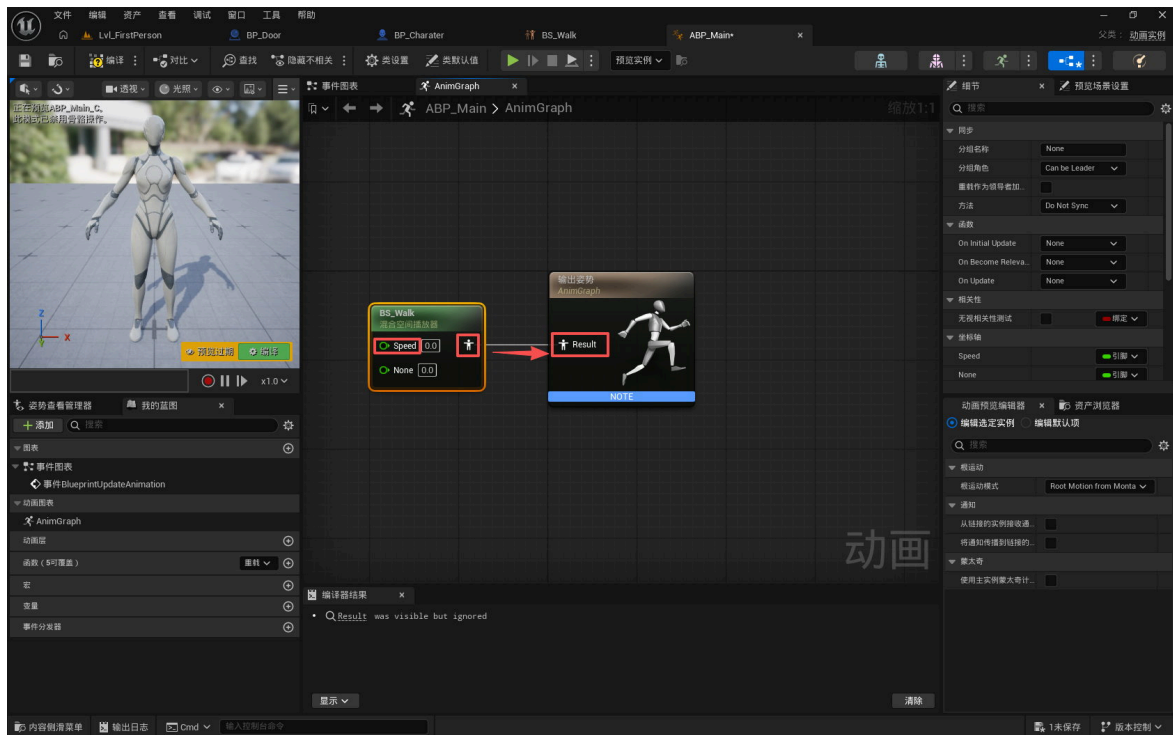


3. 编写动画蓝图

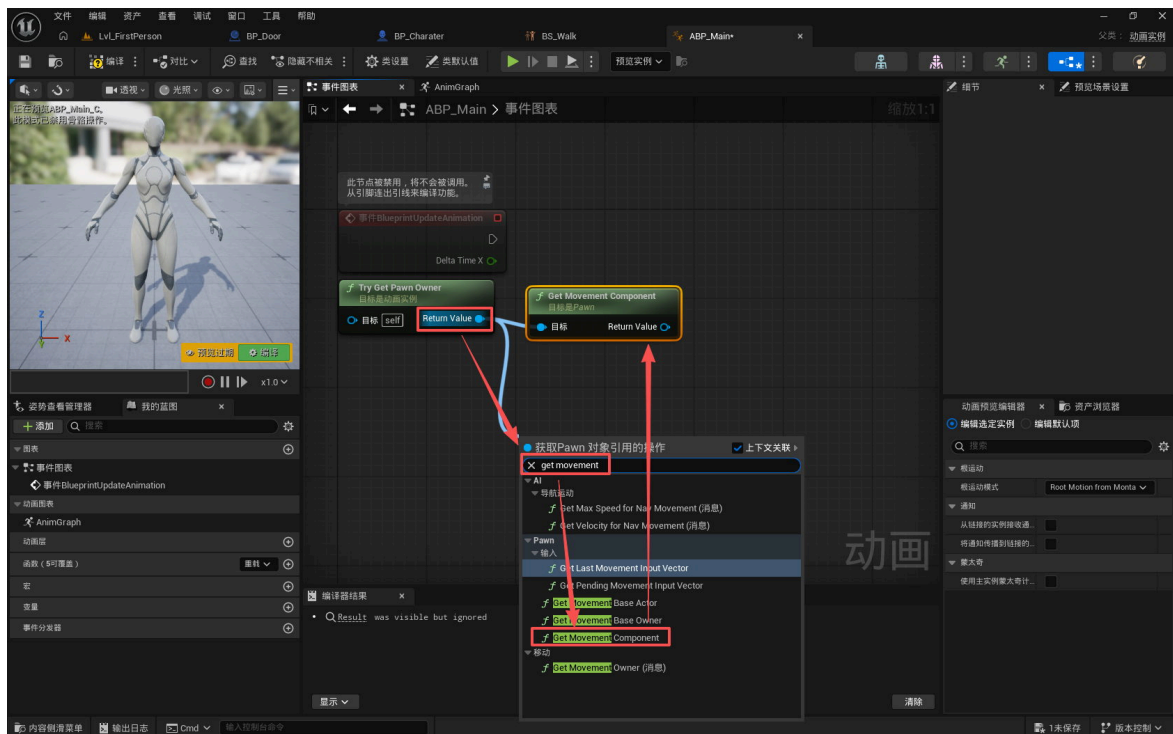
打开内容侧滑菜单，双击蓝图动画ABP_Main，进入AnimGraph，再次打开内容侧滑菜单，左键长按设置好的混合空间BS_Walk拖拽到界面内



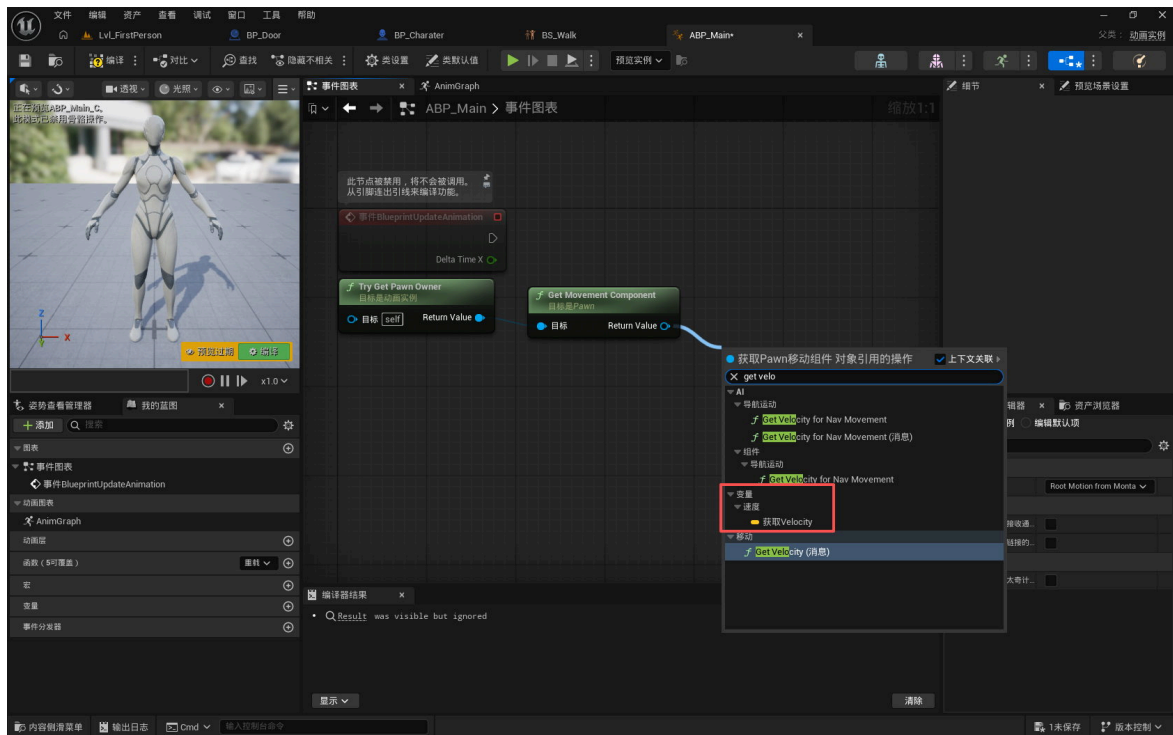
将混合空间的输出引脚连接到动画蓝图的执行引脚，混合动画的左侧Speed和None分别对应混合空间内水平坐标和垂直坐标的命名



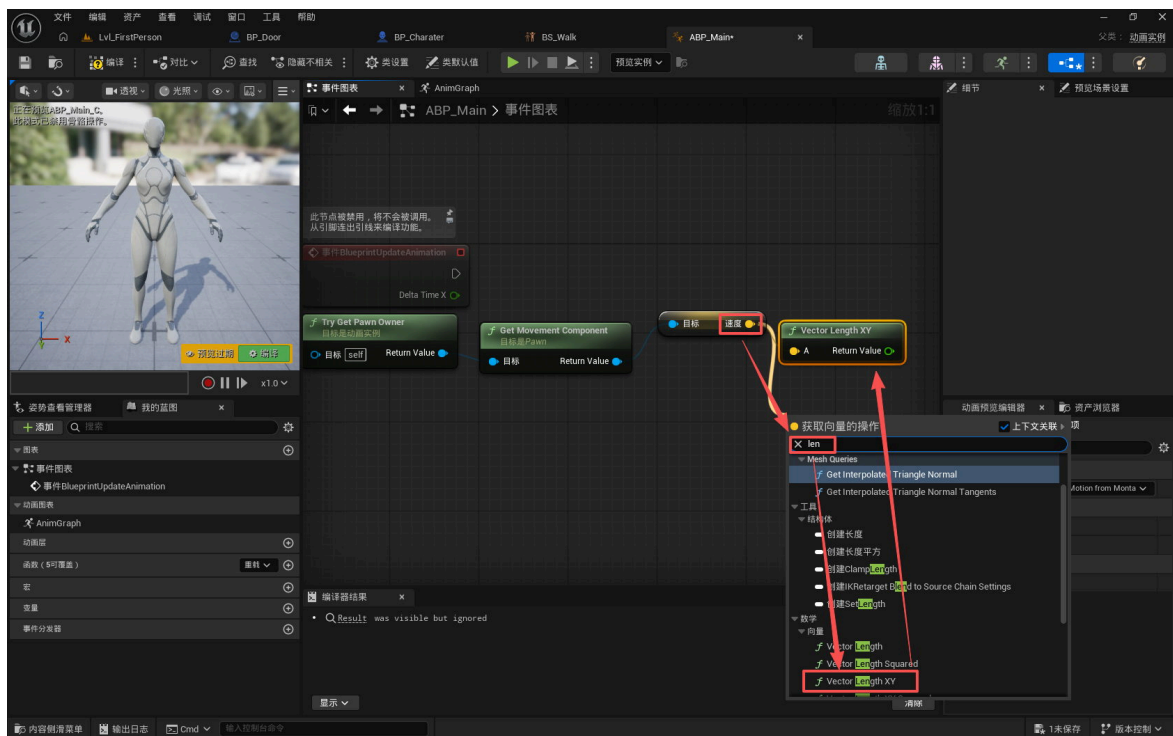
- 当角色速度为0时，应该播放站立动画，当角色达到一定速度时，应该播放奔跑动画，因此需要在混合空间的Speed左侧获取到角色的实时速度
- 点击进入动画蓝图ABP_Main的事件图表界面，左键长按尝试获取Pawn拥有者的Return Value引脚，搜索输入get movement，点击Get Movement Component，角色移动组件包含角色移动速度



左键长按Get Movement Component的Return Value引脚，搜索输入get velo，选择变量-速度-获取Velocity

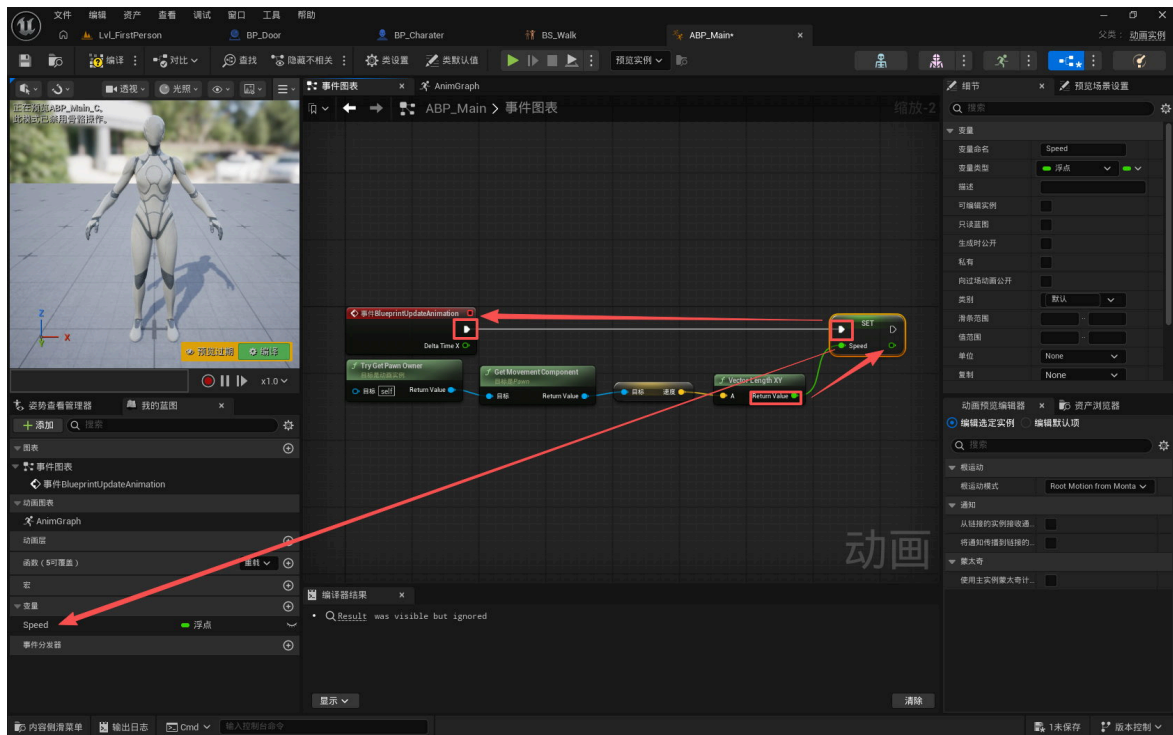


获取Velocity中的速度其实包含角色移动方向和速度，此处仅需速度，因此左键长按获取Velocity的速度引脚，搜索输入len，选择 **Vector Length XY**



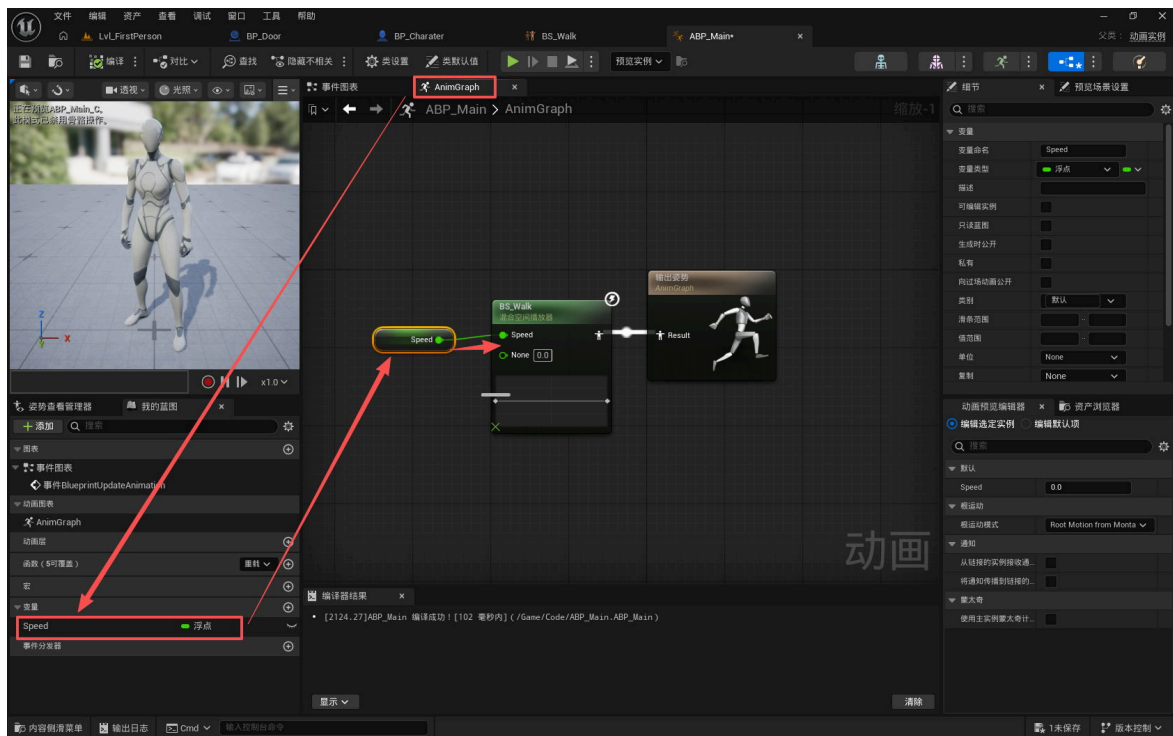
因为需要将实时速度传递到混合空间节点处，因此需要将向量长度XY进行保存，左键长按向量长度XY的Return Value，选择提升为变量

在左下角变量栏命名为**Speed**，将新增的变量左侧引脚连接到**事件蓝图更新动画**的右侧引脚

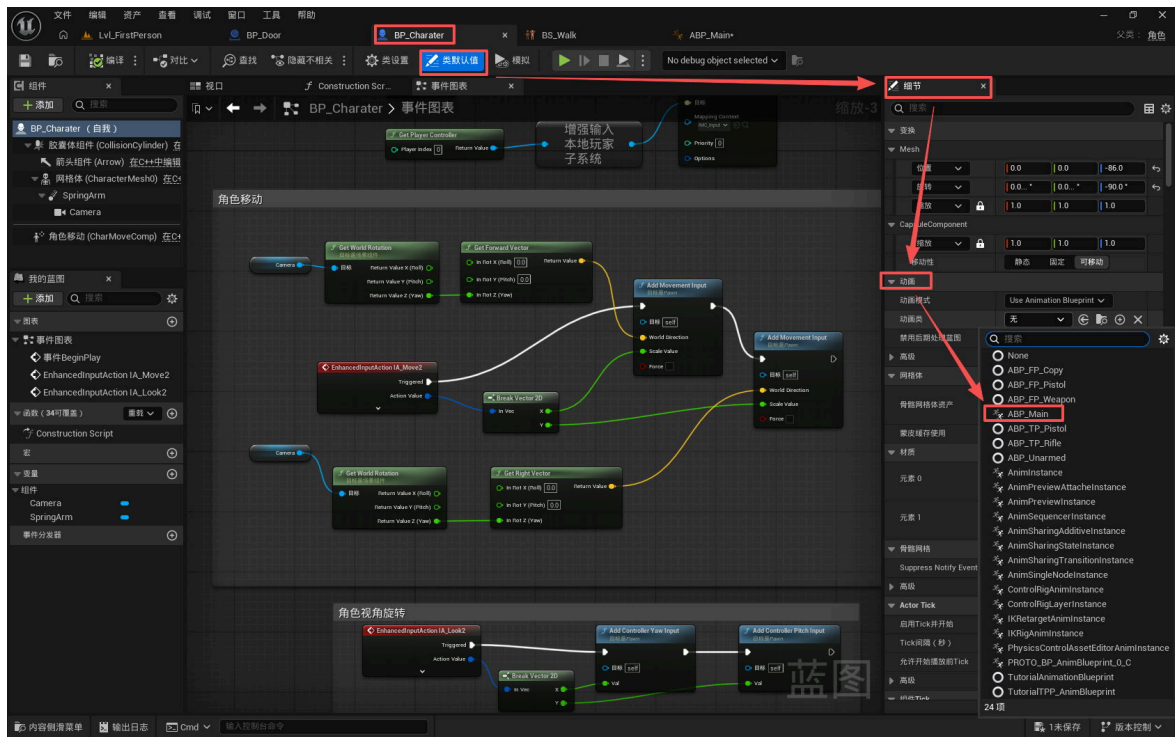


4. 应用角色动画

返回AnimGraph，左键长按变量栏的Speed拖拽到界面内，选择获取Speed，将变量Speed连接到BS_Walk的Speed引脚处，点击编译



返回BP_Character，点击类默认值，在右侧细节窗口的动画栏中选择动画模式为Use Animation Blueprint，动画类选择编写好的ABP_Main，点击编译



- 此时返回关卡界面会发现跑动和停止已经有动画，但是停止动作过于僵硬
- 返回BS_Walk页面，在资产详情窗口的水平坐标栏中，修改平滑时间为0.2或更大后回车，之后选择平滑类型为Cubic（立方），点击保存

